

멀티클라우드 자동화 기반 이상형 매칭 웹 서비스 구현

조원 : 문수정, 박수한, 최예람

목 차

1. 프로젝트 개요
2. 클라우드 아키텍처
3. AWS 인프라 구성 (Terraform)
4. GCP 인프라 구성 (Terraform)
5. 인프라 구성 (Kubernetes)
6. CI/CD 파이프라인
7. APP/ DB
8. 운영 및 모니터링
9. 기타 사항
10. 향후 개선 계획

1. 프로젝트 개요

1-1. 사업개요

최근 데이팅 앱 시장은 새로운 전환점을 맞이하고 있다. 대표적인 글로벌 데이팅 앱 “틴더”는 Z세대 사이에서 '흑업 앱'이라는 이미지가 고착화되며, 단순한 스와이프 방식의 만남에 대한 피로감이 확산되고 있다. 포브스 등 주요 매체 역시 Z세대가 더 이상 가벼운 만남을 추구하지 않으며, 가치관과 깊은 연결을 중요시한다는 점을 강조하고 있다.

이러한 트렌드 변화에 주목하여 사용자의 성향과 가치관을 중심으로 진정성 있는 관계 형성을 지원하는 국내 데이팅 웹 플랫폼을 기획하고자 한다. 본 플랫폼 "LoveBridge"는 성격, 라이프스타일, 가치관 등 개개인의 세부적인 특성을 중심으로 매칭하여, 사용자들이 실시간으로 접속하여 프로필을 탐색하거나 채팅을 통해 자연스럽게 소통할 수 있도록 설계된다.

LoveBridge는 Spring Boot 기반의 Java와 JSP를 사용하여 개발되며, 마이크로서비스 아키텍처를 활용하여 기능별 코드 분리를 통해 서버 관리 및 유지보수에 용이하도록 설계되었다. 각 서비스는 독립적인 모듈로 구성되어 있어 장애 발생 시 영향 범위를 최소화하고, 서비스별 독립적인 배포와 롤백을 가능하게 한다.

또한 AWS와 GCP를 활용한 멀티클라우드 전략을 통해 능동적인 재해복구(DR)를 구축하여, 한 클라우드 서비스에 장애가 발생하더라도 다른 클라우드 서비스에서 즉각적으로 서비스 연속성을 확보할 수 있다. 이는 각 클라우드 플랫폼의 강점을 최대한 활용하여, 높은 가용성과 안정성을 제공한다.

인프라 측면에서는 Kubernetes 클러스터 위에 각 기능별로 독립적인 파드를 구성하여 유연한 자원 관리와 수평 확장을 통해 사용자 증가에 따른 트래픽 대응이 가능하도록 하였다. 이러한 구조는 서비스 확장성과 비용 효율성을 극대화하며, 높은 서비스 품질을 유지하는 데 중요한 역할을 수행한다.

이번 프로젝트에서는 이러한 기술적 장점과 시장 트렌드를 결합하여, 변화하는 Z세대의 니즈를 공략하는 진정성 있는 데이팅 앱 "LoveBridge"를 제작하는 것을 목표로 한다.

1-2. 주요 서비스

- Application

기능	설명
회원 정보	개인 회원별 데이터 저장
프로필 등록/조회	사진/성격 등 정보가 포함된 프로필 카드 제공
실시간 채팅	websocket 기반
양방향 선호 고려 이상형 매칭	저장된 프로필 카드 기반으로 상대 매칭

애플리케이션에서는 위 표의 서비스 기능들을 구현하는 것으로 목표로 하고 있다.

1. 회원정보

회원 개개인의 기본 정보, 취향, 라이프스타일 등의 데이터를
안전하게 저장하고 관리한다.

2. 프로필 등록/조회

사용자의 사진, 성격(MBTI), 관심사, 생활 습관 등의 정보가 포함된 프로필 카드를
제공하며, 다른 사용자의 프로필 탐색과 매칭 가능 여부를 쉽게 확인할 수 있도록
지원한다.

3. 실시간 채팅

websocket을 기반으로 한 빠르고 안정적인 실시간 메시지 교환 시스템을
제공하여, 사용자 간 원활한 의사소통을 가능하게 한다.

4. 프로필 기반 이상형 매칭

사용자가 입력한 취향 및 가치관 프로필 데이터를 기반으로 유사 성향을 가진 사용자와의 매칭을 제공한다. 이를 통해 단순한 외모 중심 매칭을 넘어 더욱 의미 있고 진정성 있는 만남을 지향한다.

- Server Infra

기능	설명
인프라 전체 코드화(lac)	Terraform을 이용한 전체 코드 자동화
쿠버네티스 기반 컨테이너 구성	Docker에 이미지 저장
멀티 클라우드	aws와 gcp 멀티 클라우드
멀티 리전	서울 리전 / 미국 리전
CI/CD 자동화	Gitops+Argo cd
운영 및 모니터링	prometheus + grafana 통한 정밀 분석

1. 인프라 전체 코드화(lac)

Terraform을 이용하여 클라우드 인프라 구성요소(네트워크, 보안그룹, 데이터베이스, 서버 등)를 코드로 정의하고 관리하여, 인프라의 일관성을 유지하고 배포와 관리 자동화 및 효율성 향상을 지원한다.

2. 쿠버네티스 기반 컨테이너 구성

애플리케이션을 **Docker** 컨테이너 이미지로 패키징하고 **Docker Registry**에 이미지 저장 및 관리. 쿠버네티스를 사용하여 컨테이너의 배포, 스케일링,고가용성 보장, 자원 관리 및 오케스트레이션을 수행하여 애플리케이션의 안정성 및 확장성 향상한다.

3. 멀티 클라우드

AWS를 메인 운영 환경으로 활용하고, GCP를 재해 복구(DR, Disaster Recovery) 환경으로 설정하여, 장애 발생 시 GCP가 자동으로 활성화되어 서비스의 지속성 및 안정성 확보한다.

4. 멀티 리전

기본 서비스 운영은 AWS의 서울 리전에서 수행하며, 장애나 재해 발생 시 미국 리전의 GCP 인프라로 빠르게 페일오버(failover)를 수행하여 데이터 손실 최소화 및 서비스 연속성 보장한다.

5. CI/CD 자동화

GitOps 방식을 기반으로 소스 코드의 변경이 Git 저장소에 푸시되면 자동으로 Argo CD가 최신 애플리케이션 상태를 확인하여 쿠버네티스 클러스터로 변경 사항을 자동 반영하고 배포하는 시스템을 구축하여, 배포의 정확성, 투명성 및 속도 향상한다.

6. 운영 및 모니터링

사용자 활동 및 접속 트래픽을 분석하여 사용자의 선호도와 이용 패턴을 파악하는 사용자 트렌드 분석 시스템 구축. 이를 통해 시스템 운영 상태를 실시간으로 점검하고, 예측 가능한 문제를 사전에 감지하며, 사용자 경험(UX)을 지속적으로 개선하고 서비스 전략 수립을 지원한다.

1-3. 프로젝트 목표

1. 사용자 중심의 가치 기반 매칭 시스템

단순한 외모나 피상적인 조건이 아니라 사용자의 취향, 성격, 가치관 등의 심층적 요소를 기반으로 사용자 간의 깊은 연결과 공감을 이끌어낸다.

자체 개발된 알고리즘을 통해 사용자들이 원하는 만남의 형태와 방향성을 최적화하여, 만족도 높은 매칭 결과를 제공한다.

2. 안정적이고 유연한 멀티 클라우드 및 멀티 리전 인프라 구축

AWS 서울 리전을 주요 운영 환경으로 설정하고 GCP 미국 리전을 장애 복구 환경으로 활용하여, 장애 발생 시 데이터 손실을 최소화하고 서비스 연속성을 확보한다.

Terraform을 활용한 IaC(Infrastructure as Code)를 통해 인프라 전체를 코드화하여 관리의 효율성과 확장성을 극대화한다.

3. 쿠버네티스 기반의 고가용성 컨테이너 환경 구축

쿠버네티스 기반으로 애플리케이션을 컨테이너화하여 빠르고 안정적인 배포 및 운영을 지원하며, 운영 과정에서의 문제점 식별 및 해결 속도를 높인다.

트래픽 증가나 서비스 확장 요구에도 빠르게 대응할 수 있는 스케일링 능력을 확보하여 최적의 사용자 경험을 유지한다.

4. GitOps 및 Argo CD를 활용한 효율적인 CI/CD 환경 구성

GitOps 기반의 자동화된 배포 환경을 구축하여 소프트웨어 개발에서 배포까지의 프로세스를 명확히 하고 투명성을 높이며, 배포 주기를 단축하여 빠른 서비스 개선이 가능하도록 한다.

Argo CD를 통한 지속적인 배포 및 자동 롤백 기능을 도입하여 운영 효율성과 서비스 안정성을 강화한다.

5. 고도화된 서비스 운영 및 모니터링

prometheus와 Grafana를 활용하여 모니터링 시스템을 통한 애플리케이션 성능, 시스템 자원 사용량, 트래픽 등을 실시간으로 수집하고 시각화한다.

이를 기반으로 이상 징후를 신속히 탐지할 수 있고 서비스 장애를 사전에 예방하며 안정적인 운영을 지원할 수 있다. 모니터링 결과를 주기적으로 분석하여 서비스 운영 전반의 효율성을 높이는 데 집중한다.

6. 체계적인 클라우드 비용 절감과 운영 최적화

멀티 클라우드 구성 시 **AWS**와 **GCP** 간의 효율적 자원 배분과 장애 복구 전략을 통해 인프라 비용을 최소화한다.

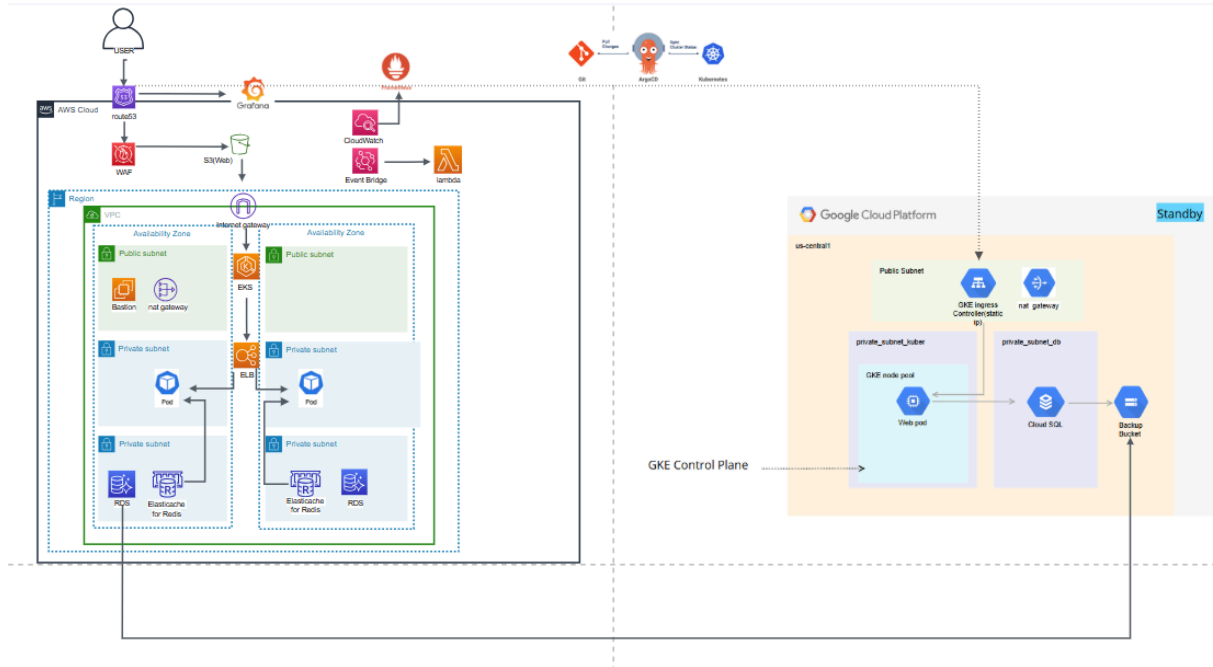
리소스의 자동화된 관리와 수요 예측을 통한 서버 리소스 최적화를 통해 운영 비용을 지속적으로 절감하고, 비용 효율적인 인프라 운영 체계를 구축한다.

또한 **DB** 역시 **Aurora RDS** 대신 일반 관계형 **DB**에 스토리지를 통한 덤프 전송을 이용해 멀티리전에서의 **DB**연동의 비용을 최대한 절감하고자 한다.

클라우드 비용 모니터링 도구를 활용하여 정기적으로 자원 소비를 분석하고 불필요한 리소스를 신속히 제거하여 비용 낭비를 최소화한다.

2. 클라우드 아키텍처

2-1. 전체 인프라 설계도



2-2. 인프라 구성 - AWS

1. 네트워크

 vpc	aws 내에서 정의하는 가상 네트워크다. VPC 구성을 통해서 IP 대역, 서브넷, 라우팅을 설정해서 격리된 네트워크 환경을 구성할 수 있다.
 subnet	VPC 내에서 IP대역을 논리적으로 분할한 네트워크 단위이다. 퍼블릭 또는 프라이빗으로 구분된다.
 Internet gateway	퍼블릭 서브넷에 위치한 리소스들이 인터넷과 통신할 수 있도록 해주는 통로역할을 한다.
 Nat instance	프라이빗 서브넷에 위치한 인스턴스가 인터넷에 접속할 수 있도록 중개 역할을 한다. Nat gateway를 사용하기도 하지만, 본 프로젝트에서는 비용절감을 위해서 nat instance를 사용했다. Nat instance 사용으로, 외부 접근은 차단되고 내부 리소스만 아웃바운드 트래픽을 허용한다.

2. 컴퓨팅 및 운영 리소스

 EC2	AWS에서 제공하는 가상 서버이다. 인스턴스 유형을 선택하여 애플리케이션, 웹서버 등을 실행할 수 있다. 유연한 컴퓨팅 환경을 제공한다.
 ALB	OSI 7계층에서 동작하는 로드밸런서로 HTTP/HTTPS 요청을 기반으로 트래픽을 분산시킨다.
 EKS	AWS에서 관리형 Kubernetes 클러스터를 제공하는 서비스이다. 컨테이너 기반 애플리케이션을 kubernetes 를 통해서 배포하고 운영할 수 있다.

3. 데이터 리소스

 S3	AWS에서 제공하는 개체 스토리지 서비스이다. 이미지, 동영상, 로그와 같이 다양한 데이터를 저장할 수 있다. 스토리지 유형이 다양해서 버전관리와 수명 주기 설정의 기능도 제공된다.
 RDS	MySQL, MariaDB, SQL Server등과 같은 관계형 데이터베이스를 AWS에서 관리형으로 제공하는 서비스이다.

4. 모니터링 및 보안 리소스

 ROUTE53	도메인 등록 및 DNS 라우팅 서비스이다. ALB 연결 또는 failover 구성시에 자주 사용이 된다.
 IAM	AWS 리소스에 대한 접근 권한을 제어할 수 있는 서비스이다. 사용자, 정책, 역할을 정의하여 인가를 세밀하게 관리할 수 있다.

3. AWS 인프라 구성 (Terraform)

0. 인프라 구조 개요

본 챕터에서는 Terraform을 통해 구축한 AWS 기반 인프라 구조에 대해 설명한다.

kubernetes 클러스터 내부 구성은 다음 장에서 별도로 기술하며 본 장에서는 vpc, 보안, rds, ingress, externaldns 와 같은 리소스 생성과 네트워크 구축을 중심으로 다룬다.

1. 네트워크

1-1. vpc

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.1.1"

  name = "lovebridge-vpc"
  cidr = "10.0.0.0/16"
  azs = ["ap-northeast-2a", "ap-northeast-2c"]

  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24", "10.0.21.0/24", "10.0.22.0/24"]

  manage_default_route_table = false
  create_igw = true
  enable_nat_gateway = false
  enable_dns_support = var.enable_dns_support
  enable_dns_hostnames = var.enable_dns_hostnames

  map_public_ip_on_launch = true

  public_subnet_tags = {
    "kubernetes.io/role/elb" = "1"
    "kubernetes.io/cluster/lovebridge-eks" = "shared"
  }

  tags = {
    Project = "lovebridge"
  }
}
```

본 프로젝트에서는 **terraform**을 사용하여 인프라를 코드로 정의하고 자동화하였다. 가상네트워크는 모듈을 사용하여 구현하였다. **10.0.0.0** 네트워크 대역으로 설정하였고, **ap-northeast-2a** 와 **ap-northeast-2c** 두 개의 가용영역을 활용하여고가용성을 확보하였다.

서브넷은 퍼블릭 서브넷과 프라이빗 서브넷으로 구성했다. 퍼블릭 서브넷의 경우, **map_public_ip_on_launch=true**로 설정해서 퍼블릭 IP가 자동으로 할당되도록 하였다. 프라이빗 서브넷은 **EKS** 노드나 **RDS**와 같이 외부와 직접 통신하지 않는 리소스를 배치하기 위한 서브넷으로 총 **4**개의 프라이빗 서브넷을 정의하였다.

create_igw=true 로 설정하여 퍼블릭 서브넷에서 외부 인터넷으로 트래픽 라우팅이 가능하도록 설정하였다.

enable_nat_gateway=false로 설정한 이유는 비용절감을 위해서 **nat instance**를 따로 정의하여 사용할 계획이었기에 **vpc** 구성에서는 **nat gateway**를 비활성화하였다.

1-2. NAT

```
resource "aws_eip" "nat_eip" {
  domain    = "vpc"
  instance  = aws_instance.nat.id
}

resource "aws_instance" "nat" {
  ami                  = "ami-01ad0c7a4930f0e43"
  instance_type        = var.instance_type
  subnet_id            = var.public_subnet_ids[0]
  vpc_security_group_ids = [var.nat_security_group_id]
  source_dest_check     = false
  associate_public_ip_address = false

  tags = {
    Name = "nat-instance"
  }
}

resource "aws_eip_association" "nat_eip_assoc" {
  instance_id    = aws_instance.nat.id
  allocation_id  = aws_eip.nat_eip.id
}
```

프라이빗 서브넷에 위치하는 리소스가 아웃바운드 통신을 할 수 있게 하기 위해서 NAT 인스턴스를 구성하였다. NAT Gateway 대신에 EC2 인스턴스를 NAT 역할로 사용함으로써 비용 절감에 중점을 두어 설계하였다.

NAT 생성 코드에서 `source_dest_check=false` 구문을 통해 ec2인스턴스가 라우팅 기능을 할 수 있도록 설정하였다. `aws_eip.nat_eip` 를 통해 elastic ip를 생성하고 NAT 인스턴스에 할당하였다. 그 후에 `aws_eip_association.nat_eip_assoc` 을 통해 위에서 생성해놓은 EIP를 NAT인스턴스에 연동하였다. 또한 NAT인스턴스가 퍼블릭 서브넷에 위치하지만 보안그룹을 적용하여 접근을 제어할 수 있도록 보안설정을 했다.

2. 보안설정

2-1. IAM 역할 및 정책

사용자 (4) 정보
IAM 사용자는 계정에서 AWS와 상호 작용하는 데 사용되는 장기 자격 증명을 가진 자격 증명입니다.

☐	사용자 이름	▲	경로	▼	그룹	▼	마지막 활동	▼	MFA
☐	admin		/		0		🟢 54분 전		-
☐	choiyeram		/		1		-		-
☐	moonsujeong		/		1		-		-
☐	parksuhan		/		1		-		-

본 프로젝트에서는 IAM을 통해 세가지 사용자 그룹에 대해 역할을 분리하고, 각 그룹에 권한 정책을 적용하였다. 세명의 사용자에게 적용한 정책은 **TerraformAdminPolicy**, **KubernetesDeveloperPolicy**, **MonitoringPolicy**이다. 각 정책에 대한 간단한 설명은 다음과 같다. **TerraformAdmin**의 경우에는 IAC관리 및 인프라를 설정하는 담당자에게 부여되는 것으로 **EC2** 또는 네트워킹 같은 리소스를 생성하거나 수정할 수 있는 권한을 포함한다. 그리고 **terraform**에서 생성이 이루어지는 **eks**, **rds** 등을 설정하거나 변경할 수 있는 권한도 가진다. 또한 리소스에 문제가 있는지 확인하기 위해 로그 및 지표를 조회할 수 있는 권한도 있다. 클라우드 인프라 전체 자원에 대한 생성, 삭제, 수정 권한을 보유한다.

두번째로, **kubernetesDeveloperPolicy**는 **kubernetes** 기반 개발 및 운영 담당자에게 할당되는 권한이다. **eks** 클러스터를 조회하거나 접근이 가능하며 **ecr** 이미지를 조회하거나 다운로드할 수 있는 권한을 가진다. 그리고 로그 그룹이나 메트릭 데이터를 조회할 수 있다. 이 정책의 경우 인프라 생성권한은 없고 **eks**에 제한된 읽기 권한만 부여한다.

마지막으로 **MonitoringPolicy**는 리소스 전반에 대한 조회 권한을 가진다. 주요 리소스의 **Describe**, **List**, **Get**권한을 가지고, **S3**읽기 전용 권한, 로그 그룹 조회권한을 가진다. 각 사용자는 사용자 그룹을 통해 정책이 할당되고, 이 권한은 최소권한 원칙에 따라 설계되었다.

3. 컴퓨팅 리소스

3-1. bastion

```
data "aws_ssm_parameter" "al2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-6.1-x86_64"
}

# Bastion EC2 인스턴스 생성
resource "aws_instance" "bastion" {
  ami                  = data.aws_ssm_parameter.al2023_ami.value
  instance_type        = "t3.micro"
  subnet_id            = var.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.bastion_sg.id]
  associate_public_ip_address = true
  key_name              = "ABC2"
  iam_instance_profile  = var.bastion_instance_profile_name

  user_data = file("${path.module}/userdata.sh")

  tags = {
    Name = "bastion-host"
  }
}
```

퍼블릭 서브넷에 위치하는 **bastion instance**를 생성하는 코드이다. RDS나 EKS노드와 같이 프라이빗 서브넷에 위치한 리소스에 직접 접근할 수 없기 때문에 **bastion host**를 구성하여 제한된 관리자만 접근하여 운영 및 유지보수를 할 수 있도록 설정하였다.

첨부한 이미지를 보면, **SSM Parameter Store**에서 관리되는 최신 **AMI**를 사용하여 **bastion** 역할을 수행하는 **EC2** 인스턴스를 생성했다. **bastion**은 퍼블릭 서브넷에 배치시켜서 외부 인터넷에 접근할 수 있도록 설정했다. **Key Pair**와 **instance profile**을 설정해서 **ec2** 접속시에 사용할 수 있는 **key**와 **bastion**에서 업로드를 위한 **IAM role**을 부여하였다.

```
resource "aws_security_group_rule" "allow_https_from_bastion" {
  type           = "ingress"
  from_port      = 443
  to_port        = 443
  protocol       = "tcp"
  security_group_id = module.eks0.cluster_security_group_id
  source_security_group_id = module.sg.bastion_sg_id
  description     = "Allow Bastion SG to access EKS API"
  depends_on      = [module.eks0]
}
```

이 코드는 Bastion 서버가 HTTPS를 통해 eks 클러스터에 접근할 수 있도록 허용하는 인바운드 규칙을 정의한 코드이다. `type="ingress"`로 인바운드 트래픽을 허용하도록 설정하고 `from_port`와 `to_port`를 443으로 설정함으로써 포트 443만 허용하도록 설정했다. 그리고 `security_group_id`를 EKS 클러스터의 보안그룹을 대상으로 하여 설정하였다. 이 설정으로 eks 클러스터의 보안그룹이 bastion 서버에서 오는 트래픽을 허용할 수 있게 된다.

또한 bastion instance는 프라이빗 서브넷에 있는 리소스에 접근하여 운영 및 유지보수를 할 때만 필요하기에 보안과 비용을 고려해서 업무시간에만 자동으로 실행되고 이후 자동 종료되도록 구성하였다. 이를 위해서 Lambda와 eventbridge를 이용하여 bastion 인스턴스 제어기능을 구현하였다.

간략한 흐름은 다음과 같다. Event Bridge 스케줄링 규칙이 동작하게 되면 lambda 함수가 실행이 된다. 그 결과로 EC2 인스턴스가 시작되거나 중지될 수 있다.

```

resource "aws_iam_role" "lambda_ec2_control" {
  name = "lambda-ec2-control-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Action = "sts:AssumeRole",
      Effect = "Allow",
      Principal = {
        Service = "lambda.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "lambda_ec2_policy" {
  role          = aws_iam_role.lambda_ec2_control.name
  policy_arn    = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

```

bastion 제어기능을 구현하는 코드 중 role에 관한 코드이다. Lambda가 ec2인스턴스를 제어할 수 있도록 하는 IAM Role을 생성한 후에, `aws_iam_role_policy_attachment.lambda_ec2_policy` 로 위에서 생성한 Role에 권한을 부여한다.

```

data "archive_file" "lambda_zip" {
  type      = "zip"
  source_file = "${path.module}/lambda.py"
  output_path = "${path.module}/lambda_bastion.zip"
}

```

루트에 위치하는 [lambda.py](#) 파일을 zip으로 패키징한다.

이 코드에서 사용하는 [lambda.py](#) 코드는 다음과 같다.

```

import boto3
import os

def lambda_handler(event, context):
    ec2 = boto3.client('ec2', region_name='ap-northeast-2')
    instance_id = os.environ['INSTANCE_ID']
    action = event.get('action')

    if action == 'start':
        ec2.start_instances(InstanceIds=[instance_id])
    elif action == 'stop':
        ec2.stop_instances(InstanceIds=[instance_id])
    else:
        return {'status': 'Invalid action'}

    return {'status': f'{action} executed'}

```

[lambda.py](#)에서는 boto3 라이브러리를 통해 EC2 클라이언트를 생성하고 전달받은 INSTANCE_ID를 통해서 START 또는 STOP메서드를 실행하게 된다.

```

resource "aws_lambda_function" "bastion_controller" {
    function_name = "bastion-controller"
    role          = aws_iam_role.lambda_ec2_control.arn
    handler       = "lambda.lambda_handler"
    runtime      = "python3.9"
    filename      = "lambda_bastion.zip"
    source_code_hash = data.archive_file.lambda_zip.output_base64sha256

    environment {
        variables = {
            INSTANCE_ID = module.bastion.bastion_instance_id
        }
    }
}

```

[lambda.py](#)를 zip으로 패키징 한 후에 lambda함수를 정의한다. 이 코드에서는 생성될 lambda함수의 이름, lambda가 사용할 IAM Role, 실행할 python함수의 위치, 실행환경,

해시값 등을 지정한다. 이 코드를 기반으로 lambda함수는 python3.9 환경에서 실행되며 사전에 패키징된 [lambda.py](#) 파일을 기반으로 배포된다.

```
resource "aws_lambda_permission" "allow_eventbridge_start" {
  statement_id = "AllowStart"
  action       = "lambda:InvokeFunction"
  function_name = aws_lambda_function.bastion_controller.function_name
  principal    = "events.amazonaws.com"
  source_arn    = aws_cloudwatch_event_rule.bastion_start.arn
}

resource "aws_lambda_permission" "allow_eventbridge_stop" {
  statement_id = "AllowStop"
  action       = "lambda:InvokeFunction"
  function_name = aws_lambda_function.bastion_controller.function_name
  principal    = "events.amazonaws.com"
  source_arn    = aws_cloudwatch_event_rule.bastion_stop.arn
}
```

이 코드는 EventBridge가 Lambda를 실행할 수 있도록 권한을 허용하는 코드이다.

```
resource "aws_cloudwatch_event_rule" "bastion_start" {
  name                = "bastion-start"
  schedule_expression = "cron(0 0 ? * MON-FRI *)" # KST 오전 9시 = UTC 0시
}

resource "aws_cloudwatch_event_rule" "bastion_stop" {
  name                = "bastion-stop"
  schedule_expression = "cron(0 9 ? * MON-FRI *)" # KST 오후 6시 = UTC 9시
}
```

이 부분은 크론식을 사용하여 업무시간인 월요일 ~ 금요일 오전 9시에 Bastion을 자동으로 시작하고 , 오후 6시에 Bastion을 자동 종료하도록 설정한 코드이다.

```

resource "aws_cloudwatch_event_target" "start_target" {
  rule      = aws_cloudwatch_event_rule.bastion_start.name
  target_id = "bastionStartLambda"
  arn       = aws_lambda_function.bastion_controller.arn
  input     = jsonencode({ "action" = "start" })
}

resource "aws_cloudwatch_event_target" "stop_target" {
  rule      = aws_cloudwatch_event_rule.bastion_stop.name
  target_id = "bastionStopLambda"
  arn       = aws_lambda_function.bastion_controller.arn
  input     = jsonencode({ "action" = "stop" })
}

```

위에서 정의한 스케줄 규칙은 `aws_cloudwatch_event_target`을 통해서 `lambda`함수 (`bastion-controller`)를 호출한다. `lambda`호출 시에는 `input`으로 “action”=“start” 또는 “action”=“stop”을 전달한다.

```

if action == 'start':
    ec2.start_instances(InstanceIds=[instance_id])
elif action == 'stop':
    ec2.stop_instances(InstanceIds=[instance_id])
else:
    return {'status': 'Invalid action'}

```

위 코드는 [lambda.py](#)의 일부분이다. 코드를 보면 알 수 있듯이, 전달 받은 값에 따라 `lambda`함수는 `start_instances` 또는 `stop_instance` 메서드를 실행하게 된다.

결론적으로 동작하게 될 시나리오는 다음과 같다. 월요일 오전 9시가 되면, `bastion-start` Event Bridge 규칙이 트리거가 되어 `lambda`를 호출하게 된다. 그때 `lambda`함수에 “action”: “start” 값을 넘겨주게 된다. 그러면 `lambda`함수에서 `start_instances` 메서드를 실행하게 되어 `bastion` EC2인스턴스가 부팅된다.

월요일 오후 6시가 되었을 때는, 규칙이 트리거가 되었을 때 “action”:”stop” 값을 lambda에게 넘겨주어서 lambda함수에서 stop_instances 메서드가 실행되어서 인스턴스가 종료되게 된다.

이 기능으로 수동으로 bastion을 끄지 않아도 되고, 제시간에 자동으로 꺼진다는 점에서 비용절감을 할 수 있다는 장점이 있다.

3-2. eks

```
resource "aws_iam_role" "eks_cluster_role" {
  name = "eks-cluster-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Action = "sts:AssumeRole",
        Effect = "Allow",
        Principal = {
          Service = "eks.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  role          = aws_iam_role.eks_cluster_role.name
  policy_arn    = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}
```

```

resource "aws_iam_role_policy_attachment" "eks_vpc_resource_controller_policy" {
  role       = aws_iam_role.eks_cluster_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSVPCResourceController"
}

resource "aws_eks_cluster" "main" {
  name     = "lovebridge-eks"
  role_arn = aws_iam_role.eks_cluster_role.arn

  vpc_config {
    subnet_ids         = [var.private_subnets[0], var.private_subnets[1]]
    endpoint_private_access = true
    endpoint_public_access = true
    security_group_ids  = [var.bastion_sg_id]
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy,
    aws_iam_role_policy_attachment.eks_vpc_resource_controller_policy
  ]
}

```

- cluster.tf

이 cluster.tf 파일은 AWS EKS(Elastic Kubernetes Service) 클러스터를 Terraform으로 프로비저닝하기 위한 주요 리소스들을 정의한다. 해당 구성은 IAM Role 생성, 클러스터 생성, 인증 데이터 출력을 포함하며, EKS 클러스터가 프라이빗 서브넷 내에서 동작하면서도 퍼블릭 접근을 허용하도록 설정되어 있다.

구성 요소

1. IAM Role 생성 (aws_iam_role.eks_cluster_role)

- eks-cluster-role이라는 이름으로 IAM Role을 생성한다.
- eks.amazonaws.com 서비스가 해당 Role을 가정(assume)할 수 있도록 sts:AssumeRole 정책을 설정한다.
- 이 Role은 EKS 클러스터가 AWS 리소스에 접근할 때 사용된다.

2. IAM 정책 연결

- AmazonEKSClusterPolicy: EKS 클러스터 운영을 위한 기본 정책.
- AmazonEKSVPCResourceController: VPC 내 리소스 관리(ENI 등)를 위한 정책.
- 이 두 정책을 eks-cluster-role에 연결하여 클러스터 관리와 네트워크 자원 할당을 가능하게 한다.

3. EKS 클러스터 생성 (aws_eks_cluster.main)

- lovebridge-eks라는 이름으로 클러스터를 생성한다.
- IAM Role(eks-cluster-role)을 클러스터에 연결하여 AWS 리소스 접근을 허용한다.

4. VPC 네트워크 설정:

- 프라이빗 서브넷 2개(var.private_subnets[0], var.private_subnets[1])를 사용한다.
- 퍼블릭 및 프라이빗 API 엔드포인트 접근을 모두 허용(endpoint_public_access = true, endpoint_private_access = true).
- 보안그룹(var.bastion_sg_id)을 지정하여 접근 제어를 강화한다.

종속성 관리: IAM Role과 정책 연결이 완료된 후에만 클러스터를 생성하도록 depends_on으로 순서를 보장한다.

4. 클러스터 인증 데이터 (`data.aws_eks_cluster_auth.this`)

- 클러스터 인증을 위한 토큰을 추출한다.
- 이 데이터는 **Kubernetes provider** 또는 **Helm provider**에서 클러스터 인증 시 사용된다.

특징 및 사용 목적

- 이 구성은 **EKS** 클러스터를 프라이빗 네트워크 기반으로 운영하면서도, 관리 편의를 위해 퍼블릭 엔드포인트를 열어 **kubectl** 및 **Helm** 접근을 허용한다.
- 클러스터는 **IAM Role** 기반으로 **AWS** 리소스를 제어하며, 보안그룹을 통해 외부 접근을 관리한다.
- 추출된 인증 토큰(`data.aws_eks_cluster_auth.this`)은 추가 모듈(**Helm** 차트 설치, 애플리케이션 배포 등)에서 재사용된다.

```

resource "aws_security_group" "eks_cluster_sg" {
  name           = "eks-cluster-sg"
  description    = "Security group for EKS cluster endpoint"
  vpc_id        = var.vpc_id

  ingress {
    description = "Allow Bastion SG to access EKS API"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    security_groups = [var.bastion_sg_id] # Bastion 보안그룹 ID를 변수로 전달
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "eks-cluster-sg"
  }
}

```

- eks_sg.tf

이 eks_sg.tf 파일은 AWS EKS(Elastic Kubernetes Service) 클러스터의 보안 그룹(Security Group)을 정의하여, 클러스터 API 서버와 외부 리소스 간의 네트워크 접근을 제어한다. 해당 보안 그룹은 Bastion 호스트와의 안전한 통신을 보장하고, 클러스터 노드들이 인터넷에 접근할 수 있도록 아웃바운드 트래픽을 허용한다.

구성 요소

1. 보안 그룹 생성 (aws_security_group.eks_cluster_sg)

- eks-cluster-sg라는 이름으로 보안 그룹을 생성한다.
- 설명(description)을 통해 EKS 클러스터 API 엔드포인트 보호용 보안 그룹임을 명시한다.

- `vpc_id`는 변수를 통해 지정된 VPC(`var.vpc_id`)에 연결되어, 해당 네트워크 내에서만 적용된다.

2. 인바운드 규칙 (Ingress)

- **Bastion** 호스트만 클러스터 API 서버(443 포트, HTTPS)에 접근할 수 있도록 허용한다.
- `security_groups = [var.bastion_sg_id]`로 Bastion의 보안 그룹 ID를 변수로 받아, Bastion에서만 안전하게 `kubectl` 등으로 API 서버에 접근할 수 있게 한다.
- 이를 통해 외부 인터넷에서 직접 접근을 차단하고, **Bastion**을 통한 보안된 관리 경로만 허용한다.

3. 아웃바운드 규칙 (Egress)

- 모든 아웃바운드 트래픽(0.0.0.0/0, 모든 포트, 모든 프로토콜)을 허용한다.
- 클러스터 노드들이 인터넷 리포지토리(Docker Hub, ECR 등)와 통신하거나 업데이트 및 외부 서비스와의 연결이 가능하도록 한다.

4. 태그 (Tags)

- `Name = "eks-cluster-sg"`로 태그를 지정하여 AWS 콘솔에서 쉽게 식별할 수 있도록 한다.
- 태그는 운영 시 리소스 관리, 비용 추적, 모니터링 등에 유용하게 활용된다.

특징 및 사용 목적

- 이 보안 그룹은 **EKS** 클러스터 **API** 서버의 보안을 강화하면서도, **Bastion** 호스트를 통해 필요한 관리 작업을 수행할 수 있게 한다.
- 인바운드 규칙은 최소한으로 제한(**Bastion** 호스트만 허용)하고, 아웃바운드 트래픽은 전면 허용하여 클러스터 노드가 외부 리소스와 자유롭게 통신할 수 있도록 설계되었다.
- 이 보안 그룹은 `cluster.tf`의 `aws_eks_cluster.main`에서 `security_group_ids` 변수로 참조되어, 클러스터와 직접 연결된다.

```

resource "aws_iam_role" "eks_node_role" {
  name = "eks-node-role3"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
        Action = "sts:AssumeRole"
      }
    ]
  })
}

```

```

resource "aws_iam_role_policy_attachment" "worker_node_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "cni_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

resource "aws_iam_role_policy_attachment" "registry_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

```

```

resource "aws_eks_node_group" "web_nodes" {
  cluster_name = aws_eks_cluster.main.name

  node_group_name = "web-node-group"
  node_role_arn   = aws_iam_role.eks_node_role.arn

  subnet_ids = [var.private_subnets[0], var.private_subnets[1]]

  instance_types = ["t3.medium"]

  scaling_config {
    desired_size = 2
    max_size     = 3
    min_size     = 1
  }

  tags = {
    Name = "web-node"
  }
}

```

```

depends_on = [
  aws_eks_cluster.main,
  aws_iam_role_policy_attachment.worker_node_policy,
  aws_iam_role_policy_attachment.cni_policy,
  aws_iam_role_policy_attachment.registry_policy
]
}

```

- nodegroup.tf

이 nodegroup.tf 파일은 AWS EKS(Elastic Kubernetes Service) 환경에서 **노드 그룹(Node Group)**을 프로비저닝하여, 클러스터에서 애플리케이션을 실행할 워커 노드(EC2 인스턴스)를 배치한다.

IAM Role과 정책 설정을 통해 EKS와 연동하고, 프라이빗 서브넷에 노드를 배치해 보안성을 높이면서도 확장 가능한 오토스케일링 구성을 지원한다.

구성 요소

1. IAM Role 생성 (aws_iam_role.eks_node_role)

- eks-node-role3라는 이름으로 IAM Role을 생성한다.
- ec2.amazonaws.com 서비스가 이 Role을 가정할 수 있도록 sts:AssumeRole 정책을 설정한다.
- 이 Role은 워커 노드(EC2)가 AWS 서비스와 상호작용할 수 있도록 권한을 부여하는 핵심 요소다.

2. IAM 정책 연결

- AmazonEKSWorkerNodePolicy: EKS 워커 노드가 클러스터와 상호작용할 수 있는 권한을 제공.
- AmazonEKS_CNI_Policy: VPC CNI 플러그인을 통해 Pod 네트워킹을 관리할 수 있는 권한을 제공.
- AmazonEC2ContainerRegistryReadOnly: ECR(Amazon Elastic Container Registry)에서 컨테이너 이미지를 가져올 수 있는 권한을 부여.
- 이 세 가지 정책을 eks-node-role3에 연결하여 Pod 실행, 네트워크 관리, 이미지 배포를 가능하게 한다.

3. EKS 노드 그룹 생성 (aws_eks_node_group.web_nodes)

- web-node-group이라는 이름으로 EKS 노드 그룹을 생성한다.

- 상위 클러스터(`aws_eks_cluster.main`)에 연결되어, 해당 클러스터의 일부로 워커 노드들이 동작한다.
- 노드 Role(`eks-node-role3`)을 연결하여 EKS 및 AWS 리소스 접근이 가능하도록 한다.

4. 네트워크 설정

- 노드들은 `var.private_subnets[0]`, `var.private_subnets[1]`으로 지정된 프라이빗 서브넷에 배치된다.
- 외부와의 직접적인 노출을 피하고, NAT 인스턴스 또는 Bastion 호스트를 통해서만 외부 연결을 허용함으로써 보안성을 강화한다.

5. 인스턴스 유형

- `t3.medium` 인스턴스를 사용하여 워커 노드를 구성한다.
- 이 인스턴스는 EKS 운영 시 적절한 성능과 비용 효율성을 제공한다.

6. 오토스케일링 설정 (`scaling_config`)

- 최소 1개(`min_size`), 최대 3개(`max_size`), 기본 2개(`desired_size`)로 노드 개수를 유연하게 조정할 수 있도록 설정.
- 클러스터의 부하에 따라 자동 확장/축소(`Auto Scaling`)가 가능하다.

7. 태그 (`Tags`)

- Name = "web-node" 태그를 지정하여, AWS 콘솔에서 리소스 식별과 비용 추적을 용이하게 한다.

8. 종속성 관리 (depends_on)

- 노드 그룹은 EKS 클러스터(`aws_eks_cluster.main`)와 IAM 정책 연결이 완료된 이후에 생성되도록 `depends_on`으로 리소스 생성 순서를 보장한다.

특징 및 사용 목적

- 이 구성은 EKS 클러스터의 워커 노드 그룹을 프라이빗 서브넷에서 안전하게 운영하면서, 오토스케일링을 통한 유연한 확장성을 제공한다.
- IAM Role과 필수 정책을 연결하여, Pod 실행, 네트워크 관리, 이미지 배포 등 EKS 운영에 필요한 모든 권한을 갖춘 노드를 구성한다.
- 인스턴스 유형과 스케일링 설정을 통해 비용 효율성과 가용성을 동시에 만족하도록 설계되었다.
- 이 노드 그룹은 애플리케이션 배포를 위한 실제 실행 환경(Compute Layer)으로 사용되며, 클러스터의 워크로드를 처리한다.

3-2-1. ecr read only 정책 정의 (terraform)

```
resource "aws_iam_policy" "ecr_read_only" {
  name       = "ECRReadOnlyPolicyForEKS"
  description = "Policy for allowing EKS to pull images from ECR"
  policy     = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Action = [
          "ecr:GetAuthorizationToken",
          "ecr:BatchCheckLayerAvailability",
          "ecr:GetDownloadUrlForLayer",
          "ecr:BatchGetImage"
        ],
        Resource = "*"
      }
    ]
  })
}
```

EKS 클러스터의 pod가 AWS의 ECR에서 컨테이너 이미지를 pull 하기 위해 필요한 권한을 부여하는 정책이다. 위 코드에서는 다음과 같은 권한을 포함하고 있다. ECR 인증 토큰을 요청하고, 이미지 레이어 존재여부를 확인 후에 이미지 레이어를 다운로드 한다. 그리고 ECR로부터 이미지를 가져오는 권한을 포함한다. 이렇게 정의해놓은 정책은 이후 IRSA 통해서 EKS POD에 부여된다.

3-2-2. OIDC 연동 및 Role 생성 (terraform)

```
resource "aws_iam_openid_connect_provider" "oidc" {
  url = var.oidc_url
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = ["9e99a48a9960b14926bb7f3b02e22da0afd10df6"]
}
```

IRSA를 사용하기 위해 필요한 OIDC 설정을 하는 부분이다. EKS는 OIDC를 통해 AWS IAM Role을 연동하여 pod 내부에서 aws 리소스에 안전하게 접근할 수 있다.

```

resource "aws_iam_role" "ecr_role" {
  name = "eks-ecr-access-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Principal = {
          Federated = aws_iam_openid_connect_provider.oidc.arn
        },
        Action = "sts:AssumeRoleWithWebIdentity",
        Condition = {
          StringEquals = {
            "${replace(var.oidc_url, "https://", "")}:sub" = "system:serviceaccount:default:ecr-sa"
          }
        }
      }
    ]
  })
}

```

EKS의 ServiceAccount에 부여할 IAM Role인 `ecr_role` 을 정의하는 코드이다. 여기에서 생성한 Role은 위에서 설정한 OIDC Provider를 통해 연동되고, 코드에 명시되어있는 것처럼 `default` 네임스페이스의 `ecr-sa`라는 이름의 ServiceAccount를 사용하는 pod에만 해당 역할이 부여되도록 설정했다.

```

resource "aws_iam_role_policy_attachment" "ecr_policy_attach" {
  role          = aws_iam_role.ecr_role.name
  policy_arn    = aws_iam_policy.ecr_read_only.arn
}

```

마지막으로, 앞서 생성한 Role에 ECR ReadOnly정책을 연결해서 권한을 부여하는 코드이다. 이 코드를 마지막으로, 이 Role을 사용하는 Pod는 ECR에서 컨테이너 이미지를 가져올 수 있다.

3-2-3. ServiceAccount와 role연동 (kubernetes)

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ecr-sa
  namespace: default
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::680993828418:role/eks-ecr-access-role
```

Kubernetes에서는 pod가 앞에서 terraform으로 생성한 eks-ecr-access-role을 assume할 수 있도록 연결해주는 ServiceAccount를 정의한다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lovebridge-basic
  labels:
    app: lovebridge
spec:
  replicas: 1
  selector:
    matchLabels:
      app: lovebridge
  template:
    metadata:
      labels:
        app: lovebridge
    spec:
      serviceAccountName: ecr-sa
      containers:
        - name: lovebridge
          image: 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin:latest
          ports:
            - containerPort: 8080
```

이 코드는 pod를 생성하는 deployment 코드이다. 이후 pod의 spec.serviceAccountName에 ecr-sa를 지정하여 pod가 실행되면 eks는 이 SA와 연결된 IAM Role을 assume할 수 있도록 처리하게 된다.

4. 데이터베이스 및 스토리지

4-1. /modules/db/ 데이터베이스

```
2
3  ✓ resource "aws_db_subnet_group" "rds_subnet_group" {
4      name      = "rds-subnet-group"
5      ✓ subnet_ids = [
6          module.vpc.private_subnets[2],
7          module.vpc.private_subnets[3]
8      ]
9
10     ✓ tags = {
11         Name = "RDS subnet group"
12     }
13 }
14
15  ✓ resource "aws_security_group" "rds_sg" {
16      name      = "rds-sg"
17      description = "Allow EKS to access RDS"
18      vpc_id = module.vpc.vpc_id
19
20     ✓ ingress {
21         description      = "Allow MySQL from EKS"
22         from_port        = 3306
23         to_port          = 3306
24         protocol         = "tcp"
25         cidr_blocks      = ["10.0.0.0/16"]
26     }
27
28     ✓ egress {
29         from_port        = 0
30         to_port          = 0
31         protocol         = "-1"
32         cidr_blocks      = ["0.0.0.0/0"]
33     }
34
35     ✓ tags = {
36         Name = "rds-sg"
37     }
```

본 구성은 Terraform을 사용하여 AWS RDS 인스턴스를 기존에 생성해 둔 스냅샷으로부터 복원하는 방식으로 정의되어 있다. 복원 대상은 MariaDB이며, 해당 인스턴스는 ap-northeast-2a 가용 영역(AZ)에 배치된다. 또한 VPC 내 EKS 클러스터가 접근할 수 있도록 필요한 서브넷 그룹과 보안 그룹이 정의되어 있다. 먼저, aws_db_subnet_group 리소스를 통해 RDS 인스턴스가 위치할 서브넷 그룹을 생성한다. 이 그룹은 VPC의 프라이빗 서브넷 중 (module.vpc.private_subnets[2], [3])를 포함하며, Name 태그를 통해 "RDS subnet group"이라는 명칭이 부여된다. 이는 RDS가 여러 서브넷 중 하나에 자동으로 할당될 수 있도록 하기 위한 구성이다.

다음으로, `aws_security_group` 리소스를 통해 RDS용 보안 그룹을 생성한다. 이 보안 그룹의 이름은 `"rds-sg"`이며, 주 목적은 EKS 클러스터에서 DB로 접근을 허용하는 것이다.

인바운드 규칙에서는 MySQL의 기본 포트인 3306 포트를 tcp 프로토콜로 허용하고 있으며, CIDR 블록 `"10.0.0.0/16"`에 대해 열려 있다.

이는 VPC 내 EKS 노드 IP 범위에 해당한다.

아웃바운드 규칙은 모든 트래픽(`0.0.0.0/0`)에 대해 전송을 허용한다.

```
41 resource "aws_db_instance" "mariadb_2a" {
42     identifier            = "mariadb-2a"
43     snapshot_identifier   = "lovebridge-db-backup"
44     instance_class        = "db.t3.micro"
45     engine                = "mariadb"
46     engine_version        = "11.4.5"
47     port                  = 3306
48
49     availability_zone      = "ap-northeast-2a"
50     multi_az               = false
51     publicly_accessible   = false
52     deletion_protection   = false
53     skip_final_snapshot   = true
54
55
56
57     db_subnet_group_name   = aws_db_subnet_group.rds_subnet_group.name
58     vpc_security_group_ids = [aws_security_group.rds_sg.id]
59     backup_retention_period = 7
60     apply_immediately      = true
61     tags = {
62         Name          = "EKS-RDS-MariaDB-2a"
63         Environment   = "prod"
64     }
65 }
66
```

43줄 부터는 `lovebridge-db-backup`이라는 기존 RDS 스냅샷을 기반으로 `ap-northeast-2a` 가용영역에 MariaDB 인스턴스(`mariadb-2a`)를 복원 생성하는 리소스를 정의한 것이다.

인스턴스 유형은 `db.t3.micro`, 엔진 버전은 `11.4.5`이며, VPC 내 특정 프라이빗 서브넷 그룹 및 보안 그룹과 연동되도록 설정되어 있다. 외부에서 접근할 수 없도록

`publicly_accessible`은 `false`이며, 다중 가용영역 배포도 비활성화되어 있다. 삭제 시 최종 스냅샷을 건너뛰도록 구성되었고, 백업은 7일간 보존되며 변경 사항은 즉시 반영된다.

"EKS-RDS-MariaDB-2a"라는 이름과 "prod" 환경 태그가 지정되어 운영 환경에서 사용되는 DB 인스턴스로 식별된다.

```
66
67  resource "aws_db_instance" "mariadb_replica_2c" {
68      identifier          = "lovebridge-db-copy"
69      replicate_source_db = aws_db_instance.mariadb_2a.arn
70      instance_class      = "db.t3.micro"
71      engine              = "mariadb"
72      availability_zone   = "ap-northeast-2c"
73      publicly_accessible = false
74      auto_minor_version_upgrade = true
75      apply_immediately   = true
76      skip_final_snapshot = true
77
78      db_subnet_group_name = aws_db_subnet_group.rds_subnet_group.name
79      vpc_security_group_ids = [aws_security_group.rds_sg.id]
80
81      tags = {
82          Name          = "RDS-ReadReplica-2c"
83          Environment = "prod"
84          Role          = "read-replica"
85      }
86  }
87
88
```

69줄부터는 mariadb-2a 기본 DB 인스턴스의 장애 또는 성능 저하 상황에 대비하여, 이를 복제한 ****Read Replica(mariadb_replica_2c)****를 ap-northeast-2c 가용영역에 생성하도록 정의한 설정이다. 복제 대상은 mariadb-2a 인스턴스이며, 이를 지정할 때 반드시 .arn을 사용해야 한다. 복제 인스턴스는 db.t3.micro 타입의 MariaDB 엔진(11.4.5 생략됨)을 사용하며, 외부 노출은 금지(publicly_accessible = false)되어 있다. 자동 마이너 버전 업그레이드는 활성화되었으며(true), 장애 발생 시 빠르게 대체 가능한 구조를 위해 apply_immediately = true 옵션이 설정되어 있다. 이 Read Replica는 기존의 RDS 서브넷 그룹과 보안 그룹을 공유하며, 태그에는 환경(prod) 및 역할(read-replica) 정보가 포함되어 운영용 리드 복제본으로 식별된다.


```

91 resource "aws_elasticache_subnet_group" "redis_subnet_group" {
92     name = "redis-subnet-group"
93     subnet_ids = [
94         module.vpc.private_subnets[2],
95         module.vpc.private_subnets[3]
96     ]
97
98     tags = {
99         Name = "redis-subnet-group"
100     }
101 }
102
103 # Redis용 Security Group
104 resource "aws_security_group" "redis_sg" {
105     name = "redis-sg"
106     vpc_id = module.vpc.vpc_id
107
108     ingress {
109         from_port = 6379
110         to_port = 6379
111         protocol = "tcp"
112         cidr_blocks = ["10.0.0.0/16"]
113     }
114
115     egress {
116         from_port = 0
117         to_port = 0
118         protocol = "-1"
119         cidr_blocks = ["0.0.0.0/0"]
120     }
121
122     tags = {
123         Name = "redis-sg"
124     }
125 }
126

```

93줄부터는 AWS ElastiCache용 Redis 클러스터를 구성하기 위한 서브넷 그룹 및 보안 그룹 설정을 포함하고 있다. 먼저, `aws_elasticache_subnet_group` 리소스를 통해 Redis가 배포될 프라이빗 서브넷 그룹을 정의하고 있으며, 이는 VPC 내 2번과 3번 서브넷을 지정하여 외부 노출 없이 클러스터를 내부에 안전하게 배치할 수 있도록 구성된다.

이와 함께 정의된 `aws_security_group` 리소스인 `redis_sg`는 Redis 인스턴스에 접근 가능한 보안 그룹을 설정하는 역할을 한다. 인바운드 규칙에서는 Redis 기본 포트인 6379번에 대해 TCP 프로토콜로 10.0.0.0/16 CIDR 범위(즉, EKS 노드가 포함된 서브넷)에서 접근을 허용하며, 이는 EKS에서 Redis에 접근이 가능하도록 하기 위한 설정이다. 아웃바운드

규칙은 모든 포트 및 모든 IP 대역에 대해 허용되어 있으며, 태그를 통해 **redis-sg**로 명시되어 리소스를 식별할 수 있도록 한다. 이 보안 그룹은 이후 **Redis** 클러스터 생성 시 연동되어 안전한 네트워크 트래픽 제어를 가능하게 한다.

```
127 # Redis Replication Group (멀티 AZ, 자동 Failover 포함)
128 resource "aws_elasticache_replication_group" "redis" {
129     replication_group_id      = "lovebridge-redis"
130     description                = "LoveBridge Redis Cluster"
131     engine                    = "redis"
132     engine_version            = "7.1"
133     node_type                  = "cache.t3.micro"
134     automatic_failover_enabled = true
135     multi_az_enabled          = true
136     transit_encryption_enabled = true
137
138     num_node_groups      = 1
139     replicas_per_node_group = 1
140
141     preferred_cache_cluster_azs = ["ap-northeast-2a", "ap-northeast-2c"]
142
143     subnet_group_name = aws_elasticache_subnet_group.redis_subnet_group.name
144     security_group_ids = [aws_security_group.redis_sg.id]
145
146     tags = {
147         Name = "redis-replication"
148     }
149 }
150
151 # Primary & Reader 엔드포인트 주소
152 output "redis_primary_endpoint" {
153     value = aws_elasticache_replication_group.redis.primary_endpoint_address
154 }
155
156 output "redis_reader_endpoint" {
157     value = aws_elasticache_replication_group.redis.reader_endpoint_address
158 }
159 }
```

130줄부터는 AWS ElastiCache for Redis의 Replication Group을 생성하는 구성으로, 멀티 AZ 및 자동 Failover 기능을 포함한고가용성 Redis 클러스터를 정의하고 있다. replication_group_id는 lovebridge-redis로 설정되어 있으며, 클러스터 이름은 LoveBridge Redis Cluster로 명시되었다. Redis 엔진 버전은 7.1이고, 인스턴스 타입은 소규모 테스트에 적합한 cache.t3.micro를 사용한다. 클러스터는 단일 노드 그룹(num_node_groups = 1)에 대해 1개의 리플리카(replicas_per_node_group = 1)를 갖도록 설정되어 있으며, 이를 통해 자동 Failover가 가능하다. 실제 배포 시에는 서울 리전 내 2개 가용영역(ap-northeast-2a, ap-northeast-2c)을 preferred AZ로 지정하여 장애 발생 시 가용성을 높인다. 보안을

위해 데이터 전송 시 암호화가 활성화(`transit_encryption_enabled = true`)되어 있고, 지정된 서브넷 그룹과 보안 그룹을 참조하여 클러스터를 네트워크에 연결한다.

추가로, `tags`를 통해 해당 리소스를 `redis-replication`으로 태깅하고 있으며, Terraform 출력값으로는 `Primary` 엔드포인트와 `Reader` 엔드포인트 주소를 각각 `redis_primary_endpoint`와 `redis_reader_endpoint`로 제공하여 외부 애플리케이션이 접근할 수 있도록 설정되어 있다. 이 구성을 통해 안정성과 확장성을 고려한 Redis 인프라를 IaC 방식으로 선언적으로 구축할 수 있다.

helm과 elasticache의 비교

초기 개발 및 테스트 단계에서는 Bitnami Helm Chart를 활용하여 Redis를 EKS 내부에 배포하여서 확인해보았다. 그런데 이 구성은 캐시 테스트 목적에 부합하고 데이터 영속은 보장되지 않기 때문에 운영 단계에서는 AWS ElastiCache for Redis를 사용하여 구성하였다. 먼저 Helm Chart를 사용하여 Redis를 EKS 클러스터 내부에 배포한 구성은 다음과 같다.

```
data "aws_eks_cluster" "eks0" {
  name = module.eks0.cluster_name
}

data "aws_eks_cluster_auth" "eks0" {
  name = module.eks0.cluster_name
}

resource "helm_release" "redis" {
  provider      = helm.eks # 이미 선언된 provider 사용
  name          = "redis"
  repository    = "https://charts.bitnami.com/bitnami"
  chart         = "redis"
  namespace     = "default"

  set = [
    {
      name  = "architecture"
      value = "standalone"
    },
    {
      name  = "auth.enabled"
      value = "false"
    }
  ]
}
```

```
    },
    {
      name = "master.persistence.enabled"
      value = "false"
    }
  ]

  depends_on = [module.eks0]
}
```

이 코드는 Terraform을 사용하여 Bitnami Redis Helm Chart를 배포하는 코드이다. `architecture=satndalone` 은 단일 Redis 인스턴스로 구성하는 설정이고, `auth.enabled=false` 는 내부 테스트 용도로 간단히 하기 위해서 인증을 비활성화하였다. 그리고 `master.persistence.enabled=false` 는 Redis를 디스크에 영구 저장하지 않고 메모리 기반으로 처리하기 위한 설정이다. 이러한 구성으로 redis 기반 캐싱을 테스트하였다. 이후 운영환경에서는 안정적인 AWS ElastiCache Redis로 전환하였다.

	Helm Redis	AWS ElastiCache Redis
위치	클러스터 내부	Aws 운영 Redis 서비스
고가용성	단일 pod	Multi-AZ, replica 지원
데이터 유지	설정 필요	기본적으로 영속성 지원
환경	테스트, 개발 환경/ 내부 전용 캐시	실제 서비스, 고가용성이나 장애복구가 중요한 환경

위 표는 Helm으로 배포한 redis와E ElastiCache Redis를 비교한 표이다.

4-2. 스토리지

```
modules > s3_helm_repo > main.tf
1  resource "aws_s3_bucket" "helm_repo" {
2      bucket          = var.bucket_name
3      force_destroy   = true
4      tags = {
5          Name = "helm-repo-private"
6      }
7  }
8
9  resource "aws_s3_bucket_website_configuration" "helm_repo" {
10     bucket = aws_s3_bucket.helm_repo.id
11
12     index_document {
13         suffix = "index.yaml"
14     }
15
16     error_document {
17         key = "error.html"
18     }
19 }
20
21 # 버킷 ACL 및 퍼블릭 액세스 차단 (중요)
22 resource "aws_s3_bucket_public_access_block" "helm_repo" {
23     bucket = aws_s3_bucket.helm_repo.id
24
25     block_public_acls       = true
26     block_public_policy     = true
27     ignore_public_acls     = true
28     restrict_public_buckets = true
29 }
30
```

Helm chart 저장소로 활용할 비공개 S3 버킷을 생성하고, 이를 웹사이트 형태로 호스팅할 수 있도록 설정한 구성이다.

우선 `aws_s3_bucket` 리소스를 통해 Helm chart 파일을 저장할 S3 버킷(`helm_repo`)을 생성하며, `force_destroy = true`를 설정하여 버킷 삭제 시 내부 객체도 함께 삭제되도록 구성한다.

이후 `aws_s3_bucket_website_configuration`을 사용하여 웹사이트 호스팅 설정을 적용하고, index 문서는 `index.yaml`, 오류 문서는 `error.html`로 지정한다.

또한 보안을 위해 `aws_s3_bucket_public_access_block` 리소스로 퍼블릭 액세스를 전면 차단한다.

구체적으로는 퍼블릭 ACL과 퍼블릭 정책을 모두 차단하고(`block_public_acls`, `block_public_policy`), 퍼블릭 ACL을 무시하며(`ignore_public_acls`), 퍼블릭 버킷으로의 접근을 제한(`restrict_public_buckets`)하여 Helm 저장소가 외부에 노출되지 않도록 보호한다.

결과적으로 이 구성은 **Helm chart** 파일을 내부적으로 안전하게 보관하고, 필요 시 웹 호스팅 방식으로 접근 가능한 프라이빗 **Helm** 저장소를 구축하는 데 목적이 있다.

S3- 이미지 버킷

범용 버킷모든 AWS 리전디렉터리 버킷

범용 버킷 (1) 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

이름으로 버킷 찾기

< 1 > ⚙

이름

▲

AWS 리전

▼

생성 날짜

▼

☐

lovebridgeimage

아시아 태평양(서울) ap-northeast-2

2025. 7. 8. pm 2:16:49 PM KST

버킷 태그를 사용하여 스토리지 비용을 추적하고 버킷을 구성할 수 있습니다. 자세히 알아보기

키

값

이 리소스와 연결된 태그 없음

기본 암호화 정보

편집

서버 측 암호화는 이 버킷에 저장된 새 객체에 자동으로 적용됩니다.

암호화 유형 정보

Amazon S3 관리형 키(SSE-S3)를 사용한 서버 측 암호화

버킷 키

KMS 암호화가 이 버킷의 새 객체를 암호화하는 데 사용되는 경우 버킷 키는 AWS KMS에 대한 호출을 줄여 암호화 비용을 줄입니다. 자세히 알아보기

활성화됨

지능형 계층화 아카이브 구성 (1)

세부 정보 보기편집삭제구성 생성

지능형 계층화 스토리지 클래스에 저장된 객체를 활성화하여 아카이브 액세스 계층이나 딥 아카이브 액세스 계층으로 하향됩니다. 이들 액세스 계층은 장기간 거의 액세스되지 않는 객체에 최적화되어 있습니다. 자세히 알아보기

지능형 계층화 아카이브 구성 찾기

이름

▲

상태

범위

아카이브 액세스 계층으로 전환하기...

딥 아카이브 액세스 계층으로 전환...

☐

lovebridge-Archive

Enabled

점두사: s3

90

-

모든 아카이브 계층 보기

S3 이미지 버킷의 경우에는 버킷을 삭제하거나 수정할 일이 적다고 판단하여 **Terraform** 코드를 통한 관리가 아닌 수동 생성을 진행하였다. 타입은 아카이브로 갱신된지 오래된 이미지는 아카이브 저장소로 들어가 비용을 절감하고자 하였다.

5. 도메인 및 트래픽 제어

5-1. ALB

본 프로젝트에서는 **AWS Load Balancer Controller**를 활용하여 **Ingress** 리소스를 정의하면 **AWS**에서 **ALB**가 자동으로 생성되도록 구성하였다. **Terraform**에서 **EKS** 클러스터의 **OIDC** 공급자 정보를 불러와서 **IAM Role** 및 정책을 설정하고 해당 **IAM Role**을 사용하는 **sa**를 생성한다. 이후 **Helm provider**를 사용하여 **AWS LoadBalancer Controller**를 배포하고 **ingress** 리소스 또한 **Terraform**을 통해 정의한다. 이를 통해 **terraform apply** 한 번으로 **IAM** 연결, **ALB Controller**설치, **ingress** 생성, **ALB** 프로비저닝까지 자동화된 로드밸런서 구성이 가능해진다.

5-1-1. alb-ingress-policy

alb ingress controller를 위한 정책을 **json**파일로 구성했다. 이는 **EKS** 클러스터에 **ALB**를 자동으로 생성하고 관리할 수 있게 해주는 컨트롤러가 **irsa**를 통해서 사용하게 될 권한을 정의해놓은 목록이다. 이 정책은 **alb controller**가 **Terraform**으로 정의된 **Ingress** 리소스를 기반으로 **ALB**를 생성하고 관리할 수 있도록 한다. **vpc**, 서브넷, 보안그룹 등의 리소스를 제어하고 **load balancer** 및 **Listener**를 생성 또는 수정, 삭제 할 수 있는 권한을 포함하고 있다.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iam:CreateServiceLinkedRole"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:AWSServiceName": "elasticloadbalancing.amazonaws.com"
        }
      }
    }
  ],
}
```

본 기술문서에는 정책 파일 내용을 주요 권한 그룹의 기능별로 나누어 설명하겠다.

위 코드는 **alb** 컨트롤러가 **alb**리소스를 관리하기 위해 필요한 **ServiceLinked Role** 생성 권한을 허용하겠다는 것을 의미한다.

```
{
  "Effect": "Allow",
  "Action": [
    "ec2:DescribeAccountAttributes",
    "ec2:DescribeAddresses",
    "ec2:DescribeAvailabilityZones",
    "ec2:DescribeInternetGateways",
    "ec2:DescribeVpcs",
    "ec2:DescribeVpcPeeringConnections",
    "ec2:DescribeSubnets",
    "ec2:DescribeSecurityGroups",
    "ec2:DescribeInstances",
    "ec2:DescribeNetworkInterfaces",
    "ec2:DescribeTags",

```

ec2 및 네트워크 정보를 조회하는 권한을 의미한다. 이를 허용하면 **ALB**와 관련된 **EC2**리소스 상태를 조회할 수 있는 권한을 가질 수 있다. 이 권한은 로드밸런서 대상그룹을 구성하거나 인터페이스 연결 등에 필요하다.


```
"elasticloadbalancing:DescribeLoadBalancers",
"elasticloadbalancing:DescribeLoadBalancerAttributes",
"elasticloadbalancing:DescribeListeners",
"elasticloadbalancing:DescribeListenerCertificates",
"elasticloadbalancing:DescribeSSLPolicies",
"elasticloadbalancing:DescribeRules",
"elasticloadbalancing:DescribeTargetGroups",
"elasticloadbalancing:DescribeTargetGroupAttributes",
"elasticloadbalancing:DescribeTargetHealth",
"elasticloadbalancing:DescribeTags",
"elasticloadbalancing:DescribeTrustStores",
"elasticloadbalancing:DescribeListenerAttributes",
"elasticloadbalancing:DescribeCapacityReservation"
```

기존의 ALB, Target Group, Listener 등에 대한 정보를 조회하는 권한이다. 로드밸런서 상태를 확인하고 라우팅 규칙을 파악하는 작업에 필요하다.

```
{
  "Effect": "Allow",
  "Action": [
    "cognito-idp:DescribeUserPoolClient",
    "acm:ListCertificates",
    "acm:DescribeCertificate",
    "iam:ListServerCertificates",
    "iam:GetServerCertificate",
    "waf-regional:GetWebACL",
    "waf-regional:GetWebACLForResource",
    "waf-regional:AssociateWebACL",
    "waf-regional:DisassociateWebACL",
    "wafv2:GetWebACL",
    "wafv2:GetWebACLForResource",
    "wafv2:AssociateWebACL",
    "wafv2:DisassociateWebACL",
    "shield:GetSubscriptionState",
    "shield:DescribeProtection",
    "shield:CreateProtection",
    "shield>DeleteProtection"
  ],
  "Resource": "*"
}
```

WAF ,ACM, Cognito, Shield에 관한 통합 권한이다. ALB와 연동되는 보안 서비스 리소스에 접근할 수 있는 권한이다.

```
{
  "Effect": "Allow",
  "Action": [
    "ec2:AuthorizeSecurityGroupIngress",
    "ec2:RevokeSecurityGroupIngress"
  ],
  "Resource": "*"
},
{
  "Effect": "Allow",
  "Action": [
    "ec2:CreateSecurityGroup"
  ],
  "Resource": "*"
}
```

보안그룹과 관련된 권한이다. ALB가 자동으로 보안그룹을 생성하고 Ingress 규칙을 생성할 수 있도록 허용하는 권한이다.

```

{
  "Effect": "Allow",
  "Action": [
    "elasticloadbalancing:AddTags",
    "elasticloadbalancing:RemoveTags"
  ],
  "Resource": [
    "arn:aws:elasticloadbalancing:*:*:targetgroup/*/*",
    "arn:aws:elasticloadbalancing:*:*:loadbalancer/net/*/*",
    "arn:aws:elasticloadbalancing:*:*:loadbalancer/app/*/*"
  ],
  "Condition": {
    "Null": {
      "aws:RequestTag/elbv2.k8s.aws/cluster": "true",
      "aws:ResourceTag/elbv2.k8s.aws/cluster": "false"
    }
  }
},
{
  "Effect": "Allow",
  "Action": [
    "elasticloadbalancing:AddTags",
    "elasticloadbalancing:RemoveTags"
  ],

```

태그 기반 리소스를 제어할 수 있는 권한이다. ALB 및 Target Group에 태그를 붙여서 클러스터 단위 리소스 관리가 가능하도록 한다.

```

{
  "Effect": "Allow",
  "Action": [
    "elasticloadbalancing:RegisterTargets",
    "elasticloadbalancing:DeregisterTargets"
  ],
  "Resource": "arn:aws:elasticloadbalancing:*:*:targetgroup/*/*"
},
{
  "Effect": "Allow",
  "Action": [
    "elasticloadbalancing:SetWebAcl",
    "elasticloadbalancing:ModifyListener",
    "elasticloadbalancing:AddListenerCertificates",
    "elasticloadbalancing:RemoveListenerCertificates",
    "elasticloadbalancing:ModifyRule"
  ],
  "Resource": "*"
}

```

Target 등록 및 트래픽 라우팅을 제어하는 권한이다. pod를 대상 그룹에 등록하거나 해제하고, ingress 규칙에 따라서 라우팅을 설정할 수 있다.

5-1-2. alb-irsa(Terraform)

AWS Load Balancer Controller는 EKS 내부에서 실행되는 ingress controller이다. AWS 리소스를 생성하거나 관리하기 위해서는 IAM 권한이 필요하다. 그래서 IRSA(IAM Roles for Service Accounts)를 사용하여 컨트롤러가 ServiceAccount를 통해서 IAM Role을 assume 할 수 있도록 구성한 코드가 alb-irsa 코드이다.

IRSA는 Kubernetes의 serviceaccount와 AWS IAM Role을 OIDC를 통해 연동하는 방식이다. 이를 통해 액세스 키 없이도 aws 리소스 접근이 가능하다.

다음은 코드 구성에 대해 기술하겠다.

```
data "aws_eks_cluster" "cluster" {
  name = var.cluster_name
}

data "aws_eks_cluster_auth" "cluster" {
  name = var.cluster_name
}

data "aws_iam_openid_connect_provider" "oidc" {
  url = data.aws_eks_cluster.cluster.identity[0].oidc[0].issuer
}
```

이 코드는 OIDC Provider URL을 가져오기 위한 준비 단계이다.

data.aws_eks_cluster.cluster.identity[0].oidc[0].issuer를 통해 OIDC URL을 추출한다.

```
resource "aws_iam_policy" "alb_controller_policy" {
  name     = "AWSLoadBalancerControllerIAMPolicy"
  path     = "/"
  policy   = file("${path.module}/alb_ingress_iam_policy.json")
}
```

앞에 5-1-1에서 설명한 alb_ingress_iam_policy.json파일을 적용하는 코드이다. 정책 파일을 불러와서 IAM 정책 리소스로 등록한다. 정책파일에는 앞서 설명한 것과 같이 ALB 컨트롤러가 EC2, ELB 등의 리소스를 조작할 수 있도록 하는 권한 목록이 포함되어있다.

```
data "aws_iam_policy_document" "alb_assume_role_policy" {
  statement {
    effect = "Allow"

    principals {
      type       = "Federated"
      identifiers = [data.aws_iam_openid_connect_provider.oidc.arn]
    }

    actions = ["sts:AssumeRoleWithWebIdentity"]

    condition {
      test     = "StringEquals"
      variable = "${replace(data.aws_iam_openid_connect_provider.oidc.url, "https://", "")}:sub"
      values   = ["system:serviceaccount:kube-system:aws-load-balancer-controller"]
    }
  }
}

resource "aws_iam_role" "alb_irs_role" {
  name               = "alb-irs-role"
  assume_role_policy = data.aws_iam_policy_document.alb_assume_role_policy.json
}
```

ALB Controller가 사용할 IAM Role을 생성하는 코드이다. OIDC Provider를 federated principal로 명시하고 특정 SA에서만 Role을 assume할 수 있도록 제한한다. sub 조건을 넣어서 system:serviceaccount:kube-system:aws-load-balancer-controller만 권한을 가질 수 있게 설정했다.

```
resource "kubernetes_service_account" "alb_sa" {
  metadata {
    name      = "aws-load-balancer-controller"
    namespace = "kube-system"
    annotations = {
      "eks.amazonaws.com/role-arn" = aws_iam_role.alb_irsa_role.arn
    }
  }
  depends_on = [aws_iam_role_policy_attachment.alb_policy_attach]
}
```

kube-system 네임스페이스에 aws-load-balancer-controller라는 SA를 생성하고, 이 Service Account가 위에서 생성한 Role을 assume할 수 있도록 연결하는 코드이다. 이 코드를 실행하고 나면, SA를 사용할 때 자동으로 IAM Role이 적용된다.

5-1-3. Helm으로 Load Balancer Controller배포

```
provider "helm" {
  alias = "eks"
  kubernetes = {
    host                = module.eks0.cluster_endpoint
    cluster_ca_certificate = base64decode(module.eks0.cluster_certificate_authority_data)
    token               = data.aws_eks_cluster_auth.eks0.token
  }
}
```

Terraform에서 Helm provider 사용하여 alb controller를 eks 클러스터에 설치하는 작업을 자동화한 코드이다. 이 부분은 Terraform에서 Helm provider를 설정하고, 앞에서 생성한 module.eks0에서 클러스터 정보를 불러온다.

```

resource "helm_release" "alb_controller" {
  provider      = helm.eks
  name          = "aws-load-balancer-controller"
  repository    = "https://aws.github.io/eks-charts"
  chart         = "aws-load-balancer-controller"
  namespace     = "kube-system"

  values = [
    yamlencode({
      clusterName = var.cluster_name
      region      = var.region
      vpcId       = local.vpc_id
      extraArgs = {
        "subnet-ids" = join(",", local.public_subnet_ids)
      }
      serviceAccount = {
        create = false
        name   = "aws-load-balancer-controller"
      }
      image = {
        repository = "602401143452.dkr.ecr.ap-northeast-2.amazonaws.com/amazon/aws-load-balancer-controller"
      }
    })
  ]

  depends_on = [module.eks0, module.iam_role]
}

```

Helm repository에서 가져온 ALB Controller Chart를 kube-system 네임스페이스에 설치한다.
Helm으로 배포되는 이름은 aws-load-balancer-controller로 정의했다.

그리고 values를 통해 Helm chart에 넘겨줄 설정 값을 정의한다.

ServiceAccount.create=false로 설정한 이유는 앞에서 생성한 SA를 사용할 계획이기에
여기에서는 sa를 생성하지 않는 것으로 설정했다. subnet-ids의 경우 public subnet으로
명시하여 alb가 public 서브넷에 생성되도록 설정하였다. image.repository의 경우 aws
ecr에서 제공하는 alb controller 이미지로 지정 하였다. 이 코드는 eks 클러스터와 IAM
Role이 생성되어야 실행되도록 의존성을 명시해두었다.

5-1-4. Ingress (Terraform)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: lovebridge-ingress
  namespace: default
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:ap-northeast-2:680993828418:certificate/9f19d5d4-1ed0-47b7-b3d6-b1ac1f880d39
    alb.ingress.kubernetes.io/ssl-policy: ELBSecurityPolicy-2016-08
    external-dns.alpha.kubernetes.io/hostname: lovebridge.click
spec:
  ingressClassName: alb
  rules:
    - host: lovebridge.click
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: lovebridge-service
                port:
                  number: 80
```

이 리소스 정의는 다음과 같은 설정을 포함한다. **internet-facing**으로 설정함으로써 퍼블릭 **ALB**가 생성되도록 구성하였다. 그리고 **80**포트와 **443**포트를 동시에 리스닝하도록 설정하였다. **HTTPS** 트래픽 처리를 위해서 **ACM** 인증서를 명시했다.

또한 [external-dns.alpha.kubernetes.io/hostname](#) 어노테이션을 통해 **ExternalDNS**연동을 지원한다. 이 설정으로 5-2에서 생성되는 도메인에 대한 **A**레코드가 **Route53**에 자동으로 생성된다.

이렇게 **terraform**을 사용하여 **alb** 자동화를 구성하였다. 정리하자면, **Terraform**으로 **IAM** 및 **OIDC** 연동, **Helm**을 사용한 **controller** 설치, **irsa** 적용, **Ingress** 리소스 정의, 도메인 연결까지 모든 과정을 코드로 관리하여 일관된 방식으로 자동화 할 수 있다..

5-2. External DNS

퍼블릭 lovebridge.click 정보

영역 삭제레코드 테스트쿼리 로깅 구성

▼ 호스팅 영역 세부 정보

호스팅 영역 편집

호스팅 영역 이름 lovebridge.click	쿼리 로그 -	이름 서버 ns-537.awsdns-03.net ns-1556.awsdns-02.co.uk ns-152.awsdns-19.com ns-1422.awsdns-49.org
호스팅 영역 ID Z057064919GL8WQUETYS	유형 퍼블릭 호스팅 영역	
설명 HostedZone created by Route53 Registrar	레코드 수 4	

도메인 lovebridge.click은 aws 콘솔에서 수동으로 등록하였다. 이를 ingress리소스와 연동하여 route53에서 자동으로 A레코드를 생성하거나 갱신하기 위해서 external-dns를 사용했다. dns 구성은 Terraform을 통해 관리하며 AWS IAM과 Kubernetes 간의 연동은 IRSA 방식으로 처리된다. External DNS 리소스는 아래와 같은 순서로 구성된다.

5-2-1. IAM Role 및 IRSA 구성 (Terraform)

가장 먼저 Terraform을 활용하여 IAM Role, 정책, OIDC 연동을 포함한 IRSA 구성을 자동화하였다.

```
data "aws_iam_openid_connect_provider" "eks0" {
  url = module.eks0.oidc_url
}
```

Terraform에서 EKS 모듈의 oidc_url을 불러와서 identity 설정에 활용한다.

```
data "aws_iam_policy_document" "external_dns_assume_role_policy" {
  statement {
    effect = "Allow"

    principals {
      type       = "Federated"
      identifiers = [data.aws_iam_openid_connect_provider.eks0.arn]
    }

    actions = ["sts:AssumeRoleWithWebIdentity"]

    condition {
      test     = "StringEquals"
      variable = "${replace(module.eks0.oidc_url, "https://", "")}:sub"
      values   = ["system:serviceaccount:kube-system:external-dns"]
    }
  }
}
```

external-dns ServiceAccount에서만 IAM Role을 assume할 수 있도록 IRSA 조건을 구성하였다.

```
resource "aws_iam_role" "external_dns" {
  name               = "AWSEKSRoute53Role"
  assume_role_policy = data.aws_iam_policy_document.external_dns_assume_role_policy.json
}

# external-dns가 사용할 Policy (Route53 접근 권한)
resource "aws_iam_policy" "external_dns" {
  name = "ExternalDNSPolicy-v2"

  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect = "Allow",
      Action = [
        "route53:ChangeResourceRecordSets",
        "route53:ListHostedZones",
        "route53:ListResourceRecordSets"
      ],
      Resource = "*"
    }]
  })
}
```

위에서 작성한 Assume Role Policy를 기반으로 AWSEKSRoute53Role 이라는 이름의 IAM Role을 생성했다. 그리고 이 역할은 앞에서 정의한 OIDC 기반 신뢰정책을 통해서 Kubernetes ServiceAccount에 연결된다.

그리고 아래 External DNS Policy-v2로 정의된 정책은 Route53에서 DNS 레코드를 등록하거나 조회할 수 있는 권한을 명시한 것이다.

```
resource "aws_iam_role_policy_attachment" "external_dns_attach" {  
  role      = aws_iam_role.external_dns.name  
  policy_arn = aws_iam_policy.external_dns.arn  
}
```

위에 정의한 IAM 정책을 Role에 부착해서 External DNS가 Route53에 접근해서 레코드를 생성, 조회 또는 삭제할 수 있도록 했다.

5-2-2. ServiceAccount 및 External-DNS 배포 정의 (Terraform)

1) ServiceAccount 정의

```
# 1. ServiceAccount  
resource "kubernetes_service_account" "external_dns" {  
  metadata {  
    name      = "external-dns"  
    namespace = "kube-system"  
    labels = {  
      "app.kubernetes.io/name" = "external-dns"  
    }  
    annotations = {  
      "eks.amazonaws.com/role-arn" = aws_iam_role.external_dns.arn  
    }  
  }  
}
```

ExternalDNS Pod가 사용할 Kubernetes ServiceAccount를 생성하는 코드이다. 위에서 생성한 IAM Role(external_dns)을 annotation에 추가하여 IAM Role과 연결한다. 해당 Role은 Route53레코드를 조작할 수 있는 권한을 가지고 있기에 이 연결을 함으로써 aws 리소스에 접근할 수 있는 권한을 갖게 된다.

2) ClusterRole

```
# 2. ClusterRole
resource "kubernetes_cluster_role" "external_dns" {
  metadata {
    name = "external-dns"
    labels = {
      "app.kubernetes.io/name" = "external-dns"
    }
  }

  rule {
    api_groups = [""]
    resources  = ["services", "endpoints", "pods", "nodes"]
    verbs      = ["get", "watch", "list"]
  }

  rule {
    api_groups = ["extensions", "networking.k8s.io"]
    resources  = ["ingresses"]
    verbs      = ["get", "watch", "list"]
  }
}
```

External DNS가 클러스터 내 리소스를 감지할 수 있도록 정의한 권한이다. service, pod, node와 같은 리소스를 get, watch, list할 수 있도록 허용한다.

3) ClusterRoleBinding

```
# 3. ClusterRoleBinding
resource "kubernetes_cluster_role_binding" "external_dns_viewer" {
  metadata {
    name = "external-dns-viewer"
    labels = {
      "app.kubernetes.io/name" = "external-dns"
    }
  }

  role_ref {
    api_group = "rbac.authorization.k8s.io"
    kind      = "ClusterRole"
    name      = kubernetes_cluster_role.external_dns.metadata[0].name
  }

  subject {
    kind      = "ServiceAccount"
    name      = kubernetes_service_account.external_dns.metadata[0].name
    namespace = "kube-system"
  }
}
```

위에서 정의한 ClusterRole을 external-dns라는 ServiceAccount와 연결한다. 이를 통해 External DNS pod가 클러스터 내 리소스를 읽을 수 있는 권한을 부여받는다.

4) Deployment

```
# 4. Deployment
resource "kubernetes_deployment" "external_dns" {
  metadata {
    name      = "external-dns"
    namespace = "kube-system"
    labels = {
      "app.kubernetes.io/name" = "external-dns"
    }
  }

  spec {
    replicas = 1

    selector {
      match_labels = {
        "app.kubernetes.io/name" = "external-dns"
      }
    }

    strategy {
      type = "Recreate"
    }

    template {
      metadata {
        labels = {
          "app.kubernetes.io/name" = "external-dns"
        }
        annotations = {
          "eks.amazonaws.com/role-arn" = aws_iam_role.external_dns.arn
        }
      }
    }
  }
}
```

```

spec {
  service_account_name = kubernetes_service_account.external_dns.metadata[0].name

  security_context {
    fs_group = 65534
  }

  container {
    name = "external-dns"
    image = "bitnami/external-dns:0.13.1"

    args = [
      "--source=service",
      "--source=ingress",
      "--domain-filter=lovebridge.click",
      "--provider=aws",
      "--policy=sync",
      "--aws-zone-type=public",
      "--registry=txt",
      "--txt-owner-id=Z057064919GL8WQWUETYS",
    ]
  }
}

```

ExternalDNS 애플리케이션을 실행할 Deployment를 정의하는 코드이다. 라벨은 external-dns로 통일하여 로그나 모니터링을 할 때 해당 라벨로 관리가 가능하다. 그리고 IRSA를 위해 어노테이션에 role external dns를 한 번 더 명시했다. 또한 image는 Bitnami에서 제공하는 ExternalDNS 공식 이미지를 사용하여 안정적인 배포가 이루어지도록 구현했다.

args 아래에 있는 항목은 다음과 같다. ExternalDNS 컨테이너는 kubernetes리소스를 감시하고, lovebridge.click 도메인에 대해서만 Route53에 레코드를 생성하도록 필터링 한다. 그리고 Route53에 레코드를 생성하게 하고 충돌 방지를 위해서 txt 레코드를 추가로 생성하여 상태기록을 하는 동작을 수행하도록 구성했다.

이와 같이 external DNS를 Terraform 기반으로 구성하여 ingress를 감지하여 AWS Route53에 자동으로 A레코드 및 TXT 레코드를 생성할 수 있도록 구현하였다. IAM Role과 정책은 최소 권한으로 정의되었고, IRSA를 통해 kubernetes리소스와 연동되도록 설계했다. 또한 External DNS는 lovebridge.click도메인에 대해 레코드 등록을 자동으로 수행함으로써

관리 효율성을 보여준다. 이런 자동화 구현은 도메인 관리를 수동으로 작업해야하는 것을 최소화 하고 안정적인 운영 환경을 제공할 수 있다.

4. GCP 인프라 구성 (terraform)

4-1 network

```
resource "google_compute_network" "vpc" {
  name                = var.vpc_name
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name                = var.subnet_name
  ip_cidr_range       = var.subnet_cidr
  region              = var.region
  network              = google_compute_network.vpc.id
}

resource "google_compute_route" "allow_internet" {
  name                = "dr-vpc-allow-internet"
  network              = google_compute_network.vpc.name
  dest_range           = "0.0.0.0/0"
  next_hop_gateway     = "default-internet-gateway"
  priority              = 1000
}

resource "google_compute_router" "router" {
  name                = "${var.vpc_name}-router"
  network              = google_compute_network.vpc.id
  region              = var.region
}
```



```
resource "google_compute_router_nat" "nat" {  
  name                = "${var.vpc_name}-nat"  
  router              = google_compute_router.router.name  
  region              = var.region  
  nat_ip_allocate_option = "AUTO_ONLY"  
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"  
}
```

Private Subnet A (DB용)

```
resource "google_compute_subnetwork" "private_subnet_db" {  
  name                = "${var.vpc_name}-private-db"  
  ip_cidr_range       = var.subnet_cidr_db  
  region              = var.region  
  network              = google_compute_network.vpc.id  
  private_ip_google_access = true  
}
```

```

# Private Subnet B (Kubernetes용)
resource "google_compute_subnetwork" "private_subnet_kuber" {
  name                = "${var.vpc_name}-private-b"
  ip_cidr_range       = var.subnet_cidr_kuber
  region              = var.region
  network              = google_compute_network.vpc.id
  private_ip_google_access = true
}

# SSH 허용 방화벽
resource "google_compute_firewall" "allow-ssh" {
  name    = "allow-ssh"
  network = google_compute_network.vpc.name

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["allow-ssh"]
}

```

```

# Cloud SQL 등 프라이빗 연결을 위한 사설 IP 주소 범위 정의
resource "google_compute_global_address" "private_ip_range" {
  name          = "sql-private-range"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  network       = google_compute_network.vpc.id
  depends_on    = [google_compute_network.vpc] # VPC 생성 후 실행
}

# VPC Peering 연결 설정 (Cloud SQL, GKE에 필수)
resource "google_service_networking_connection" "private_vpc_connection" {
  network                = google_compute_network.vpc.id
  service                 = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.private_ip_range.name]
  depends_on              = [
    google_compute_global_address.private_ip_range,
    google_compute_subnetwork.private_subnet_db,
    google_compute_subnetwork.private_subnet_kuber
  ]
}

```

```

resource "google_compute_firewall" "allow_http" {
  name      = "allow-http"
  network   = google_compute_network.vpc.name

  allow {
    protocol = "tcp"
    ports    = ["80", "8080"]
  }

  direction      = "INGRESS"
  source_ranges  = ["0.0.0.0/0"]
  #target_tags    = ["allow-http"]
}

```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 재해복구(Disaster Recovery, DR) 환경을 지원하는 VPC 네트워크 인프라를 프로비저닝한다.

해당 네트워크는 GKE 클러스터, Cloud SQL(MySQL), Compute VM, 그리고 외부 서비스 간의 안전한 통신을 지원하며, 퍼블릭/프라이빗 서브넷, NAT 게이트웨이, 방화벽, VPC Peering 등을 포함한다.

구성 요소

1. VPC 네트워크 생성 (google_compute_network.vpc)
 - 이름은 var.vpc_name으로 동적으로 지정된다.
 - auto_create_subnetworks = false로 설정하여 커스텀 서브넷 모드를 사용한다.
 - 이를 통해 퍼블릭, 프라이빗 서브넷을 용도별로 직접 설계할 수 있다.

2. 퍼블릭 서브넷 (google_compute_subnetwork.subnet)

- 이름은 `var.subnet_name`, CIDR 블록은 `var.subnet_cidr`로 지정.
- 퍼블릭 IP를 할당받아, Compute VM(웹 서버, Bastion) 및 인터넷 통신이 필요한 리소스를 호스팅한다.

3. 인터넷 라우팅 (google_compute_route.allow_internet)

- 0.0.0.0/0 대상으로 기본 인터넷 게이트웨이 경로를 설정.
- 퍼블릭 서브넷에 연결된 VM이 외부와 자유롭게 통신 가능하도록 한다.

4. Cloud Router & NAT (google_compute_router, google_compute_router_nat)

- NAT 게이트웨이를 통해 **프라이빗 서브넷의 리소스(GKE 노드, Cloud SQL 등)가 외부 인터넷으로 나갈 수 있도록 NAT IP를 자동 할당한다.
- `source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"`로 모든 서브넷과 IP 범위가 NAT를 사용하도록 지정.

5. 프라이빗 서브넷 2개

- DB용 서브넷 (google_compute_subnetwork.private_subnet_db)
 - `var.subnet_cidr_db`로 정의된 CIDR 블록을 사용.
 - Cloud SQL과 같은 DB 리소스를 호스팅하며, `private_ip_google_access = true`로 Google API와의 프라이빗 통신을 지원.

- Kubernetes용 서브넷 (google_compute_subnetwork.private_subnet_kuber)
- var.subnet_cidr_kuber로 정의된 CIDR 블록을 사용.
- GKE 클러스터 및 워커 노드 전용 네트워크.
- 동일하게 private_ip_google_access = true 설정.

6. 방화벽 규칙

- SSH 허용 (google_compute_firewall.allow-ssh)
- 0.0.0.0/0에서 22번 포트 접근 허용.
- "allow-ssh" 태그가 붙은 VM에만 적용되어, 주로 Bastion 또는 관리용 VM에서 사용.
- HTTP/웹 트래픽 허용 (google_compute_firewall.allow_http)
- 0.0.0.0/0에서 80, 8080 포트 접근 허용.
- 웹 서버나 컨테이너화된 애플리케이션 서비스에 접근할 수 있게 한다.

7. Cloud SQL 프라이빗 IP를 위한 VPC Peering

- 사설 IP 주소 범위 예약 (google_compute_global_address.private_ip_range)
- purpose = "VPC_PEERING"으로 Cloud SQL 및 기타 Google 서비스와의 프라이빗 네트워크 연결을 위한 CIDR 블록을 예약.
- 서비스 네트워킹 연결
(google_service_networking_connection.private_vpc_connection)

- 예약된 CIDR 블록과 GCP 서비스(servicenetworking.googleapis.com)를 VPC Peering으로 연결.
- Cloud SQL, GKE Pod 네트워크 등이 프라이빗 네트워크로 안전하게 상호작용할 수 있게 한다.

특징 및 사용 목적

- 이 네트워크 인프라는 DR 환경에서 GCP 리소스(GKE, Cloud SQL, Compute VM) 간의 안전하고 유연한 연결을 제공한다.
- 퍼블릭과 프라이빗 서브넷을 분리하여, 웹 서비스 및 관리 VM만 외부 접근을 허용하고 DB와 GKE는 프라이빗 네트워크에서 보호한다.
- NAT 게이트웨이와 VPC Peering을 통해, 외부 업데이트와 Google API 접근은 가능하지만 공용 노출은 방지한다.
- 방화벽 규칙을 태그 기반으로 관리해 보안 정책을 단순화한다.
- Cloud SQL 및 GKE 네트워크가 프라이빗 IP로만 통신할 수 있도록 설정하여, 보안성과 내부 트래픽 최적화를 동시에 달성한다.

4-2 gke

```
resource "google_container_cluster" "lovebridge_cluster" {
  name      = "lovebridge-dr-cluster"
  location  = var.region

  remove_default_node_pool = true
  initial_node_count       = 1 # 필수지만 사용하지 않음

  deletion_protection = false

  network    = var.vpc_id
  subnetwork = var.subnet_name

  ip_allocation_policy {}
}
```

```
resource "google_container_node_pool" "primary_nodes" {
  name      = "primary-node-pool"
  location  = var.region
  cluster   = google_container_cluster.lovebridge_cluster.name

  node_count = var.node_count

  node_config {
    machine_type = var.machine_type

    disk_type    = "pd-standard" # SSD 대신 일반 HDD 사용
    disk_size_gb = 30            # 디스크 용량 축소 (기존보다 적게 설정)

    oauth_scopes = [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  }
}
```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 **GKE(Google Kubernetes Engine) 클러스터와 노드 풀(Node Pool)**을 프로비저닝하기 위한 구성을 정의한다.

AWS에서 운영되는 주 클러스터의 재해복구(Disaster Recovery) 환경으로 활용되며, 필요 시 빠르게 워크로드를 이전하거나 확장할 수 있도록 경량화된 클러스터로 구성된다.

구성 요소

1. GKE 클러스터 생성 (google_container_cluster.lovebridge_cluster)

- lovebridge-dr-cluster라는 이름으로 GKE 클러스터를 생성한다.
- 클러스터는 var.region 변수로 지정된 GCP 리전에 배치된다.
- remove_default_node_pool = true로 기본 노드 풀을 제거하여, 커스텀 노드 풀만 생성하도록 설정한다.
- initial_node_count = 1은 GKE API의 필수값이지만, 실제로 사용하지 않으며 노드 풀 생성 시 별도로 정의한다.
- deletion_protection = false로 설정하여, 필요 시 Terraform으로 쉽게 클러스터를 삭제할 수 있다.
- network와 subnetwork는 각각 var.vpc_id, var.subnet_name으로 지정되어, GCP VPC 네트워크와 서브넷에 연결된다.
- ip_allocation_policy {}를 통해 **Pod 및 Service IP를 위한 VPC 네이티브 모드(CIDR 블록 자동 할당)**를 활성화한다.

2. 노드 풀 생성 (google_container_node_pool.primary_nodes)

- primary-node-pool이라는 이름으로 GKE 노드 풀을 생성한다.
- 클러스터(google_container_cluster.lovebridge_cluster)와 연결되어, 해당 클러스터의 워크로드 실행 환경으로 사용된다.

- 노드 수(`node_count`)는 `var.node_count` 변수로 동적으로 설정되어, 운영 환경의 부하와 DR 시 요구사항에 맞게 조정 가능하다.

3. 노드 설정 (`node_config`)

- `machine_type = var.machine_type`으로 노드의 머신 타입을 동적으로 지정한다. (예: `e2-standard-2`)
- `disk_type = "pd-standard"`: SSD 대신 표준 HDD를 사용하여 비용 절감을 우선시한다.
- `disk_size_gb = 30`: 기본 디스크 용량을 30GB로 축소하여, DR 환경에서 최소한의 스토리지로 운영된다.
- `oauth_scopes`: 노드가 Google Cloud API(`cloud-platform`)에 접근할 수 있도록 범용 권한을 부여한다.

특징 및 사용 목적

- 이 구성은 AWS의 메인 클러스터를 보조하는 DR(Disaster Recovery) 용도로 설계되었으며, 최소 자원으로 운영되다가 필요 시 빠르게 스케일업 가능하다.
- GKE 클러스터는 VPC 네이티브 모드로 설정되어, Pod와 Service IP가 GCP VPC 내에서 관리된다.
- 기본 노드 풀을 제거하고, 커스텀 노드 풀을 별도로 구성하여 머신 타입, 디스크 종류, 용량 등을 유연하게 조정할 수 있도록 한다.
- HDD 기반의 디스크와 소형 디스크 용량을 사용해 비용 최적화를 달성하며, DR 상황에서는 노드 수와 스펙을 쉽게 조정할 수 있다.

4-3 VM

```
resource "google_compute_instance" "vm_instance" {
  name          = var.vm_name
  machine_type  = var.machine_type
  zone          = var.zone
  allow_stopping_for_update = true # ← 이 줄 추가하면 해결

  boot_disk {
    initialize_params {
      image = var.image # 예: "debian-cloud/debian-11"
    }
  }
}
```

```
network_interface {
  network      = var.vpc_id
  subnetwork   = var.subnet_name
  access_config {} # 퍼블릭 IP 부여
}

tags = ["allow-ssh", "allow-http"] # 방화벽 규칙 적용용 태그

metadata = {
  startup-script = <<-EOT
  #!/bin/bash
  apt-get update
  apt-get install -y docker.io

  echo "${var.docker_password}" | docker login -u "${var.docker_username}" --password-stdin
  docker pull ${var.docker_image}
  docker run -d -p 80:8080 ${var.docker_image}
  EOT
}
```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 Compute Engine 가상 머신(VM)을 생성하고, Docker 기반 애플리케이션을 자동 배포하기 위한 구성을 정의한다.

이 VM은 GKE(쿠버네티스) 클러스터 외부에서 독립적으로 애플리케이션을 실행하거나, 임시 테스트 환경 혹은 보조 서버로 사용된다.

구성 요소

1. VM 인스턴스 생성 (google_compute_instance.vm_instance)

- `var.vm_name`으로 정의된 이름을 가진 VM 인스턴스를 생성한다.
- 머신 타입(`machine_type`)과 배치 존(`zone`)은 변수로 지정되어, 환경에 따라 유연하게 조정할 수 있다.
- `allow_stopping_for_update = true` 옵션을 설정하여, VM 업데이트 시 자동으로 정지 후 재시작이 가능하게 한다. 이를 통해 머신 타입, 디스크, 메타데이터 등 변경 시 오류를 방지한다.

2. 부트 디스크 (Boot Disk)

- `initialize_params.image = var.image`로 초기 운영체제(OS) 이미지를 지정한다.
- 예시: "debian-cloud/debian-11" 같은 GCP 제공 기본 이미지 사용.
- 클라우드 이미지로 VM이 부팅되며, 이후 초기화 스크립트(`startup-script`)로 환경이 구성된다.

3. 네트워크 인터페이스 (Network Interface)

- VM은 `var.vpc_id`와 `var.subnet_name`으로 지정된 네트워크와 서브넷에 연결된다.
- `access_config {}` 블록을 추가하여 퍼블릭 IP를 할당, 외부에서 직접 접근 가능하게 한다.
- 퍼블릭 IP를 통해 브라우저나 SSH 접속이 가능하다.

4. 방화벽 태그 (Tags)

- "allow-ssh"와 "allow-http" 태그를 부여하여, 미리 정의된 방화벽 규칙과 연동된다.
- 이를 통해 SSH(22 포트)와 HTTP(80 포트) 접근을 허용할 수 있다.

5. 메타데이터 (Startup Script)

- `startup-script`를 통해 VM 부팅 시 Docker 환경 설치 및 애플리케이션 자동 실행을 수행한다.

1. `apt-get update`로 패키지 정보를 최신화.

2. `apt-get install -y docker.io`로 Docker 설치.

3. 환경 변수(`var.docker_username`, `var.docker_password`)를 사용하여 Docker 레지스트리 로그인.

4. `docker pull ${var.docker_image}`로 지정된 이미지 가져오기.

5. `docker run -d -p 80:8080 ${var.docker_image}`로 컨테이너 실행, VM의 80 포트를 컨테이너의 8080 포트와 매핑.

특징 및 사용 목적

- 이 구성은 **GCP**에서 컨테이너 기반 애플리케이션을 신속히 배포하고 실행할 수 있는 **VM**을 자동으로 생성한다.
- **startup-script**로 부팅 시 자동 환경 설정이 이루어지므로, 개발, 테스트, 또는 **DR** 상황에서 빠르게 인프라를 구축할 수 있다.
- **GKE** 클러스터 외부에서 독립 실행형 서버로 동작하거나, 임시 테스트/확장 환경으로 활용 가능하다.
- 방화벽 태그와 퍼블릭 **IP** 할당으로, 외부 접근과 서비스 테스트가 용이하다.

4-4 DB

```
resource "google_sql_database_instance" "instance" {  
  name          = var.db_instance_name  
  region        = var.region  
  database_version = "MYSQL_8_0"  
  deletion_protection = false  
  
  settings {  
    tier = var.tier  
  
    ip_configuration {  
      ipv4_enabled    = false  
      private_network = var.vpc_network_id  
    }  
  
    backup_configuration {  
      enabled            = true  
      binary_log_enabled = true  
    }  
  
    availability_type = "REGIONAL"  
  }  
}
```

```

resource "google_sql_database" "main" {
  name      = var.db_name
  instance  = google_sql_database_instance.instance.name
  charset   = "utf8mb4"
  collation = "utf8mb4_general_ci"
}

resource "google_sql_user" "user" {
  name      = var.db_user
  instance  = google_sql_database_instance.instance.name
  password  = var.db_password
}

```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 Cloud SQL 기반 MySQL 데이터베이스 인스턴스와 관련 리소스(데이터베이스 및 사용자)를 프로비저닝한다.

해당 인스턴스는 AWS RDS(MySQL)와 비동기 복제(Replication)를 구성하여 재해 복구(Disaster Recovery, DR) 용도로 사용되며, GCP 환경의 GKE 애플리케이션 또는 Compute VM과 프라이빗 네트워크로 연결된다.

구성 요소

1. Cloud SQL 인스턴스3 생성 (google_sql_database_instance.instance)
 - 이름은 var.db_instance_name, 배치 리전은 var.region으로 변수화되어 동적으로 설정된다.
 - MySQL 8.0을 데이터베이스 버전으로 사용하여 최신 호환성을 보장한다.
 - deletion_protection = false로 설정하여, Terraform을 통해 인스턴스를 손쉽게 삭제하거나 재구성할 수 있도록 한다.

2. 인스턴스 설정 (settings)

- tier = var.tier: MySQL 인스턴스의 머신 타입을 지정 (예: db-custom-2-3840), DR 상황과 운영 부하에 따라 조정 가능.
- IP 및 네트워크 구성 (ip_configuration)
 - ipv4_enabled = false: 공용 IP 비활성화.
 - private_network = var.vpc_network_id: GCP VPC 네트워크에만 연결되어, 외부 공개 없이 프라이빗 통신만 허용.
- 백업 및 바이너리 로그 설정 (backup_configuration)
 - enabled = true: 백업 기능 활성화.
 - binary_log_enabled = true: 이진 로그를 활성화하여 복제(Replication)와 데이터 복구 기능을 지원.
- 고가용성(availability_type)
 - availability_type = "REGIONAL"로 설정하여, 리전 내 여러 영역(Zone)에 걸쳐 고가용성(HA)을 보장한다.

3. 데이터베이스 생성 (google_sql_database.main)

- var.db_name으로 지정된 이름의 기본 데이터베이스를 생성한다.
- 문자셋(charset)은 utf8mb4, 정렬(collation)은 utf8mb4_general_ci로 설정하여 다국어와 이모지까지 지원한다.

4. 사용자 생성 (google_sql_user.user)

- 데이터베이스 사용자 이름(var.db_user)과 비밀번호(var.db_password)를 기반으로 MySQL 유저를 생성한다.
- GKE 및 Compute 환경에서 이 계정을 통해 DB에 접근하도록 구성할 수 있다.

특징 및 사용 목적

- 이 구성은 GCP Cloud SQL(MySQL)을 AWS RDS(MySQL)와의 비동기 복제를 통한 DR 환경으로 사용하기 위해 설계되었다.
- 프라이빗 네트워크 기반 통신을 사용하여, 외부 공격을 차단하고 보안성을 강화한다.
- 백업 및 이진 로그 활성화로 데이터 손실 방지 및 복제/복구 기능을 지원한다.
- 고가용성(HA) 옵션으로, GCP 내에서 장애가 발생해도 자동 장애 조치(Failover)가 가능하다.
- google_sql_database와 google_sql_user를 별도로 정의하여, 즉시 사용 가능한 DB 환경을 자동으로 구성한다.

4-5. GCP Cloud Storage 버킷

```
resource "google_storage_bucket" "db_backup_bucket" {
  name          = var.bucket_name
  location      = var.region
  storage_class = "STANDARD"

  uniform_bucket_level_access = true
  force_destroy               = true

  versioning {
    enabled = false
  }

  lifecycle_rule {
    action {
      type = "Delete"
    }
    condition {
      age = 30
    }
  }
}
```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 Cloud Storage 버킷을 프로비저닝하고, 데이터 백업(특히 Cloud SQL 백업 파일 저장)을 위한 자동 수명 주기 관리(Lifecycle Management) 기능을 설정한다.

해당 버킷은 DR(Disaster Recovery) 시나리오에서 MySQL 백업 저장소로 활용되며, 일정 기간 후 불필요한 데이터를 자동으로 삭제해 스토리지 비용을 절감한다.

구성 요소

1. 버킷 생성 (google_storage_bucket.db_backup_bucket)

- 이름(name)은 var.bucket_name으로 지정되어, 프로젝트별로 동적으로 설정할 수 있다.
- location = var.region으로, Cloud SQL 인스턴스 및 DR 환경(GCP 리전)과 동일한 위치에 버킷을 생성하여 지연(latency)을 최소화한다.
- storage_class = "STANDARD"로 지정하여, 일반적인 액세스 빈도의 데이터 저장에 적합하게 설정한다.

2. 권한 관리

- uniform_bucket_level_access = true: 버킷 단위의 일관된 액세스 제어를 사용하여 객체별 ACL 대신 단일 권한 정책으로 관리한다.
- force_destroy = true: 버킷 내 객체가 남아 있어도 Terraform destroy 시 강제 삭제 가능하게 설정한다.

(이는 테스트 환경 또는 자동화된 DR 시나리오에서 유용하다.)

3. 버전 관리 및 수명 주기(Lifecycle)

- versioning.enabled = false: 객체 버전 관리를 비활성화하여 스토리지 비용 최소화.
- lifecycle_rule: 30일이 지난 객체는 자동으로 삭제되도록 설정.

(백업 파일과 로그가 불필요하게 오래 저장되는 것을 방지하고 비용 최적화를 달성한다.)

특징 및 사용 목적

- 이 구성은 Cloud SQL(MySQL) 백업, 애플리케이션 로그, 또는 DR 관련 데이터를 저장하기 위한 GCP Cloud Storage 버킷을 자동으로 생성한다.
- 자동 삭제 정책(30일 후 삭제)을 통해, 스토리지 사용량을 관리하고 장기 보관 비용을 절감한다.
- `uniform_bucket_level_access`를 사용하여 정책 관리 단순화와 보안 일관성을 제공한다.
- `force_destroy` 설정으로, 테스트 환경이나 DR 환경에서 빠른 재구성이 가능하다.

4-6. GCP GKE Ingress용 고정 외부 IP

```
resource "google_compute_address" "gke_ingress_ip" {  
  name          = "gke-ingress-ip"  
  address_type  = "EXTERNAL"  
  network_tier  = "PREMIUM"  
  region        = var.region  
  project       = var.project_id  
}
```

이 `main.tf` 파일은 Google Cloud Platform(GCP)에서 GKE 클러스터의 Ingress Controller나 외부 노출 서비스에 사용할 고정(Static) 외부 IP 주소를 프로비저닝한다.

고정 IP를 사용하면 DNS 설정과 외부 클라이언트 접근 시 IP가 변경되지 않으므로, 안정적인 서비스 노출 및 도메인 연동이 가능하다.

구성 요소

1. 고정 외부 IP 생성 (google_compute_address.gke_ingress_ip)

- name = "gke-ingress-ip"로 리소스 이름을 지정하여, GKE Ingress Controller나 외부 Load Balancer에서 쉽게 참조할 수 있다.
- address_type = "EXTERNAL"로 설정하여 퍼블릭 인터넷에서 접근 가능한 고정 IP를 생성한다.
- network_tier = "PREMIUM"을 사용하여 Google Global Network의 고성능 라우팅을 활용한다.

(이는 기본(Standard) 티어보다 더 낮은 지연시간(Latency)과 높은 가용성을 제공한다.)

- region = var.region으로, GKE 클러스터 및 해당 Load Balancer와 동일한 리전에 배치하여 성능과 비용을 최적화한다.
- project = var.project_id로 프로젝트 범위 내에서 IP를 관리할 수 있다.

특징 및 사용 목적

- 이 고정 외부 IP는 GKE Ingress Controller 또는 외부 Load Balancer에서 사용되며, 서비스 노출 시 안정적인 DNS 연결을 보장한다.
- network_tier = "PREMIUM"을 통해, Google 글로벌 네트워크 기반의 고속, 저지연, 고가용성 네트워크 경로를 제공한다.

- 도메인(Route 53, Cloud DNS 등)과 연계해 변하지 않는 고정 IP 기반의 레코드 등록이 가능하다.
- GCP 환경에서 DR 시나리오 또는 외부 클라이언트와의 안정적인 통신을 보장하며, GKE 기반 애플리케이션 서비스의 진입점(Entry Point) 역할을 한다.

4-7. GCP DR 인프라 루트 모듈 구성

```
terraform {  
  required_providers {  
    kubectl = {  
      source = "gavinbunney/kubectl"  
      version = "~> 1.14.0"  
    }  
  }  
}  
  
provider "google" {  
  credentials = file("terraform-gcp-key.json")  
  project     = var.project  
  region      = var.region  
}  
  
provider "kubectl" {  
  config_path = "~/.kube/config"  
  config_context = "gke_direct-tribute-463400-f2_us-central1_lovebridge-dr-cluster"  
}
```

```
module "network" {  
  source      = "./modules/network"  
  project     = var.project  
  region      = var.region  
  vpc_name    = "dr-vpc"  
  subnet_name = "dr-subnet"  
  
  subnet_cidr      = "10.0.0.0/24"  
  subnet_cidr_db   = "10.0.2.0/24"  
  subnet_cidr_kuber = "10.0.3.0/24"  
}
```

```
module "compute" {  
    source          = "./modules/compute"  
    vm_name         = var.vm_name  
    machine_type    = var.machine_type  
    zone            = var.zone  
    image           = var.image  
    vpc_id          = module.network.vpc_id  
    subnet_name     = module.network.subnet_name  
  
    startup_script = <<-EOT  
        #!/bin/bash  
        sudo apt update  
        sudo apt install -y default-jdk  
        echo "Spring Boot Ready"  
    EOT  
  
    docker_username = var.docker_username  
    docker_password = var.docker_password  
    docker_image    = var.docker_image  
}
```



```
module "db" {  
  source = "../modules/db"  
  
  db_instance_name = "lovebridge-instance"  
  db_name           = "lovebridge_main"  
  db_user           = "admin"  
  db_password       = var.db_password  
  region            = "us-central1"  
  tier               = "db-g1-small"  
  vpc_network_id    = module.network.vpc_id  
  depends_on        = [module.network]  
}
```

```
module "bucket_backup" {  
  source      = "../modules/bucket"  
  project_id  = var.project  
  region      = var.region  
  bucket_name = "lovebridge-db-backups"  
}
```

```
module "gke" {  
  source      = "../modules/gke"  
  region      = var.region  
  vpc_id      = module.network.vpc_id  
  subnet_name = module.network.subnet_name  
  node_count  = var.node_count  
  machine_type = var.machine_type  
}
```

```
module "static_ip" {  
  source      = "../modules/static-ip"  
  region      = var.region  
  project_id  = var.project  
}
```

이 main.tf 파일은 Google Cloud Platform(GCP)에서 재해복구(Disaster Recovery, DR) 환경을 위한 인프라를 Terraform으로 자동 프로비저닝하기 위한 최상위(Root) 모듈이다. 모듈화된 하위 구성 요소들을 호출하여 네트워크, 컴퓨팅, 데이터베이스, 스토리지, Kubernetes(GKE), 고정 IP 등 DR 환경의 전체 인프라를 통합적으로 배포하고 관리한다.

구성 요소 및 역할

1. 필수 Provider 정의

- google:

- GCP 리소스 생성을 위한 기본 Provider.
- credentials = file("terraform-gcp-key.json")로 서비스 계정 키를 사용해 인증.
- project와 region을 변수로 받아 모든 리소스에 일관되게 적용.

- kubectl:

- gavinbunney/kubectl Provider를 사용해 GKE 클러스터와 상호작용.

- config_path와 config_context로 로컬 Kubernetes 설정(~/.kube/config)을 참조하여, 배포 이후 클러스터 리소스를 자동으로 적용할 수 있게 한다.

2. 모듈 기반 인프라 구성

- 네트워크 모듈 (module.network)

- ./modules/network를 통해 DR 전용 VPC, 퍼블릭 및 프라이빗 서브넷(DB용, GKE용), NAT, 방화벽, VPC Peering을 생성.
- CIDR 블록(subnet_cidr, subnet_cidr_db, subnet_cidr_kuber)을 인자로 받아 네트워크 세분화.

- Compute VM 모듈 (module.compute)

- ./modules/compute를 통해 Docker 애플리케이션을 실행하는 VM 인스턴스를 생성.
- 부팅 시 startup_script를 실행하여 Java 환경 및 애플리케이션 준비 상태를 세팅.
- Docker 자격 증명(docker_username, docker_password)과 배포할 이미지(docker_image)를 인자로 전달.

- Cloud SQL 모듈 (module.db)

- ./modules/db를 통해 MySQL 8.0 기반 Cloud SQL 인스턴스와 기본 데이터베이스 및 사용자를 생성.
- vpc_network_id로 프라이빗 네트워크에 연결, 백업 및 바이너리 로그 활성화로 복제/DR 지원.

- 백업 버킷 모듈 (`module.bucket_backup`)
 - `./modules/bucket`을 통해 Cloud Storage 버킷을 생성, DB 백업 파일을 저장.
 - 30일 후 자동 삭제 라이프사이클 정책을 적용해 스토리지 비용 최적화.
- GKE 클러스터 모듈 (`module.gke`)
 - `./modules/gke`를 통해 GKE 클러스터 및 노드 풀을 생성.
 - `vpc_id`, `subnet_name`을 전달해 프라이빗 네트워크 상에서 동작하며, 노드 수(`node_count`)와 머신 타입(`machine_type`)을 인자로 받아 확장 가능.
- 고정 IP 모듈 (`module.static_ip`)
 - `./modules/static-ip`을 통해 Ingress Controller 및 외부 서비스 노출용 고정 외부 IP를 프로비저닝.
 - `network_tier = "PREMIUM"`으로 글로벌 네트워크 성능을 활용해 외부 접근의 안정성과 저지연성을 제공.

3. 출력 값 정의

- `client_key`, `client_certificate`, `ca_certificate`:
 - GKE 클러스터 접근을 위한 인증서와 키를 출력.
 - `sensitive = true` 옵션으로 민감 정보 노출을 방지.
- 이 출력 값들은 CI/CD 파이프라인이나 Helm/Kubectl Provider에서 클러스터와 직접 연동할 때 활용된다.

특징 및 사용 목적

- 이 루트 모듈은 **GCP** 기반 **DR** 인프라를 모듈 단위로 자동화 배포하도록 설계되어, 관리와 유지보수가 용이하다.
- 네트워크, **Compute**, **DB**, 스토리지, **Kubernetes**, **Static IP** 등 **DR** 인프라 구성 요소들을 하나의 **Terraform** 프로젝트에서 통합 관리한다.
- 모든 리소스가 프라이빗 네트워크 기반으로 연결되며, 필요 시 **NAT**와 **Bastion**을 통해 외부와 통신 가능.
- **Cloud SQL**과 **GKE**는 **VPC Peering**을 통해 프라이빗 연결, 백업 버킷과 **DR** 구성을 통해 데이터 복원 및 재해복구 전략을 지원.
- **CI/CD** 또는 **ArgoCD** 환경과 연동 시, **GKE** 클러스터 인증 정보를 자동으로 활용해 애플리케이션을 신속하게 배포할 수 있다.

5. 인프라 구성(Kubernetes)

안정적인 서비스 운영과 유연한 확장성을 확보하기 위해 **kubernetes** 기반의 컨테이너 오케스트레이션 환경을 구축했다. **Deployment**, **Service**, **HPA**, **ConfigMap** 등을 정의하였고 이를 통해 자동화된 배포 및 관리가 가능하도록 설계하였다.

5장에서는 각 리소스 파일별로 **kubernetes**에서의 설정 내용을 설명하고자 한다.

5-1. Deployment 구성

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lovebridge-basic
  labels:
    app: lovebridge
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lovebridge
  template:
    metadata:
      labels:
        app: lovebridge
    spec:
      serviceAccountName: ecr-sa
      containers:
        - name: lovebridge
          image: 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin:latest
          ports:
            - containerPort: 8080
```

lovebridge라는 이름의 **springboot** 애플리케이션을 **Kubernetes** 클러스터에 배포하기 위한 코드이다. **replica**를 3으로 설정하여 동일한 **pod**를 3개 복제하여 가용성을 보장한다. 또한 **aws ecr**에 올려놓은 이미지를 받아와서 컨테이너를 실행한다. **pod** 내 컨테이너는 8080 포트를 통해 외부와 통신한다.

5-2. 서비스 구성

```
apiVersion: v1
kind: Service
metadata:
  name: lovebridge-service
  labels:
    app: lovebridge
spec:
  selector:
    app: lovebridge
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
```

위 코드는 **lovebridge** 애플리케이션을 외부 또는 클러스터 내부에서 접근 가능하도록 설정한다. **type**이 **ClusterIP**타입으로 설정되어 있기에 클러스터 내부에서만 접근 가능한 내부 서비스를 제공한다.

metadata.labels.app: lovebridge 는 라벨을 설정하는 코드로, 리소스 식별이나 선택자 매칭 시에 사용된다. **spec.selector.app : lovebridge** 는 **service**가 연결한 **pod**를 선택하는 라벨을 명시하는 코드이다. 즉 동일한 라벨을 가진 **pod**에 트래픽을 라우팅한다.

5-3. Hpa 구성

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: lovebridge-prod-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: lovebridge-basic
  minReplicas: 3
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

hpa는 cpu 사용률 기준으로 자동으로 pod수를 조절하기 위한 리소스이다. 리소스 사용량에 따라 동적으로 스케일링 되기에 성능 최적화와 리소스 자원을 절약이 가능하다.

spec.scaleTargetRef에 lovebridge-basic Deployment를 명시하여 스케일링 대상 리소스를 지정한다. 그리고 minReplicas는 3개, maxReplicas는 6개로 지정하여 최소 pod는 항상 3개 이상 유지되고 부하가 높을 때는 6개까지 확장되도록 설정했다. cpu 사용률에 대한 기준은 target.averageUtilization:60 으로 설정하여 전체 pod의 평균 cpu 사용률이 60%를 초과할 경우에 pod가 확장되도록 설정했다.

5-4. db

```
love-kube > db > ! db-pod.yml > {} spec > [ ] containers > {} 0
io.k8s.api.core.v1.Pod (v1@pod.json)
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: db-client
5  spec:
6    containers:
7      - name: db-client-container
8        image: mariadb:10.7
9        command: ["sleep", "3600"]
10       env:
11         - name: DB_HOST
12           valueFrom:
13             configMapKeyRef:
14               name: db-config
15               key: DB_HOST
16         - name: DB_PORT
17           valueFrom:
18             configMapKeyRef:
19               name: db-config
20               key: DB_PORT
21         - name: DB_USER
22           valueFrom:
23             secretKeyRef:
24               name: db-secret
25               key: DB_USER
26         - name: DB_PASS
27           valueFrom:
28             secretKeyRef:
29               name: db-secret
30               key: DB_PASS
31
```

해당 db-pod.yml 파일은 db-client라는 이름의 Kubernetes Pod를 정의한 것으로, MariaDB 클라이언트 역할을 수행하는 컨테이너를 1시간 동안 실행하며 외부 DB 서버에 접속 테스트를 할 수 있도록 구성되어 있다. 이 Pod는 mariadb:10.7 이미지를 기반으로 하고, 컨테이너 내부에서 즉시 종료되지 않도록 sleep 3600 명령어를 실행하여 1시간 동안 대기한다.

컨테이너는 환경변수를 통해 DB 접속 정보를 주입받는데, DB_HOST와 DB_PORT는 db-config라는 이름의 ConfigMap에서, DB_USER와 DB_PASS는 db-secret이라는 이름의

Secret에서 각각 읽어온다. 이 설정을 통해 사용자는 해당 Pod에 접속해 MySQL 클라이언트 명령어를 직접 실행함으로써, 외부에 위치한 데이터베이스 연결을 확인하거나 트러블슈팅할 수 있는 테스트 환경을 구성할 수 있다.

```
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
1 # configmap.yaml
2 apiVersion: v1
3 kind: ConfigMap
4 metadata:
5   name: db-config
6 data:
7   DB_HOST: mariadb-2a.c566mcwwy70y.ap-northeast-2.rds.amazonaws.com
8   DB_PORT: "3306"
9
```

```
love-kube > db > ! secret.yml > {} stringData
```

```
io.k8s.api.core.v1.Secret (v1@secret.json)
```

```
1  # secret.yml
2  apiVersion: v1
3  kind: Secret
4  metadata:
5    name: db-secret
6  type: Opaque
7  stringData:
8    DB_USER: root
9    DB_PASS: rhdown9953!
```

```
10
```

6. CI/CD 구성

본 프로젝트에서는 멀티 클라우드 환경(AWS, GCP)에서 각각 독립적으로 CI/CD와 GitOps 자동화 시스템을 구축하였다. 각 클라우드 환경의 특성과 요구사항을 반영하여 GitLab CI/CD 파이프라인, Argo CD 기반의 GitOps 배포, 그리고 쿠버네티스 클러스터 내의 리소스 자동화를 단계적으로 구성하였다. 이를 통해 서비스 배포의 일관성과 재현성을 확보하고, 인프라 변경 사항을 버전 관리 시스템을 통해 추적 가능하게 하였다.

6-1 인프라 설정

Argo CD는 Helm을 사용하여 자동 설치되었으며, ALB와 External DNS를 연동하여 `argocd.lovebridge.click` 도메인으로 외부 접근이 가능하도록 설정하였다. 또한 ACM SSL 인증서를 적용하여 HTTPS 연결을 지원한다.

```
provider "helm" {
  alias = "eks"

  kubernetes = {
    host = module.eks0.cluster_endpoint
    cluster_ca_certificate = base64decode(module.eks0.cluster_certificate_authority_data)
    token = module.eks0.cluster_token
    config_path = "~/.kube/config"
  }
}
```

✅ Argo CD 설치 + Ingress (ALB + external-dns + 인증서) 자동 구성

```
resource "helm_release" "argocd" {
  provider      = helm.eks
  name          = "argocd"
  repository    = "https://argoproj.github.io/argo-helm"
  chart         = "argo-cd"
  version       = "5.51.6"
  namespace     = "argocd"
  create_namespace = true

  values = [
    yamlencode({
      # _____ 1) 관리자 비밀번호 설정 _____
      configs = {
        secret = [
          # admin1234 를 bcrypt 로 해시한 예시
          argocdServerAdminPassword = "$2a$12$2IMNkoxtsib33xo/thSvgOdVzTtEbiMha3dHPDrc/C1Dqc0lrVRQu"
        ]
      }
    })
  ]
}
```

```
server = {
  extraArgs = ["--insecure"]

  service = {
    type = "ClusterIP"
    ports = [
      { name = "http", port = 80, targetPort = 8080 },
      { name = "https", port = 443, targetPort = 8083 }
    ]
  }
}
```

```
ingress = {
  enabled          = true
  ingressClassName = "alb"
  servicePort      = "http"
  hosts            = ["argocd.lovebridge.click"]
  annotations = {
    "alb.ingress.kubernetes.io/scheme"           = "internet-facing"
    "alb.ingress.kubernetes.io/target-type"       = "ip"
    "alb.ingress.kubernetes.io/listen-ports"      = "[{\"HTTP\":80},{\"HTTPS\":443}]"
    "alb.ingress.kubernetes.io/certificate-arn"    = "arn:aws:acm:ap-northeast-2:680993828418:certi"
    "alb.ingress.kubernetes.io/ssl-policy"         = "ELBSecurityPolicy-2016-08"
    "external-dns.alpha.kubernetes.io/hostname"    = "argocd.lovebridge.click"
    "alb.ingress.kubernetes.io/healthcheck-path"   = "/"
    "alb.ingress.kubernetes.io/healthcheck-port"   = "80"
    "alb.ingress.kubernetes.io/healthcheck-protocol" = "HTTP"
    "alb.ingress.kubernetes.io/backend-protocol"   = "HTTP"
    "alb.ingress.kubernetes.io/backend-port"       = "http"
  }
  paths          = ["/"]
  pathType       = "Prefix"
}
}
```

본 환경에서는 AWS EKS 클러스터와 네트워크, IAM, 보안 그룹 등의 기본 인프라가 Terraform으로 자동화되었으며, 그 위에서 Argo CD와 ALB Ingress Controller를 Helm을 사용하여 배포하였다. 이를 통해 GitOps 기반 애플리케이션 배포 환경을 구축하고, Argo CD UI를 외부 도메인으로 안전하게 노출할 수 있는 HTTPS 접근 방식을 지원한다.

1. ALB Ingress Controller 설치

- aws-load-balancer-controller Helm Chart를 사용하여 ALB Controller를 kube-system 네임스페이스에 배포하였다.
- ALB Controller는 Kubernetes Ingress 리소스를 감시하면서, AWS Application Load Balancer(ALB)를 자동 생성하고 관리한다.
- 코드에서 `clusterName`, `region`, `vpclId` 값이 지정되어 있어, ALB Controller는 해당 EKS 클러스터와 VPC 설정에 맞춰 리소스를 연동한다.
- `image.repository`는 AWS ECR에서 제공되는 ALB Controller 공식 이미지를 사용하며, `serviceAccount` 설정은 OIDC(IRSA) 기반 IAM Role과 연동된 사전 생성 계정을 사용하도록 구성되어 있다.
- `ingressClass = "alb"` 로 설정하여, Ingress 리소스가 ALB Controller를 통해 라우팅되도록 명시하였다.

2. Argo CD 설치 및 배포

- Argo CD는 argo-helm Chart를 사용하여 `argocd` 네임스페이스에 설치되었으며, Helm Provider를 통해 Terraform 코드 내에서 자동으로 배포된다.
- Helm values에서 ****관리자 비밀번호(argocdServerAdminPassword)****를 `bcrypt` 해시로 사전에 지정해 초기 보안 설정을 강화하였다.

- Argo CD 서버는 ClusterIP 타입으로 내부 서비스를 구성하되, ALB Ingress를 통해 외부에서 접근 가능하도록 설정되어 있다.
- extraArgs에 --insecure 옵션을 추가해 초기 접속 시 TLS 검증 문제를 우회할 수 있도록 구성했지만, 외부 접속은 ACM 인증서가 적용된 HTTPS를 사용해 보안성을 유지한다.

3. Ingress, External DNS, ACM을 통한 HTTPS 구성

- Argo CD Ingress 리소스는 alb 클래스를 사용하며, ALB Controller가 이를 감지하여 인터넷 공개용 ALB를 생성한다.
- ALB는 80/443 포트 모두 리스닝하며, HTTPS 연결을 위해 alb.ingress.kubernetes.io/certificate-arn에 ACM 인증서를 지정하였다.
- external-dns.alpha.kubernetes.io/hostname 주석을 통해, External DNS가 Route 53에 argocd.lovebridge.click DNS 레코드를 자동으로 생성 및 관리한다.
- 헬스체크(alb.ingress.kubernetes.io/healthcheck-path 및 healthcheck-port) 설정으로 ALB가 Argo CD 서비스의 상태를 지속적으로 감시하며, 장애 시 트래픽을 자동으로 재분배한다.
- 최종적으로 사용자는 https://argocd.lovebridge.click 도메인을 통해 Argo CD 웹 UI에 접속할 수 있으며, TLS 암호화된 안전한 접속이 보장된다.

4. 구성의 특징

- Terraform + Helm을 통한 완전 자동화: ALB Controller, Argo CD, Ingress, External DNS, ACM 인증서 연계까지 인프라 및 애플리케이션 배포가 코드로 관리된다.
- 확장성과 안정성: ALB 기반 Ingress는 HTTPS 트래픽을 처리하며, External DNS가 Route 53 레코드를 동적으로 관리해 수동 설정 없이 도메인 운영이 가능하다.
- 보안 강화: ACM 인증서를 사용한 TLS, IRSA 기반 IAM Role 연동(ServiceAccount), 관리자 비밀번호 해시 지정 등 보안 요소가 반영되었다.
- GitOps 환경의 기반: 이 구성 위에서 GitLab과 Argo CD를 연계하여 CI/CD 파이프라인을 운영하며, Argo CD를 통해 배포 애플리케이션의 상태를 중앙에서 모니터링할 수 있다.

6-2. GitLab CI/CD 파이프라인 구성

이 CI/CD 파이프라인은 AWS EKS 환경에서 애플리케이션을 자동으로 빌드하고 배포하기 위해 GitLab CI와 Argo CD를 연계한 방식으로 동작한다.

개발자가 코드 변경 사항을 main 브랜치에 푸시하면, GitLab CI가 Docker 이미지 빌드 → ECR 업로드 → 배포 YAML 파일의 이미지 태그 업데이트 → EKS에 배포를 순차적으로 수행한다.

Argo CD는 GitLab 저장소를 모니터링하며, 배포 YAML이 업데이트될 때마다 자동으로 클러스터와 동기화하여 GitOps 기반으로 최종 상태를 보장한다.

동작 과정:

1. 개발자가 GitLab 저장소에 코드 푸시
2. GitLab CI Pipeline이 Docker 이미지 빌드 → ECR 업로드
3. deploy-basic.yaml의 image 태그를 최신 커밋 해시로 자동 업데이트
4. Argo CD가 GitLab 저장소를 모니터링하고, 변경된 매니페스트를 EKS 클러스터에 자동 적용
5. 사용자는 ALB를 통해 lovebridge.click 도메인으로 서비스에 접근

```
stages:
  - deploy

variables:
  AWS_REGION: ap-northeast-2
  CLUSTER_NAME: lovebridge-eks # 실제 클러스터 이름으로 수정
  ECR_REPO: 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin
  IMAGE_TAG: $CI_COMMIT_SHORT_SHA

deploy:
  stage: deploy
  image: docker:20.10.24-cli

  services:
    - name: docker:20.10.24-dind
      command: ["--dns", "8.8.8.8", "--dns", "1.1.1.1"]
```

```
before_script:
  - apk add --no-cache curl bash git python3 py3-pip
  - pip3 install awscli
  - aws --version

# ✅ ECR 로그인
- aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $ECR_REPO

# ✅ kubectl 설치
- curl -LO "https://dl.k8s.io/release/v1.29.0/bin/linux/amd64/kubectl"
- install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
- kubectl version --client

# ✅ kubeconfig 자동 구성
- aws eks update-kubeconfig --region $AWS_REGION --name $CLUSTER_NAME

# ✅ Git 사용자 설정
- git config --global user.email "mks0301140@gmail.com"
- git config --global user.name "mks0301140"
- git remote set-url origin https://mks0301140:$GITLAB_ACCESS_TOKEN@gitlab.com/sooj-group/dating-app-k8s-aws.git
```

```

script: |
  echo "🔧 1. Docker 이미지 빌드 및 ECR 푸시"
  docker build -t $ECR_REPO:$IMAGE_TAG .
  docker push $ECR_REPO:$IMAGE_TAG

  echo "📄 2. YAML 이미지 태그 변경"
  sed -i "s|image: .*|image: $ECR_REPO:$IMAGE_TAG|" k8s/deploy-basic.yml

  echo "📦 3. 변경 커밋 (git push는 생략)"
  git add k8s/deploy-basic.yml
  git commit -m "Update image tag to $IMAGE_TAG" || echo "No changes to commit"
  echo "🔒 Git push 생략: 수동으로 git push 필요"

  echo "🚀 4. kubectl로 배포"
  kubectl apply -f k8s/deploy-basic.yml --validate=false

only:
  - main

```

이 파이프라인은 deploy 스테이지로 구성되어 있으며, 내부적으로 다음과 같은 단계를 수행한다.

(1) Docker 이미지 빌드 및 Amazon ECR 푸시

- GitLab Runner는 docker:dind(Docker-in-Docker) 환경을 사용하여 애플리케이션 Docker 이미지를 빌드한다.
- aws ecr get-login-password 명령어를 통해 AWS ECR에 로그인한 후, 빌드된 이미지를 \$ECR_REPO:\$IMAGE_TAG 형태로 태깅하고 푸시한다.
- 여기서 IMAGE_TAG는 **GitLab 커밋 해시(\$CI_COMMIT_SHORT_SHA)**로 설정되어, 매 빌드마다 고유한 이미지 버전이 생성된다.

(2) 배포 매니페스트(k8s/deploy-basic.yml)의 이미지 태그 업데이트

- sed 명령어를 사용하여, 배포 YAML 파일(k8s/deploy-basic.yml) 내 image: 항목을 새로 빌드한 이미지 태그로 교체한다.
- 이를 통해 Kubernetes Deployment가 최신 빌드 이미지로 업데이트되도록 보장한다.

- 변경된 YAML은 Git에 커밋되지만, GitLab CI에서는 자동 push를 생략하여 수동으로 버전 관리할 수 있게 했다.

(3) Kubernetes 클러스터 연결 및 배포

- aws eks update-kubeconfig 명령어를 통해 **EKS 클러스터 인증 정보를 자동 구성**한다.

이 과정에서 AWS CLI를 사용하여 IAM Role 기반으로 kubeconfig를 업데이트하며, Runner가 kubectl을 통해 클러스터에 접근할 수 있도록 한다.

- kubectl apply 명령을 사용해 업데이트된 deploy-basic.yml을 EKS에 적용한다.
이로써 새로운 Docker 이미지가 Pod로 배포되며, Kubernetes는 롤링 업데이트 방식으로 기존 Pod를 교체한다.

(4) Argo CD의 GitOps 동작

- Argo CD는 argocd.lovebridge.click 도메인을 통해 외부에서 접근 가능하며, GitLab 저장소를 지속적으로 감시한다.
- deploy-basic.yml의 이미지 태그가 변경되면, Argo CD는 자동으로 해당 변경 사항을 감지하고, EKS 클러스터 상태와 Git 저장소 상태를 동기화(Sync)한다.
- 이로써 수동 배포 명령 없이도 클러스터 상태가 Git 저장소와 일치하게 유지된다.

6-3 주요 자동화 포인트

1) 자동 버전 관리

- GitLab CI가 빌드마다 커밋 해시를 이미지 태그로 사용하므로, 특정 버전으로 롤백하거나 추적하기 용이하다.

2) Argo CD와의 연계로 GitOps 운영

- GitLab CI는 이미지 빌드 및 매니페스트 업데이트를 담당하고, Argo CD는 클러스터와 Git 저장소 간 동기화를 보장한다.
- 배포 후에도 클러스터 상태가 Git 저장소의 상태와 일관되게 유지된다.

3) 완전 자동화된 배포 파이프라인

- 개발자는 단순히 코드 푸시만 하면 빌드와 배포가 자동으로 처리되며, 롤링 업데이트도 Kubernetes가 자동으로 수행한다.

4) 확장성

- 동일한 구조를 GCP GKE 환경에서도 재사용 가능하며, GitLab Runner를 IRSA 기반으로 변경하면 보안성을 더욱 강화할 수 있다.

7. App/DB 구성

```
1 spring.application.name=boot
2
3 spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
4 spring.datasource.username=root
5 spring.datasource.password=rhdm9953!
6 spring.datasource.url=jdbc:mariadb://mariadb-2a.c566mcwy70y.ap-northeast-2.rds.amazonaws.com:3306/lovebridgedb?useUnicode=true&characterEncoding=utf8mb4
7 # mybatis framework setting
8 mybatis.mapper-locations=/mappers/*.xml
9
10 # JSP settings
11 spring.mvc.view.prefix=/WEB-INF/
12 spring.mvc.view.suffix=.jsp
13 spring.data.redis.host=redis-master.default.svc.cluster.local
14 spring.data.redis.port=6379
15
16 #session store
17 spring.session.store-type=redis
18
19 # file settings
20 spring.servlet.multipart.max-file-size=10MB
21
22 # server port settings
23 server.port=8080
24
25 server.address=0.0.0.0
```

application.properties

이 설정 파일은 AWS RDS를 통해 MariaDB 데이터베이스를 연결하도록 구성되어 있다. `spring.datasource.url` 항목에는 AWS의 RDS 엔드포인트와 데이터베이스 이름이 명시되어 있으며, 사용자명과 비밀번호도 함께 지정되어 있다.

또한, Redis는 클라우드 네이티브 환경인 Kubernetes에서 서비스로 배포된 Redis 인스턴스를 사용하도록 설정되어 있다.

`redis-master.default.svc.cluster.local`는 Kubernetes 클러스터 내부 DNS를 통해 Redis에 접근하는 방식이며, 이는 클라우드 환경의 서비스 디스커버리 메커니즘을 활용한 것이다. 이러한 설정은 클라우드 기반 인프라에서 데이터베이스와 캐시 시스템을 외부 서비스나 Pod 단위로 분리하여 확장성과 유연성을 확보하기 위함이다.

환경별 연결 대상은 다음과 같이 설정된다.

```
#spring.data.redis.host=lovebridge-redis.odpczc.ng.0001.apn2.cache.amazonaws.com
#spring.data.redis.port=6379
```

이 부분은 **elasticache**를 사용한 운영환경에서의 연결 방법이다.

```
spring.data.redis.host=redis-master.default.svc.cluster.local
spring.data.redis.port=6379
```

이 코드는 **helm**을 통해 배포된 **redis** 서비스의 내부 **dns**주소이다. **redis** 인프라가 변경되면 **application.properties**의 해당 부분을 수정하면 운영 및 테스트 환경 모두 확인 가능하다.

```
62 <dependency>
63     <groupId>com.amazonaws</groupId>
64     <artifactId>aws-java-sdk-s3</artifactId>
65     <version>1.12.686</version>
66 </dependency>
67
```

AWS SDK 중 S3에 접근하기 위한 라이브러리다.

```
121 <plugin>
122     <groupId>com.google.cloud.tools</groupId>
123     <artifactId>jib-maven-plugin</artifactId>
124     <version>3.4.5</version>
125     <configuration>
126         <!-- 기본 푸시 대상: DockerHub -->
127
128     <from>
129         <image>eclipse-temurin:17-jre</image>
130     </from>
131
132     <!-- Target: ECR에만 푸시 -->
133     <to>
134         <image>680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin</image>
135         <tags>
136             <tag>1.1</tag>
137             <tag>latest</tag>
138         </tags>
139     </to>
140
141     <!-- 추가 푸시 대상: ECR -->
142
143 </configuration>
144
145 </plugin>
```

jib-maven-plugin은 **Dockerfile** 없이도 **Maven** 기반 **Spring Boot** 프로젝트를 바로 **Docker** 이미지로 빌드해주는 플러그인이다.

이 설정을 통해 애플리케이션을 컨테이너 이미지로 만들고, **AWS ECR**에 자동으로 업로드할 수 있게 구성된다.

redisconfig.java

```
1 package com.care.boot.config;
2
3 import java.time.Duration;
4
13
14 @Configuration
15 @EnableCaching
16 public class RedisConfig {
17
18     @Value("${spring.data.redis.host}")
19     private String host;
20
21     @Value("${spring.data.redis.port}")
22     private int port;
23
24     @Bean
25     public RedisConnectionFactory redisConnectionFactory() {
26         return new LettuceConnectionFactory(host, port);
27     }
28
29     @Bean
30     public RedisCacheManager redisCacheManager(RedisConnectionFactory connectionFactory) {
31         RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
32             .entryTtl(Duration.ofMinutes(20)); // ☑ TTL 30분
33
34         return RedisCacheManager.builder(connectionFactory)
35             .cacheDefaults(config)
36             .build();
37     }
38 }
```

spring 애플리케이션 내에서 redis 연결을 하는 부분이다. @value 어노테이션을 통해 application.properties에 명시되어있는 redis host와 port를 주입받는다. 그리고 LettuceConnectionFactory를 사용하여 Redis연결을 생성한다. 캐시 설정은 RedisCacheManager를 통해 캐시 TTL을 20분을 설정한다.

Awsconfig.java

```
1 package com.care.boot.config;
2
3 import com.amazonaws.auth.AWSSessionCredentialsProvider;
4 import com.amazonaws.auth.BasicAWSCredentials;
5 import com.amazonaws.services.s3.AmazonS3;
6 import com.amazonaws.services.s3.AmazonS3ClientBuilder;
7 import org.springframework.beans.factory.annotation.Value;
8 import org.springframework.context.annotation.Bean;
9 import org.springframework.context.annotation.Configuration;
10
11 @Configuration
12 public class Awsconfig {
13
14     @Value("${cloud.aws.credentials.access-key}")
15     private String accessKey;
16
17     @Value("${cloud.aws.credentials.secret-key}")
18     private String secretKey;
19
20     @Value("${cloud.aws.region.static}")
21     private String region;
22
23     @Bean
24     public AmazonS3 amazonS3() {
25         BasicAWSCredentials creds = new BasicAWSCredentials(accessKey, secretKey);
26         return AmazonS3ClientBuilder.standard()
27             .withRegion(region)
28             .withCredentials(new AWSSessionCredentialsProvider(creds))
29             .build();
30     }
31 }
```

aws s3를 통해 파일 업로드 및 다운로드 기능을 제공하기 위해 spring 애플리케이션에서 s3 클라이언트를 초기화하는 설정 클래스를 정의한 코드이다. 명시된 자격증명과 리전을 기반으로 AmazonS3 객체를 빈으로 등록하여 이후에 서비스 클래스에서 주입받아서 S3 API를 호출할 수 있도록 구성하였다. 간략히 코드 구성에 대해 설명하자면 다음과 같다.

@value 를 통해 application.properties에 정의된 값을 주입한다. 그리고 BasicAWSCredentials를 통해 기본 자격 증명 클래스를 정의한다.

8. 재해 복구

본 프로젝트에서는 멀티클라우드를 활용한 재해복구 시스템을 구축하였다.

단일 클라우드 환경은 편리성과 통합성 면에서는 장점이 있지만. 클라우드 제공자 자체의 장애, 지역적 재해, 서비스 약정 불이행등의 리스크로부터 완전히 자유로울 수 없다. 이에 따라 우리는 멀티클라우드를 통한 재해복구 전략을 통하여 아래와 같은 이점을 취하고자 한다.

1. 가용성(Availability) : 한 클라우드 제공자 장애 시 타 클라우드 전환가능 =>
서비스 중단의 최소화
2. 복원력(Resilience): 자연재해, 시스템 장애, 사이버 공격 등 다양한 리스크에 대비한
인프라 이중화
3. 유연성(Flexibility): 각 클라우드의 강점을 전략적으로 활용 가능
4. 비즈니스 연속성(Business Continuity): 예기치 못한 상황에서도 핵심 서비스 유지

주 클라우드는 **AWS**로 설정하여 서울리전에서 동작하며, 보조 클라우드는 **GCP**로 미국 리전에서 동작한다.

평상 시에는 **AWS**의 인프라만 동작하다가 **Route53**에 의한 헬스체크가 **failover** 되면 보조 **DNS**인 **GCP**의 인프라가 **apply**되면서 동작하게 된다.

위와 같은 방식으로 재해복구 작업이 이루어지게 되는데 이에 앞서 이를 구축하기 위해서 최우선적인 요소 중 하나가 **DB**의 동기화이다. 이번 프로젝트에서는 비용을 최소화 시켜서 인프라를 구축하고자 하였고 그에 따라 생각한 것은 **mysqldump**와 **cron**을 이용한 주기적인 버킷을 통한 **DB Dump**의 복제이다. 해당 목차에서는 **Route53**을 통한 **DNS** 기반 트래픽 전환과 **DB** 동기화에 대한 코드와 자세한 서술을 기술한다.

8-1.Route 53 Failover 기반 트래픽 전환

우선 가장 먼저 트래픽전환을 위한 기준이 되는 헬스체크(상태 확인)을 구축하였다.

The screenshot shows the AWS Route 53 console interface for configuring a health check. The breadcrumb navigation at the top reads: Route 53 > 상태 확인 > 상태 확인 생성. The main section is titled '구성' (Configuration). Below the title, there is a note about costs: '구성 옵션은 비용에 영향을 미칩니다. 요금은 선택한 리소스 유형에 따라 달라집니다. 고급 구성에는 추가 비용이 부과됩니다. 요금 보기 [?]'.

The '이름 - 선택 사항' (Name - Optional) section shows the name 'primary-ingress' entered in a text box. Below it, a note states: '이름은 1~256자여야 합니다. 유효한 문자는 A-Z, a-z, 0-9, -(하이픈), _(밑줄)입니다.'

The '리소스' (Resource) section has three radio button options: '엔드포인트' (Endpoint) is selected, '계산된 상태 확인' (Calculated health check) is unselected, and 'CloudWatch 경보' (CloudWatch alarm) is unselected. The '엔드포인트' option has a sub-note: '리소스와 연결을 설정하여 리소스의 상태를 결정합니다.'

The '엔드포인트 지정 기준' (Endpoint specification) section has two radio button options: 'IP 주소' (IP address) is unselected, and '도메인 이름' (Domain name) is selected.

The '도메인 이름' (Domain name) section has a note: '경로는 엔드포인트가 정상일 때 엔드포인트가 2xx 또는 3xx의 HTTP 상태 코드를 반환하는 모든 값이 될 수 있습니다.' Below this, there is a dropdown menu set to 'HTTP' and a text box containing the URL 'k8s-default-lovebrid-51d22eeddc-1670374479.ap-northeast-2.elb.amazonaws.com'.

At the bottom, there is a link labeled '고급 구성' (Advanced configuration).

상태확인명은 **primary-ingress**로 현재 프로젝트에서 **ingress**로 관리중인 **alb** 주소를 엔드포인트 도메인명으로 사용하였다.

해당 **alb** 주소를 헬스체크 하여 **failover**가 발생할 시 **unhealthy**를 띄우게 된다.

상태확인을 구축한 후에는 **Route53** 호스팅 영역에서 **Faliover** 라우팅 정책을 생성하였다.

레코드 세부 정보

레코드 편집

값

k8s-default-lovebrid-51d22eeddc-1204835103.ap-northeast-2.elb.amazonaws.com.

라우팅 정책

Failover

레코드 ID

primary

레코드 이름

lovebridge.click

별칭

예

장애 조치 레코드 유형

기본

레코드 유형

A

TTL(초)

-

상태 확인 ID

1e40e149-c22a-478a-b297-22af379dacb6

정상적인 상황에서 구동할 **Primary** 레코드 설정이다.

상태확인 설정에서 등록된 **ingress** 주소와 동일한 주소를 사용하는 **lovebridge.click** 기본 도메인 설정이다. 라우팅정책에서 장애조치를 추가하고 **primary** 레코드로 등록하였다.

다음으로 진행할 것은 **Route53** 장애조치 정책의 보조 레코드를 등록해야한다.

현 프로젝트에서는 멀티클라우드를 통한 **aws<->gcp** 간의 재해복구를 구성하고 있으므로 **gcp**에서 **static-ip**를 받아와 등록해야한다.

☐	-	35.226.59.158	외부	us-east-1	...
☐	gke-ingress-ip	34.136.42.42	외부	us-east-1	...
☐	nat-auto-ip-30214162-13-1753066359180330	34.16.28.123	외부	us-east-1	...
☐	sql-private-range	10.163.0.0	내부		
☐	web-static-ip	34.8.53.178	외부		...

페이지당 행 수: 50 ▼

1 - 19 (전체 19행)

< >

gcp의 static-ip가 34.8.53.178 임을 확인하였다.

레코드 세부 정보

레코드 편집

레코드 이름	lovebridge.click	레코드 유형	A
값	34.8.53.178	TTL(초)	300
라우팅 정책	Failover	장애 조치 레코드 유형	보조
		레코드 ID	secondary-gcp

34.8.53.178 을 Failover 보조 레코드로 등록하였다.

기본적인 장애조치를 위한 설정이 끝났으므로 정상적으로 작동하는지 테스트해보았다.

```
"alb.ingress.kubernetes.io/healthcheck-path" = "/fail"
```

확인을 위해 고의적으로 Primary 에 등록된 ingress의 헬스체크 경로를 고장내보았다.

상태 확인 (2) Info

상태 및 경보 마지막 업데이트: 1분 전

상태 확인 생성

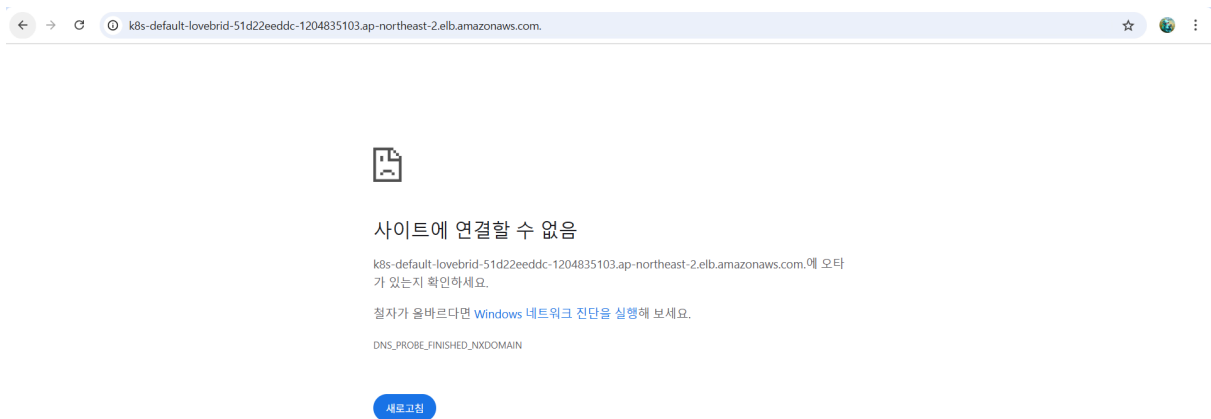
Route 53 상태 확인은 애플리케이션 서버 및 엔드포인트의 상태 및 성능을 모니터링합니다.

Q 상태 확인 검색

< 1 >

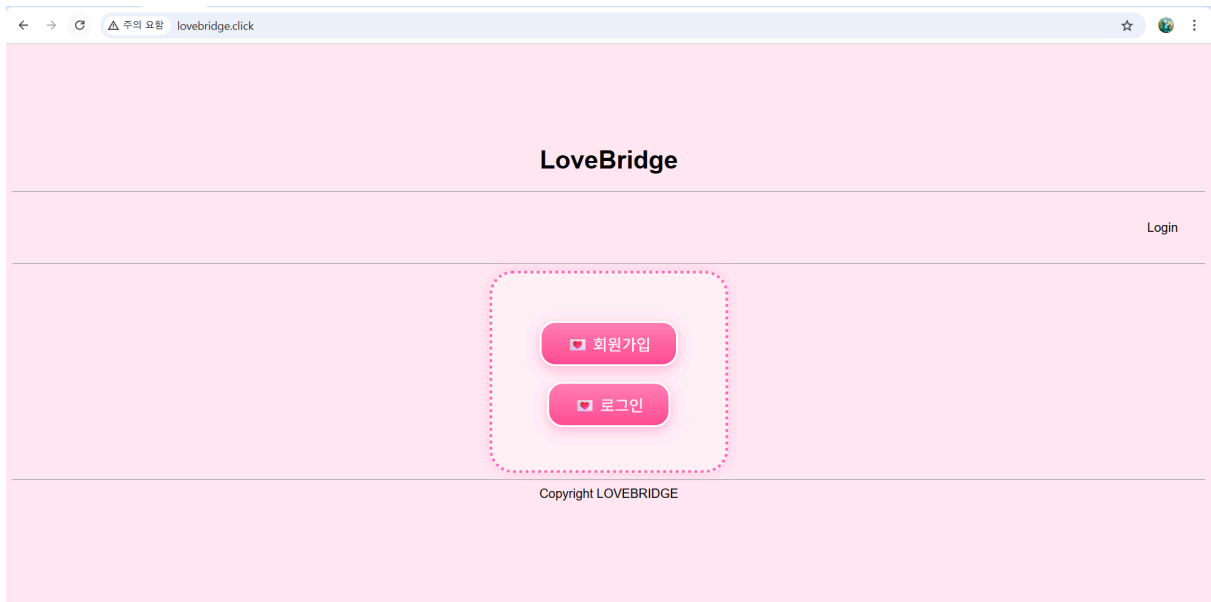
☐	ID	이름	세부 정보	지난 24시간 내 상태	현재 상태	경보	작업
☐	1e40e149-c22...	primary-i...	http://k8s-d...		비정상	없음, 경보 생성	:

상태 확인 결과가 비정상으로 뜬 것을 확인할 수 있다.



해당 로드밸런서 주소로 도메인을 들어갔을 때 에러가 나는 것을 확인하였다.

그렇다면 Route53에 등록된 lovebridge.click으로 들어갔을 경우를 확인해보겠다.



정상적으로 화면이 띄워진 것을 확인하였다.

```
Microsoft Windows [Version 10.0.26100.4652]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\victo>nslookup lovebridge.click  
서버:      bns1.hananet.net  
Address:  210.220.163.82
```

```
권한 없는 응답:  
이름:      lovebridge.click  
Address:  34.8.53.178
```

```
C:\Users\victo>
```

nslookup으로 lovebridge.click 을 탐색했을 때 google static ip 로 정상적인 트래픽 전환이 이루어진 것을 확인할 수 있었다.

8-2. DB 동기화

DB 동기화 작업에 대한 간략적인 시스템 아키텍처 개요는 아래와 같다.

주 클라우드 : AWS

운영 DB: Amazon RDS (Mysql)

백업 자동화 실행 환경 : Docker Container

인증 : GCP 서비스 계정 키(gcp-key.json)

보조 클라우드 : GCP

저장소 : Google Cloud Storage(GCS)

복구 대상: GCP Cloud Sql

```

{} gcp-keyjson > ...
1  {
2    "type": "service_account",
3    "project_id": "direct-tribute-463400-f2",
4    "private_key_id": "4d1fa91d62f5e4f46e0abde49f4fd5fcefef2aa4",
5    "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKggggSkAgEAAoIBAQDk/U3ciNd7r0Em\n\nVovh7/cwRx/2Ejp
6    "client_email": "rds-dump-981@direct-tribute-463400-f2.iam.gserviceaccount.com",
7    "client_id": "100686582936327892027",
8    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
9    "token_uri": "https://oauth2.googleapis.com/token",
10   "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
11   "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/rds-dump-981%40direct-tribute-463400-f2.iam.gservice
12   "universe_domain": "googleapis.com"
13 }
14 |

```

AWS에서 GCP의 스토리지로 파일을 전송하는 인증에 필요한 gcp-key.json이다.

```

$ backup.sh
1  #!/bin/bash
2
3  LOG_FILE="/tmp/backup.log"
4  TIMESTAMP=$(date +%Y%m%d%H%M%S)
5  FILENAME="rds-backup.sql"
6
7  echo "📅 $(date) -- 백업 시작" | tee -a "$LOG_FILE"
8
9  # 환경변수 확인
10 echo "🔍 환경변수 확인:" | tee -a "$LOG_FILE"
11 echo " DB_HOST=$DB_HOST" | tee -a "$LOG_FILE"
12 echo " DB_USER=$DB_USER" | tee -a "$LOG_FILE"
13 echo " DB_NAME=$DB_NAME" | tee -a "$LOG_FILE"
14 echo " GCP_BUCKET_NAME=$GCP_BUCKET_NAME" | tee -a "$LOG_FILE"
15 echo " GOOGLE_APPLICATION_CREDENTIALS=$GOOGLE_APPLICATION_CREDENTIALS" | tee -a "$LOG_FILE"
16
17 # 환경변수 sanitize (줄바꿈 제거)
18 DB_NAME=$(echo "$DB_NAME" | tr -d '\r\n')
19
20 # gcloud 인증
21 echo "👤 GCP 서비스 계정 인증 중..." | tee -a "$LOG_FILE"
22 gcloud auth activate-service-account --key-file="$GOOGLE_APPLICATION_CREDENTIALS" 2>&1 | tee -a "$LOG_FILE"
23 if [ $? -ne 0 ]; then
24   echo "❌ GCP 인증 실패!" | tee -a "$LOG_FILE"
25   exit 1
26 fi
27
28 # mysqldump 실행
29 echo "📦 mysqldump 실행 중..." | tee -a "$LOG_FILE"
30 mysqldump --single-transaction --routines --triggers --column-statistics=0 \
31   -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" > "$FILENAME"

```

해당 키는 추후에 인코딩되어 쿠버네티스의 yml파일에 직접 들어가게 된다.

```

32 |
33 |
34 | # 덤프 파일 확인
35 | if [ ! -f "$FILENAME" ]; then
36 |     echo "❌ mysqldump 실패: $FILENAME 생성 안됨" | tee -a "$LOG_FILE"
37 |     exit 1
38 | fi
39 |
40 | FILESIZE=$(stat -c%s "$FILENAME")
41 | if [ "$FILESIZE" -lt 3000 ]; then
42 |     echo "⚠️ mysqldump 결과 비정상적으로 작음 ($FILESIZE bytes)" | tee -a "$LOG_FILE"
43 |     head -n 20 "$FILENAME" | tee -a "$LOG_FILE"
44 | fi
45 |
46 | # gsutil 업로드
47 | echo "☁️ gsutil 업로드 시작..." | tee -a "$LOG_FILE"
48 | echo "📁 명령어: gsutil cp $FILENAME gs://$GCP_BUCKET_NAME/" | tee -a "$LOG_FILE"
49 |
50 | gsutil -h "Cache-Control:no-cache" cp "$FILENAME" gs://$GCP_BUCKET_NAME/ 2>&1 | tee -a "$LOG_FILE"
51 | STATUS=${PIPESTATUS[0]}
52 |
53 | if [ "$STATUS" -ne 0 ]; then
54 |     echo "❌ gsutil 업로드 실패! 상태코드: $STATUS" | tee -a "$LOG_FILE"
55 |     exit 1
56 | fi
57 |
58 | # 업로드 결과 확인
59 | echo "🔍 현재 버킷 상태:" | tee -a "$LOG_FILE"
60 | gsutil ls -l gs://$GCP_BUCKET_NAME/ | tee -a "$LOG_FILE"
61 |
62 | echo "✅ 백업 완료: $FILENAME ($FILESIZE bytes)" | tee -a "$LOG_FILE"

```

위 코드는 mysqldump 작업을 실질적으로 수행해주는 [backup.sh](#) 파일이다.

라인 별 설명을 덧붙이도록 하겠다.

3 : 로그 파일 경로 지정. tee 명령을 통해 이 파일에 로그를 누적 기록.

4~5: 타임스탬프 및 백업 파일 이름 설정

7: 백업 시작 로그. tee는 stdout과 로그 파일에 동시에 출력.

10~15: 환경변수 출력 (디버깅 목적, 실행 시 실제 변수값 확인)

18 : DB_NAME에 줄바꿈(\n\r) 있을 경우 제거 (환경변수 처리 오류 방지)

21~22 : GCP 서비스 계정 키(json)를 이용한 인증.

이는 gcloud CLI가 GCS 작업을 수행 가능하게 함.

GOOGLE_APPLICATION_CREDENTIALS 환경변수는 key-file 경로를 나타냄.

23~26 : \$?는 직전 명령어의 종료 상태(exit code)를 의미합니다. 0이 성공, 그 외는 실패입니다.

인증 실패 시 스크립트를 즉시 종료(exit 1).

29~31 : mysqldump 실행: --single-transaction은 InnoDB에서 덤프 일관성을 보장,
--routines과 --triggers는 저장 프로시저 및 트리거까지 백업 포함,
--column-statistics=0은 MySQL 8.x의 경고 방지 (불필요한 메타데이터 비활성화).

35~38 : mysqldump 결과 파일 존재 여부 확인. 없으면 명백한 실패이므로 즉시 종료.

40~44 : 파일 사이즈 측정. 3KB 이하이면 비정상 백업 가능성이 있어 경고 및 미리보기 출력.

stat -c%s는 리눅스에서 파일 크기를 byte 단위로 출력.

47~48 : gsutil 명령어로 백업 파일을 GCS에 업로드.

-h "Cache-Control:no-cache"는 파일을 캐시하지 않도록 설정해 항상 최신 파일로 유지.

50~51 : PIPESTATUS 배열은 파이프라인 명령의 각 단계 종료코드를 저장.

여기서는 gsutil 명령의 종료코드를 \${PIPESTATUS[0]}로 확인.

53~56 : 상태코드가 0이 아니면 업로드 실패로 간주하고 종료.

59~60 : 업로드 완료 후 현재 버킷 내 파일 목록을 출력하여 업로드 여부를 확인.

62 : 최종 성공 메시지 출력. 파일명과 사이즈를 함께 표기하여 확인 가능.

```

1 FROM ubuntu:20.04
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 # 기본 도구 설치
6 RUN apt-get update && apt-get install -y \
7     mysql-client \
8     curl \
9     gnupg \
10    unzip \
11    lsb-release \
12    python3-pip \
13    gpg
14
15 # gcloud CLI 설치 (정석 방식, apt-key 제거 대응)
16 RUN curl -sSL https://packages.cloud.google.com/apt/doc/apt-key.gpg \
17     | gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg && \
18     echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] http://packages.cloud.google.com/apt cloud-sdk main" \
19     > /etc/apt/sources.list.d/google-cloud-sdk.list && \
20     apt-get update && apt-get install -y google-cloud-sdk
21
22 ENV PATH="/usr/lib/google-cloud-sdk/bin:$PATH"
23
24 WORKDIR /app
25
26 COPY backup.sh .
27
28 RUN chmod +x backup.sh
29

```

위 코드는 도커 이미지 빌드를 위한 도커파일이다.

해당 도커파일을 통해 도커 이미지를 ECR에 올린 후, kubernetes에서 cronjob을 실행해 dump파일을 구글 버킷으로 전송한다.

1: 베이스 이미지로 Ubuntu 20.04 LTS를 사용한다.

3: APT 설치 시 사용자 입력(prompt)을 비활성화하여 무인 설치가 가능하다.

6~13 : 기본 도구를 설치하는 파트이다.

16~17 : gcloud CLI 설치하고, Google Cloud의 공개 GPG 키를 안전하게 다운로드 받고 binary 형식으로 변환하여 저장한다.

apt-key는 더 이상 권장되지 않기 때문에 keyrings 디렉토리에 직접 키를 저장한다.

18~19 : google-cloud-sdk의 APT 저장소를 system sources.list.d에 등록하고, 저장소 접근 시 위에서 저장한 GPG 키로 서명 검증한다.

20: 등록된 저장소로부터 gcloud CLI를 설치한다. (gsutil 포함됨)

22 : gcloud, gsutil 등 명령어가 포함된 디렉토리를 PATH에 추가하여 어디서든 실행 가능하다.

24: 이후 명령어 실행 경로를 `/app` 디렉토리로 설정하다.

26: 현재 디렉토리에 있는 backup.sh 파일을 컨테이너의 /app 디렉토리에 복사한다.

28: backup.sh 파일에 실행 권한 부여 (RUN 명령어는 Docker 빌드 시 실행한다.

30: 컨테이너가 실행될 때 자동으로 실행되는 명령어를 지정한다.

이 경우 `docker run` 또는 `kubectl run`으로 실행 시 바로 backup.sh 스크립트가 실행되다.

해당 파일들을 로컬 Docker Desktop에서 빌드한 후, Ecr에 태그를 붙여, 푸시하게 되면 준비는 끝난다.

```
PS C:\Users\victo\kuber\rds-dump> aws ecr get-login-password --region ap-northeast-2 | docker login --username AWS --password-stdin 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com
>>
Login Succeeded
```

```
PS C:\Users\victo\kuber\rds-dump> docker build -t newworld .
[+] Building 148.1s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                 0.0s
=> => transferring dockerfile: 854B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04     2.5s
=> [auth] library/ubuntu:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                       0.1s
=> [1/6] FROM docker.io/library/ubuntu:22.04@sha256:3c61d3759c2639d4b836d32a2d3c83fa0214e36f195a3421018dbaaf79cbe37f  5.3s
=> => resolve docker.io/library/ubuntu:22.04@sha256:3c61d3759c2639d4b836d32a2d3c83fa0214e36f195a3421018dbaaf79cbe37f  0.0s
=> => sha256:1b668a2d748d748b0460ac95024ec80af7b663ede9416c235a9c102556bc1780 2.30kB / 2.30kB  0.0s
=> => sha256:e735f3a6b70199ad991c5715d965a4d858540eca2be18be0d889698e5a0a3e8c 29.54MB / 29.54MB  3.8s
=> => sha256:3c61d3759c2639d4b836d32a2d3c83fa0214e36f195a3421018dbaaf79cbe37f 6.69kB / 6.69kB  0.0s
=> => sha256:08e2cd26ee66d0d46d6394df594f2877fc9b9381d9630a9ef5d86e27dfae9a95 424B / 424B  0.0s
=> => extracting sha256:e735f3a6b70199ad991c5715d965a4d858540eca2be18be0d889698e5a0a3e8c 1.2s
=> [internal] load build context                                    0.1s
=> => transferring context: 31B                                       0.0s
=> [2/6] RUN apt-get update && apt-get install -y mysql-client curl gnupg unzip lsb-release python3-pip gpg 81.9s
=> [3/6] RUN curl -sSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg &  51.2s
=> [4/6] WORKDIR /app                                              0.1s
=> [5/6] COPY backup.sh .                                         0.1s
=> [6/6] RUN chmod +x backup.sh                                    0.4s
=> exporting to image                                              6.3s
=> => exporting layers                                              6.3s
=> => writing image sha256:28560fdc974b2bf2099b2ca70fba38ab00687816311d98a2321ee0ad389088d9 0.0s
=> => naming to docker.io/library/newworld                          0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/up0cupgmm0rjqg87s8golw5in
PS C:\Users\victo\kuber\rds-dump> docker images
REPOSITORY TAG IMAGE ID aws ecr get-login-password --region ap-northeast-2 \
>> | docker login --username AWS --password-stdin 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com
```

ECR에 로그인

Docker Desktop에 빌드해서 올린다.

```
PS C:\Users\victo\kuber\rds-dump> docker tag newworld:latest 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin:newworld2
PS C:\Users\victo\kuber\rds-dump> docker push 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin:newworld2
The push refers to repository [680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin]
68d687c2ff84: Pushed
e60c0cfd453d: Pushed
9b1c336e7a00: Pushed
0b96d4b30851: Pushed
8d6b7eb76b62: Layer already exists
newworld2: digest: sha256:726cb0756b55cc06c273820c75c7ff8ca97d2035559cef030dc37d42e604927d size: 1577
PS C:\Users\victo\kuber\rds-dump>
```

Docker tag를 ECR에 붙이고 ECR로 push한다.

☐	newworld	Image Index	2025년 7월 01일, 14:50:04 (UTC+09)	405.28	 URI 복사	 sha256:e05129066f35cd...	2025년 7월 07일, 15:39:27 (UTC+09)
☐	-	Image	2025년 7월 01일, 14:50:04 (UTC+09)	0.00	 URI 복사	 sha256:2a68e6fb417270...	2025년 7월 01일, 14:50:04 (UTC+09)
☐	-	Image	2025년 7월 01일, 14:50:04 (UTC+09)	405.28	 URI 복사	 sha256:7a56e5f9606a86...	2025년 7월 07일, 15:39:27 (UTC+09)

ECR에 정상적으로 도커이미지가 올라간 것을 볼 수 있다.

DB의 덤프파일이 담긴 **Docker** 이미지를 정상적으로 **ECR** 까지 올리는 것에 성공하였다.
그 다음 단계는 쿠버네티스를 이용하여서 **ECR**에서 빌드 파일을 받아와 구글 스토리지에
전송하는 것이다.

```

! cronjob.yml > ...
1  apiVersion: batch/v1
2  kind: CronJob
3  metadata:
4    name: rds-dump-upload
5  spec:
6    schedule: "*/1 * * * *"
7    jobTemplate:
8      spec:
9        template:
10         spec:
11         containers:
12         - name: dumper
13           image: 680993828418.dkr.ecr.ap-northeast-2.amazonaws.com/project2-server-origin:newworld
14           env:
15             - name: DB_HOST
16               valueFrom:
17                 configMapKeyRef:
18                   name: dump-config
19                   key: DB_HOST
20             - name: DB_NAME
21               valueFrom:
22                 configMapKeyRef:
23                   name: dump-config
24                   key: DB_NAME
25             - name: DB_USER
26               valueFrom:
27                 secretKeyRef:
28                   name: dump-secret
29                   key: DB_USER
30             - name: DB_PASS
31               valueFrom:
32                 secretKeyRef:
33                   name: dump-secret
34                   key: DB_PASS
35             - name: GCP_BUCKET_NAME
36               valueFrom:
37                 configMapKeyRef:

```

```

33         name: dump-secret
34         key: DB_PASS
35     - name: GCP_BUCKET_NAME
36       valueFrom:
37         configMapKeyRef:
38           name: dump-config
39           key: GCP_BUCKET_NAME
40     - name: GOOGLE_APPLICATION_CREDENTIALS
41       value: /var/secrets/gcp/gcp-key.json
42     volumeMounts:
43     - name: gcp-creds
44       mountPath: /var/secrets/gcp/gcp-key.json
45       subPath: gcp-key.json
46       readOnly: true
47
48     volumes:
49     - name: gcp-creds
50       secret:
51         secretName: gcp-sa
52       restartPolicy: OnFailure
53
54
55

```

cronjob.yml

6 : 1분을 주기로 실행한다.

13 : 이미지를 받아오는 주소가 작성 되어 있다. 백업 이미지가 올라가있는 ECR의 주소이다.

14~41 : Config와 Secret에서 환경변수 값들을 가져온다. 아래 기술.

42~46 : GCP 서비스 계정 키를 컨테이너 내부 경로로 마운트한다.

46~52 : 볼륨 정의 - Kubernetes Secret으로부터 GCP 키를 불러온다.

```
! dump-config.yml > ...
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
1
2 apiVersion: v1
3 kind: ConfigMap
4 metadata:
5   name: dump-config
6 data:
7   DB_HOST: mariadb-2a.c566mcwwy70y.ap-northeast-2.rds.amazonaws.com
8   DB_NAME: lovebridgedb
9   GCP_BUCKET_NAME: lovebridge-db-backups
10
11
12
13
```

debug-config.yml

cronjob.yml에서 사용될 RDS의 호스트 주소와 사용할 DB의 이름, 그리고 파일을 전송할 GCP의 스토리지 이름이 작성되어 있다.

```
! dump-secret.yml > apiVersion
io.k8s.api.core.v1.Secret (v1@secret.json)
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: dump-secret
5    type: Opaque
6  stringData:
7    DB_USER: root
8    DB_PASS: rhdwn9953!
```

dump-secret.yml

cronjob.yml에서 사용될 RDS 사용자의 유저네임과 비밀번호가 작성되어 있다.

```
! gcp-sa.yml > {} data
io.k8s.api.core.v1.Secret (v1@secret.json)
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: gcp-sa
5    type: Opaque
6  data:
7    gcp-key.json: ewogICJ0eXB1IjogInNlcnZpY2VfYmNjb3VudCIsCiAgInByb2plY3RfawQioiAiZGlyZWNOXRYawJ1dGUtNDYzNDAwLWYyIiwKICAicHJpdmF0Z
8
```

gcp-sa.yml

GCP로의 인증을 위해 필요한 `gcp-key.json`이 Base64로 인코딩되어 있다.


```

$ import.sh
1  #!/bin/bash
2
3  # 인증
4  gcloud auth activate-service-account --key-file="$GOOGLE_APPLICATION_CREDENTIALS"
5
6  # Cloud SQL import (프롬프트 없이 자동 진행)
7  gcloud sql import sql "$INSTANCE_NAME" \
8  "gs://$GCP_BUCKET_NAME/rds-backup.sql" \
9  --database="$DB_NAME" \
10 --project="$PROJECT_ID" \
11 --quiet
12
13
14
15
16
17 |

```

[import.sh](#)

GCP 스토리지에 올라와 있는 mysqldump 백업 파일을 주기적으로 다운 받아서 CloudSql의 DB에 덮어씌우는 sh 파일이다.

```

📁 Dockerfile
1  # Dockerfile
2  FROM google/cloud-sdk:slim
3
4  COPY import.sh /import.sh
5  RUN chmod +x /import.sh
6
7  ENTRYPOINT ["/import.sh"]
8

```

도커이미지를 빌드하기 위한 Dockerfile 이다.

```

! cronjob.yml > {} spec > {} jobTemplate > {} spec > {} template > {} spec > [ ] volumes > {} 0 > {} secret > {} secretName
1  apiVersion: batch/v1
2  kind: CronJob
3  metadata:
4    name: cloudsql-import-job
5  spec:
6    schedule: "*/1 * * * *" # 매 1분마다
7    jobTemplate:
8      spec:
9        template:
10          spec:
11            containers:
12              - name: importer
13                image: us-central1-docker.pkg.dev/direct-tribute-463400-f2/docker/gcp-dump-docker:newgcp
14                env:
15                  - name: GOOGLE_APPLICATION_CREDENTIALS
16                    value: /secrets/gcp/gcp-key.json
17                  - name: INSTANCE_NAME
18                    valueFrom:
19                      configMapKeyRef:
20                        name: import-config
21                        key: INSTANCE_NAME
22                  - name: PROJECT_ID
23                    valueFrom:
24                      configMapKeyRef:
25                        name: import-config
26                        key: PROJECT_ID
27                  - name: DB_NAME
28                    valueFrom:
29                      configMapKeyRef:
30                        name: import-config
31                        key: DB_NAME
32                  - name: GCP_BUCKET_NAME
33                    valueFrom:
34                      configMapKeyRef:
35                        name: import-config
36                        key: GCP_BUCKET_NAME
37                volumeMounts:
38                  - name: gcp-key
39                    mountPath: /secrets/gcp
40                    readOnly: true
41                restartPolicy: OnFailure
42            volumes:
43              - name: gcp-key
44                secret:
45                  secretName: gcp-sa
46

```

GCP 쪽의 `cronjob.yml`이다.

1 : 리소스 버전을 지정한다.

2: 리소스의 종류를 지정한다. 여기서는 정기적인 `job`을 실행하는 `Cronjob`이다.

3~4 : 리소스의 식별 정보이다. `name` 은 해당 `cronjob`의 이름이고 클러스터 내에서 고유하다.

6: `cron`의 주기를 매 1분으로 설정하였다.

7~10 : `Cronjob`이 생성할 실제 `job`의 템플릿을 정의한다.

내부의 `spec.template.spec` 은 `pod`의 사양을 정의한다.

11~13 : `dump` 작업을 수행할 커스텀 이미지이다.

14~36 : 환경변수 설정들을 담고 있다. 인증을 위한 `gcp-key`와 인스턴스 이름, 프로젝트 `id`, 버킷 이름, `DB` 이름이 작성되어 있다.

37~40 : `GCP` 키의 마운트에 관한 설정이 담겨져 있다.

42~45 : `Pod`에서 사용할 볼륨을 정의하고 있다.

```

! import-config.yml > {} data > DB_NAME
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: import-config
5  data:
6    INSTANCE_NAME: lovebridge-instance
7    PROJECT_ID: direct-tribute-463400-f2
8    DB_NAME: lovebridge_main
9    GCP_BUCKET_NAME: lovebridge-db-backups
10

```

```

! secret.yml > apiVersion
io.k8s.api.core.v1.Secret (v1@secret.json)
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: gcp-sa
5  type: Opaque
6  data:
7    gcp-key.json: ewogICJ0eXB1IjogInN1cnZpY2VfYmNjb3VudCIsCiAgInByb2p1Y3RfawQioiAiZGlyZWNN0LXRyawJ1dGUTNDYzNDAwLWYyIiwKICAichJpdmF0Z
8
9

```

importe-config 에서는 cronjob이 실행될 DB의 인스턴스와 DB명, 프로젝트 id, gcp 버킷의 이름을 정의하고 있다.

gcp-sa는 aws와 같이 인증에 필요한 키를 정의하고 있다.

9. 모니터링 및 운영

본 프로젝트에서는 효율적이고 가시성 높은 모니터링 시각화를 위해 외부 모니터링 도구인 **Prometheus** 와 **Grafana**를 사용하였다.

해당 모니터링 도구들을 사용했을 때의 이점들은 다음과 같다.

1. 실시간 모니터링 가능

Prometheus는 시계열 기반 데이터를 수집하고 **15초~1분** 단위로 실시간 상태를 기록한다.

Grafana는 이 데이터를 실시간으로 시각화하여 지연 없이 상태 파악 가능하다.

2. Kubernetes 친화적

kube-state-metrics, node-exporter 등 다양한 Kubernetes용 **Exporter**을 지원한다.

Pod 상태, CPU/메모리, 노드 자원 사용량 등을 자동 수집하고 시각화 가능하다.

3. 오픈소스 기반, 효율적 비용

상용 솔루션(**Grafana Cloud**, **Datadog** 등) 대비 비용 절감이 된다.

모든 구성 요소(**Prometheus**, **Grafana**, **Alertmanager** 등)를 자체 구축 가능하다.

4. 강력한 쿼리 기능 (PromQL)

Prometheus는 자체 쿼리 언어(**PromQL**)를 통해 정밀한 지표 분석 및 조건 검색 가능하다.

예: 특정 Pod의 5분간 평균 CPU 사용률, 특정 라벨을 가진 리소스만 필터링 등

5. 유연한 알림 연동

Alertmanager와 연동하여 **Slack**, **Email**, **Webhook** 등으로 자동 알림 발송 가능하다.

CPU 과다 사용, Pod 다운 등의 조건을 지정해 자동 경고 트리거 설정 가능하다.

6. 대시보드 공유 및 커스터마이징

Grafana는 다양한 템플릿, 테마, 플러그인을 통해 팀 단위 공유 및 맞춤형 대시보드 구성 가능하다.

링크, 스냅샷, 퍼블릭 공유 등 보고용 자료로 전환도 쉽다.

7. 다양한 데이터 소스 지원

Prometheus 외에도 MySQL, PostgreSQL, Loki, ElasticSearch 등 멀티 소스 통합 시각화 가능하다.

DevOps, DB팀, 보안팀 등 각 팀별로 필요한 데이터를 한 화면에서 확인 가능하다.

본 프로젝트에서는 aws와 gcp 멀티 클라우드 환경에서 각기 다른 방식으로 모니터링을 환경을

구축해보았다. aws 환경에서는 helm을 이용하여 Prometheus&Grafana 통합 패키지를 다운받아서

진행하였고 이를 기반으로 ingress를 생성해 grafana pod에 ip를 붙이고 이를 Route53에 grafana.lovebridge.click을 생성하여 외부 도메인에서 확인 가능하도록 설정하였다.

gcp 환경에서는 gke에서 yml파일을 직접 작성하여 Prometheus&Grafana를 배포하였고 로컬 환경에서 포트포워딩을 통해 localhost 에서 확인해보는 작업을 거쳤다.

9-1. AWS monitoring

AWS 환경에서는 Kubernetes 클러스터의 상태를 직관적으로 파악하고, 시스템 자원 사용량을 실시간으로 모니터링할 수 있도록 하기 위해 Prometheus와 Grafana를 도입하였다. 설치 과정에서는 Helm을 활용하여 복잡한 설정 없이 손쉽게 모니터링 스택을 배포할 수 있도록 하였다.

먼저, Prometheus 및 Grafana의 Helm 차트를 제공하는 외부 저장소를 추가하고 최신 상태로 갱신하였다. 이는 아래 명령어를 통해 수행되었다:

```
helm repo add prometheus-community
```

```
https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

그 후, kube-prometheus-stack이라는 통합 차트를 사용하여 Prometheus 서버, Alertmanager, Grafana 대시보드 등을 한 번에 설치하였다. 이 Helm 차트는 Kubernetes 클러스터의 주요 지표들을 자동으로 수집하고 시각화할 수 있도록 다양한 컴포넌트들을 미리 구성해 제공하는 것이 특징이다. Helm 배포 시 monitoring이라는 네임스페이스를 생성하며, 해당 네임스페이스에 모든 리소스가 함께 설치되도록 다음 명령어를 실행하였다.

```
helm install prometheus prometheus-community/kube-prometheus-stack -n monitoring  
--create-namespace
```

이 과정을 통해 Prometheus는 클러스터 내부의 메트릭 데이터를 수집하고, Grafana는 해당 데이터를 시각화하는 대시보드 역할을 수행하게 된다. 설치 후에는 Grafana에 자동으로 Prometheus가 데이터 소스로 연결되어 있으며, 다양한 Kubernetes 관련 대시보드가 기본으로 제공되어 있어 즉시 활용이 가능하다.

Helm을 활용함으로써 별도의 수작업 없이 빠르고 일관된 방식으로 모니터링 환경을 구축할 수 있었고, 향후 유지보수와 확장성 측면에서도 효율적인 인프라 운영이 가능해졌다.

```

1 grafana-ingress.yml
2 kind: Ingress
3 metadata:
4   name: grafana-ingress
5   namespace: monitoring
6   annotations:
7     alb.ingress.kubernetes.io/scheme: internet-facing
8     alb.ingress.kubernetes.io/target-type: ip
9     alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80}, {"HTTPS":443}]'
10    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:ap-northeast-2:680993828418:certificate/9f19d5d4-1ed0-47b7-b3d6-b1ac1f
11    alb.ingress.kubernetes.io/ssl-policy: ELBSecurityPolicy-2016-08
12    external-dns.alpha.kubernetes.io/hostname: grafana.lovebridge.click
13 spec:
14   ingressClassName: alb
15   rules:
16   - host: grafanas.lovebridge.click
17     http:
18       paths:
19       - path: /
20         pathType: Prefix
21         backend:
22           service:
23             name: prometheus-grafana
24             port:
25               number: 80
26

```

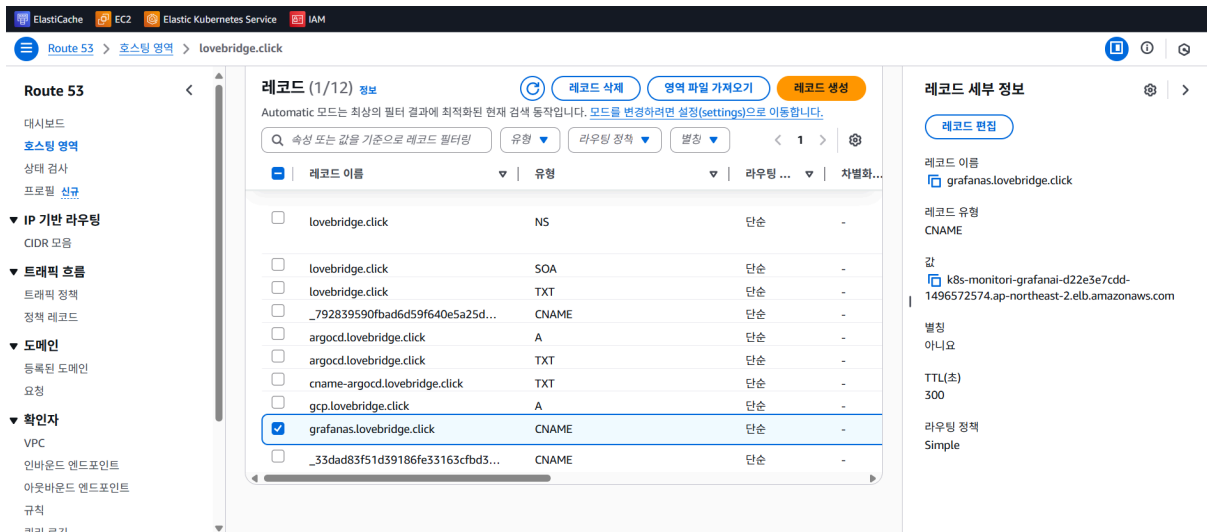
grafana-ingress.yml은 Kubernetes에서 AWS ALB(Application Load Balancer)를 활용해 Grafana 대시보드에 접근할 수 있도록 설정한 Ingress 리소스 정의이다. 리소스 이름은 grafana-ingress이며, monitoring 네임스페이스에서 동작한다.

IngressClassName은 alb로 지정되어 있으며, 이는 AWS Load Balancer Controller를 사용해 외부 ALB를 자동으로 생성하고 연결하는 역할을 한다.

metadata.annotations 부분에서는 ALB 설정을 상세히 지정하고 있는데, ALB는 퍼블릭 액세스를 위한 internet-facing 모드로 설정되어 있으며 (alb.ingress.kubernetes.io/scheme), 타겟 유형은 ip이다.

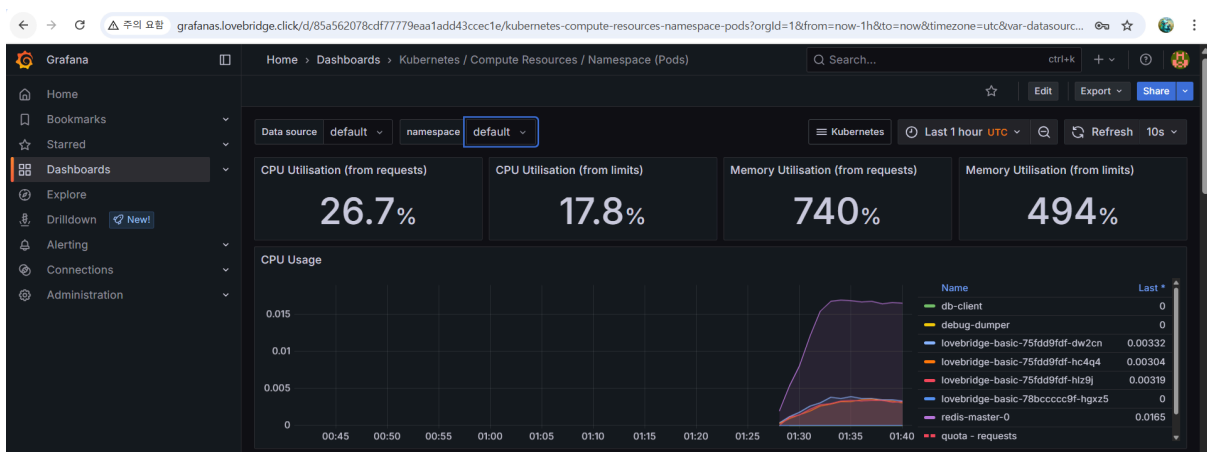
ALB는 HTTP(80) 및 HTTPS(443) 포트를 동시에 리스닝하도록 구성되어 있고, HTTPS 요청을 처리하기 위해 지정된 인증서 ARN과 보안 정책(ELBSecurityPolicy-2016-08)이 적용되어 있다. 또한 ALB의 DNS 이름은 grafana.lovebridge.click으로 명시되어 있어, 해당 도메인으로 접속 시 ALB를 통해 연결되도록 설정된다.

spec.rules에는 grafana.lovebridge.click 도메인에 대한 라우팅 규칙이 정의되어 있으며, 모든 경로(/) 요청은 monitoring 네임스페이스 내 prometheus-grafana라는 이름의 Kubernetes 서비스로 전달되고, 이 서비스는 내부적으로 80번 포트로 트래픽을 수신하게 되어 있다. 요약하면, 사용자가 https://grafana.lovebridge.click에 접속하면 외부 ALB를 거쳐 Kubernetes 내부의 Grafana 서비스로 안전하게 연결되도록 설정한 Ingress 리소스이다.



Route53 에서 grafana-ingress.yml에서 생성된 address를 route53의 값으로 입력하였다.
레코드명은 grafanas.lovebridge.click 이다.

제대로 적용되었는지 확인하기 위해서 grafanas.lovebridge.click 으로 접근해보았다.
grafanas.lovebridge.click으로 정상적인 접근이 되었음을 확인하였고 , 이후 대시보드를
통해서 kubernetes를 모니터링 시각화였다.



grafana에서 쿠버네티스의 디폴트 네임스페이스를 모니터링된 지표이다.
지표가 나타내는 바는 다음과 같다.

CPU Utilisation (from requests): 26.7%

→ 전체 파드들이 요청(request)한 CPU 자원 대비 실제로 사용 중인 비율이 26.7%로, 과도한 요청 없이 적절하게 사용되고 있음을 보여준다.

CPU Utilisation (from limits): 17.8%

→ 설정된 CPU limit 대비 사용률이 17.8%로, 자원 여유가 충분함을 나타낸다. 이는 오버프로비저닝이 있을 수 있다는 뜻이기도 하다.

Memory Utilisation (from requests): 740%

→ 파드들이 요청한 메모리 양보다 실제 사용량이 7배 이상 많다는 의미로, 메모리 요청 값이 지나치게 낮게 설정되었거나 파드의 사용 패턴이 급격하게 증가했음을 시사한다.

Memory Utilisation (from limits): 494%

→ 메모리 limit 값의 494%에 달하는 사용률로, 이는 실제로 OOMKilled(Out Of Memory)에 의해 파드가 종료되었을 가능성이 매우 높다는 경고 신호이다.

특정 시간대 이후 급격한 리소스 사용량의 상승이 관찰되는 것으로 보아, 이 시점에 부하가 집중되었거나 특정 작업(예: 배치, 로깅 등)이 발생했을 가능성이 있다.

이러한 정보들 외에 현재 실행 중인 파드들의 파드명인

db-client, worker-app, lovebridge-basic 이 구동 중인 것을 확인해볼 수 있다.

이와 같이 prometheus+grafana 조합을 helm으로 배포하고 이를 ingress + Route53을 통해 외부 도메인에서 접근해 관리 가능하게 함으로써 모니터링 환경을 구축하였다.

9-2. GCP monitoring

Gcp에서는 helm으로 간편하게 배포하는 대신 Prometheus / Grafana / Prometheus Exporters 로 모듈을 생성하여 kubernetes를 통해 yml을 배포하였다.

Prometheus Exporters

```
io.k8s.api.core.v1.Service (v1@service.json) | io.k8s.api.apps.v1.Deployment (v1@deployment.json) | io.k8s.api.core.v1.ServiceAccount (v1@serviceaccount.json)
1  apiVersion: v1
2  kind: ServiceAccount
3  metadata:
4    name: gcp-monitoring-exporter-sa
5    namespace: default
6    annotations:
7      iam.gke.io/gcp-service-account: gcp-monitoring-exporter@direct-tribute-463400-f2.iam.gserviceaccount.com
8  ---
9  apiVersion: apps/v1
10 kind: Deployment
11 metadata:
12   name: gcp-monitoring-exporter
13   namespace: default
14 spec:
15   replicas: 1
16   selector:
17     matchLabels:
18       app: gcp-monitoring-exporter
19   template:
20     metadata:
21       labels:
22         app: gcp-monitoring-exporter
23   spec:
24     serviceAccountName: gcp-monitoring-exporter-sa
25     containers:
26       - name: exporter
27         image: prometheuscommunity/stackdriver-exporter
28         args:
29           - "--google.project-id=direct-tribute-463400-f2"
30           - "--monitoring.metrics-type-prefixes=container.googleapis.com,compute.googleapis.com"
31         ports:
32           - containerPort: 9255
33     ---
34     apiVersion: v1
35     kind: Service
36     metadata:
37       name: gcp-monitoring-exporter
38       namespace: default
39     spec:
40       selector:
41         app: gcp-monitoring-exporter
42       ports:
43         - protocol: TCP
44           port: 9255
45           targetPort: 9255
46
```

- gcp-exporter.yml

Google Cloud Platform(GCP)의 다양한 서비스(예: GCE, Cloud SQL, GCS 등)에서 발생하는 메트릭을 Prometheus가 수집할 수 있는 형식으로 변환하여 노출시킨다.

```

monitor-k8s > ! gcp-exporter-service.yml > {} spec > [ ] ports > {} 0
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: gcp-monitoring-exporter
5    namespace: default
6  spec:
7    selector:
8      app: gcp-monitoring-exporter
9    ports:
10     - protocol: TCP
11       port: 9255
12       targetPort: 9255
13

```

- gcp-exporter-service.yml

클러스터 내부 또는 외부에서 gcp-monitoring-exporter라는 pod 또는 deployment 의 9255 포트로 접근할 수 있도록 정적 접점을 제공하는 리소스이다.

kube-state-metrics

kubernetes의 리소스 상태 정보를 수집하여 Prometheus에 노출되는 Exporter 이다.

해당 데이터들을 Grafana에 활용 가능하다.

```
monitor-k8s > kube-state > ! serviceaccount.yml > {} metadata
io.k8s.api.core.v1.ServiceAccount (v1@serviceaccount.json)
1  apiVersion: v1
2  kind: ServiceAccount
3  metadata:
4    name: kube-state-metrics
5    namespace: kube-system
6
```

serviceaccount.yml

kube-state-metrics가 API Server에 접근할 수 있는 주체를 정의한다.

```

monitor-k8s > kube-state > ! clusterrole.yml > [ ]rules > {} 3
io.k8s.api.rbac.v1.ClusterRole (v1@clusterrole.json)
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: ClusterRole
3  metadata:
4    name: kube-state-metrics
5  rules:
6    - apiGroups: ["" ]
7      resources:
8        - pods
9        - nodes
10       - namespaces
11       - services
12       - replicationcontrollers
13       - persistentvolumeclaims
14       - persistentvolumes
15     verbs: ["list", "watch"]
16   - apiGroups: ["apps"]
17     resources:
18       - statefulsets
19       - daemonsets
20       - deployments
21       - replicaset
22     verbs: ["list", "watch"]
23   - apiGroups: ["batch"]
24     resources:
25       - cronjobs
26       - jobs
27     verbs: ["list", "watch"]
28   - apiGroups: ["autoscaling"]
29     resources:
30       - horizontalpodautoscalers
31     verbs: ["list", "watch"]
32

```

clusterrole.yml

kube-state-metrics가 접근할 리소스를 읽기 전용(list,watch) 권한으로 정의한다.

```

monitor-k8s > kube-state > ! clusterrolebinding.yml > [ ] subjects > {} 0
io.k8s.api.rbac.v1.ClusterRoleBinding (v1@clusterrolebinding.json)
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: ClusterRoleBinding
3  metadata:
4    name: kube-state-metrics
5  roleRef:
6    apiGroup: rbac.authorization.k8s.io
7    kind: ClusterRole
8    name: kube-state-metrics
9  subjects:
10 - kind: ServiceAccount
11   name: kube-state-metrics
12   namespace: kube-system
13

```

clusterrolebinding.yml

위에서 정의한 clusterrole을 ServiceAccount와 연결한다.

kube-state-metrics가 실제로 API를 읽을 수 있는 권한을 얻게 된다.

```

monitor-k8s > kube-state > ! service.yml > {} spec > {} selector
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    labels:
5      app.kubernetes.io/name: kube-state-metrics
6      name: kube-state-metrics
7      namespace: kube-system
8  spec:
9    ports:
10 - name: http-metrics
11   port: 8080
12   targetPort: http-metrics
13 - name: telemetry
14   port: 8081
15   targetPort: telemetry
16  selector:
17    app.kubernetes.io/name: kube-state-metrics
18

```

service.yml

kube-state-metrics가 노출하는 메트릭 포트를 클러스터 내에서 접근할 수 있도록
kubernetes 서비스로 연결한다.

```

monitor-k8s > kube-state > ! deployment.yml > {} spec > {} template > {} spec > [ ] containers > {} 0 > {} securityContext > {} capabilities > [ ] drop
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    labels:
5      app.kubernetes.io/component: exporter
6      app.kubernetes.io/name: kube-state-metrics
7      app.kubernetes.io/version: 2.10.1
8    name: kube-state-metrics
9    namespace: kube-system
10 spec:
11   replicas: 1
12   selector:
13     matchLabels:
14       app.kubernetes.io/name: kube-state-metrics
15   template:
16     metadata:
17       labels:
18         app.kubernetes.io/component: exporter
19         app.kubernetes.io/name: kube-state-metrics
20         app.kubernetes.io/version: 2.10.1
21     spec:
22       automountServiceAccountToken: true
23       serviceAccountName: kube-state-metrics
24       containers:
25         - name: kube-state-metrics
26           image: registry.k8s.io/kube-state-metrics/kube-state-metrics:v2.10.1
27           args:
28             - --port=8080
29             - --telemetry-port=8081
30             - --resources=pods,nodes,deployments,statefulsets,namespaces

```

```

31     ports:
32       - name: http-metrics
33         containerPort: 8080
34       - name: telemetry
35         containerPort: 8081
36     livenessProbe:
37       httpGet:
38         path: /healthz
39         port: 8080
40       initialDelaySeconds: 5
41       timeoutSeconds: 5
42     readinessProbe:
43       httpGet:
44         path: /
45         port: 8081
46       initialDelaySeconds: 5
47       timeoutSeconds: 5
48     securityContext:
49       runAsNonRoot: true
50       runAsUser: 65534
51     readOnlyRootFilesystem: true
52     allowPrivilegeEscalation: false
53     capabilities:
54       drop: ["ALL"]
55     seccompProfile:
56       type: RuntimeDefault
57   nodeSelector:
58     kubernetes.io/os: linux
59

```


kube-state-metrics

애플리케이션의 실제 실행 컨테이너를 정의합니다.

Node-exporter

Node-exporter는 Kubernetes 노드의 시스템 메트릭을 Prometheus에 노출하는 Exporter이다.

각 노드의 CPU, Memory, Disk, Filesystem, Network I/O 같은 OS 레벨 정보를 수집해 Prometheus가 수집할 수 있게 한다.

```
monitor-k8s > node-export > ! serviceaccount-node-exporter.yml > {} metadata
io.k8s.api.core.v1.ServiceAccount (v1@serviceaccount.json)
1  apiVersion: v1
2  kind: ServiceAccount
3  metadata:
4    name: node-exporter
5    namespace: kube-system
6
```

serviceaccount-node-exporter.yml

Prometheus가 Kubernetes API를 통해 node-exporter Pod들을 검색하거나 메트릭을 가져올 수 있게 하기 위한 ServiceAccount 정의이다.

clusterrole-node-expoter.yml

node-exporter가 클러스터 노드와 서비스 정보들을 조회할 수 있는 역할이다.

```
monitor-k8s > node-export > ! clusterrolebinding-node-exporter.yml > [ ] subjects > {} 0
io.k8s.api.rbac.v1.ClusterRoleBinding (v1@clusterrolebinding.json)
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: ClusterRoleBinding
3  metadata:
4    name: node-exporter
5  roleRef:
6    apiGroup: rbac.authorization.k8s.io
7    kind: ClusterRole
8    name: node-exporter
9  subjects:
10 - kind: ServiceAccount
11   name: node-exporter
12   namespace: kube-system
13
14
```

clusterrolebinding-node-exporter.yml

위에서 만든 clusterrole을 ServiceAccount 에 바인딩한다.

```
monitor-k8s > node-export > ! service-node-exporter.yml > {} spec
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: node-exporter
5    namespace: kube-system
6  labels:
7    app: node-exporter
8  spec:
9    ports:
10 - name: http
11   port: 9100
12   targetPort: 9100
13  selector:
14    app: node-exporter
15  type: ClusterIP
16
```

node-exporter pod 에 실제로 리소스를 조회할 수 있는 권한이 주어진다.

service-node-exporter

service-node-exporter는 kubernetes 환경에서 Prometheus Node Exporter Pod에 접근할 수 있도록 하는 Kubernetes Service 리소스이다.

Prometheus

```
monitor-k8s > prometheus > ! pro-deploy.yml > {} spec > {} template
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: prometheus
5    namespace: default
6  spec:
7    replicas: 1
8    selector:
9      matchLabels:
10       app: prometheus
11  template:
12    metadata:
13      labels:
14       app: prometheus
15    spec:
16      containers:
17       - name: prometheus
18         image: prom/prometheus:v2.52.0 # 최신 버전 또는 원하는 버전 명시
19         args:
20           - "--config.file=/etc/prometheus/prometheus.yml"
21           - "--storage.tsdb.path=/prometheus"
22           - "--web.enable-lifecycle"
23         ports:
24           - containerPort: 9090
25         volumeMounts:
26           - name: config
27             mountPath: /etc/prometheus
28           - name: data
29             mountPath: /prometheus
30         volumes:
31           - name: config
32             configMap:
33               name: prometheus-config
34           - name: data
35             emptyDir: {} # PVC를 원하면 여기를 volumeClaimTemplates로 교체 가능
36
```

pro-deploy.yml

pro-deploy.yml은 Kubernetes에서 Prometheus를 테스트용으로 간단히 배포하기 위한 Deployment 리소스 정의로, default 네임스페이스에 prometheus라는 이름으로 1개의 파드를 생성한다. 컨테이너는 prom/prometheus:v2.52.0 이미지를 사용하며, 주요 실행

인자로 설정 파일 경로(prometheus.yml), 데이터 저장 경로(./prometheus), 라이프사이클 활성화 옵션 등이 포함되어 있다.

포트는 9090으로 열려 있으며, 설정 파일은 ConfigMap prometheus-config에서 마운트되고, 데이터 디렉터리는 임시 볼륨(emptyDir)으로 마운트된다.

```
monitor-k8s > prometheus > ! pro-configmap.yml > {} data > prometheus.yml
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: prometheus-config
5    namespace: default
6  data:
7    prometheus.yml: |
8      global:
9        scrape_interval: 30s
10
11      scrape_configs:
12        - job_name: 'cloudwatch-exporter'
13          static_configs:
14            - targets: ['cloudwatch-exporter.default.svc.cluster.local:9106']
15
16        - job_name: 'gcp-monitoring-exporter'
17          static_configs:
18            - targets: ['gcp-monitoring-exporter.default.svc.cluster.local:9255']
19
20        - job_name: 'node-exporter'
21          static_configs:
22            - targets: ['node-exporter.kube-system.svc.cluster.local:9100']
23
24        - job_name: 'kube-state-metrics'
25          static_configs:
26            - targets: ['kube-state-metrics.kube-system.svc.cluster.local:8080']
27
28
```

pro-configmap.yml

pro-configmap.yml은 Prometheus의 설정 파일을 담고 있는 Kubernetes ConfigMap 정의로, prometheus-config라는 이름으로 default 네임스페이스에 생성된다. 내부에는 prometheus.yml 구성 파일이 포함되어 있으며, 스크레이핑 주기는 30초(scrape_interval: 30s)로 설정되어 있다. Prometheus가 메트릭을 수집할 타겟은 총 4개로, 각각 다음과 같다:

cloudwatch-exporter: 포트 9106 gcp-monitoring-exporter: 포트 9255

node-exporter: 포트 9100 kube-state-metrics: 포트 8080

각 타겟은 클러스터 내 서비스 주소(.svc.cluster.local)를 사용하며, Prometheus가 해당 주소들로 접근해 주기적으로 메트릭을 수집하도록 구성되어 있다.

```
monitor-k8s > prometheus > ! pro-rbac.yml > [ ] subjects > {} 0
io.k8s.api.rbac.v1.ClusterRoleBinding (v1@clusterrolebinding.json) | io.k8s.api.rbac.v1.ClusterRole (v1@clusterrolebinding.json)
1  ∨ apiVersion: rbac.authorization.k8s.io/v1
2    kind: ClusterRole
3  ∨ metadata:
4    | name: prometheus-kubelet
5  ∨ rules:
6  ∨   - apiGroups: ["" ]
7    ∨   resources:
8      | - nodes/metrics
9        | - nodes/proxy
10       | - nodes/stats
11     verbs: ["get", "list", "watch"]
12   ---
13 ∨ apiVersion: rbac.authorization.k8s.io/v1
14   kind: ClusterRoleBinding
15 ∨ metadata:
16   | name: prometheus-kubelet
17 ∨ roleRef:
18   | apiGroup: rbac.authorization.k8s.io
19   | kind: ClusterRole
20   | name: prometheus-kubelet
21 ∨ subjects:
22 ∨   - kind: ServiceAccount
23     | name: prometheus
24     | namespace: default
25
```

pro-rbac.yml

pro-rbac.yml 은 Prometheus가 Kubernetes 노드의 메트릭 데이터를 수집할 수 있도록 권한을 부여하는 RBAC 설정 파일이다. 먼저 prometheus-kubelet이라는 이름의 ClusterRole을 정의하고, 이 역할은 nodes/metrics, nodes/proxy, nodes/stats 리소스에 대해 get, list, watch 권한을 가진다. 이는 Prometheus가 각 노드의 kubelet 메트릭을 가져올 수

있게 해준다. 이후 `ClusterRoleBinding`을 통해 해당 `ClusterRole`을 `default` 네임스페이스의 `prometheus` 서비스 계정에 바인딩하여, `Prometheus` 파드가 실제로 이 권한을 사용할 수 있도록 연결한다.

```
monitor-k8s > prometheus > ! pro-service.yml > {} spec
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: prometheus
5    namespace: default
6  spec:
7    selector:
8      app: prometheus
9    ports:
10     - protocol: TCP
11       port: 9090
12       targetPort: 9090
13     type: ClusterIP # 필요 시 NodePort로 변경 가능
14
```

pro-service.yml

`pro-service.yml`은 `Prometheus`에 대한 `Kubernetes Service` 리소스를 정의한 것으로, `default` 네임스페이스 내에서 `Prometheus` 파드에 접근할 수 있는 내부 서비스 엔드포인트를 생성한다. 서비스 이름은 `prometheus`이며, `app: prometheus` 레이블을 가진 파드를 대상으로 `TCP 9090` 포트를 매핑한다. `type`은 `ClusterIP`로 설정되어 있어, 클러스터 내부에서만 접근 가능하다.

Grafana

```
monitor-k8s > grafana > ! gra-deploy.yml > {} spec > {} template > {} spec > [ ] containers > {} 0
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: grafana
5    namespace: default
6  spec:
7    replicas: 1
8    selector:
9      matchLabels:
10       app: grafana
11    template:
12      metadata:
13        labels:
14         app: grafana
15      spec:
16        containers:
17          - name: grafana
18            image: grafana/grafana:10.4.1 # 원하는 버전 명시
19            ports:
20              - containerPort: 3000
21            volumeMounts:
22              - name: config
23                mountPath: /etc/grafana/provisioning/datasources
24          volumes:
25            - name: config
26              configMap:
27                name: grafana-datasources
28
```

gra-deploy.yml

gra-deploy.yml은 Kubernetes에서 Grafana를 배포하기 위한 Deployment 리소스 정의이다. default 네임스페이스에 grafana라는 이름으로 배포되며, 1개의 복제 파드(replica)를 생성한다. 컨테이너는 grafana/grafana:10.4.1 이미지를 사용하며, 이는 사용자가 명시한 특정 버전이다.

Grafana는 기본적으로 3000번 포트에서 서비스되므로 `containerPort: 3000`이 열려 있다. 또한, `/etc/grafana/provisioning/datasources` 경로에 ConfigMap을 마운트하여 Grafana가 시작 시 자동으로 데이터 소스를 읽어들이도록 구성되어 있다. 이때 마운트되는 ConfigMap의 이름은 `grafana-datasources`이다.

```
monitor-k8s > grafana > ! gra-configmap.yaml > {} data > prometheus-datasource.yaml
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: grafana-datasources
5    namespace: default
6  labels:
7    grafana_datasource: "1"
8  data:
9    prometheus-datasource.yaml: |
10     apiVersion: 1
11     datasources:
12     - name: Prometheus
13       type: prometheus
14       access: proxy
15       url: http://prometheus.default.svc.cluster.local:9090
16       isDefault: true
17
```

gra-configmap.yaml

`gra-configmap.yaml`은 Kubernetes의 `default` 네임스페이스에 생성되며, 이름은 `grafana-datasources`로 설정한다. 레이블로는 `grafana_datasource: "1"`을 부여하여 관리 용이성을 높인다. `data` 항목에는 `prometheus-datasource.yaml`이라는 키가 포함되어 있으며, 이 내부에는 Grafana에서 Prometheus를 데이터 소스로 설정하기 위한 YAML 형식의 설정값이 문자열로 정의되어 있다. 데이터 소스의 이름은 `Prometheus`로 지정하며, 타입은 `prometheus`, 접근 방식은 `proxy`로 설정한다.

Prometheus 서버의 주소는 `http://prometheus.default.svc.cluster.local:9090`으로, 클러스터 내부의 서비스 DNS를 통해 접근하도록 구성한다. 또한 `isDefault: true`로 설정하여 해당 데이터 소스를 Grafana의 기본 데이터 소스로 지정한다.

```
monitor-k8s > grafana > ! gra-service.yml > {} spec
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: grafana
5    namespace: default
6  spec:
7    selector:
8      app: grafana
9    ports:
10     - protocol: TCP
11       port: 80
12       targetPort: 3000
13   type: ClusterIP # NodePort로 바꾸면 외부 접속 가능
14
```

gra-service.yml

해당 `gra-service.yml` 파일은 Kubernetes에서 Grafana 서비스에 접근하기 위한 Service 리소스를 정의한 YAML이다. 주요 내용을 문장형(진술형)으로 설명하면 다음과 같다: 이 YAML은 `default` 네임스페이스에 `grafana`라는 이름으로 Service 리소스를 생성한다. 이 서비스는 `app: grafana`라는 라벨을 가진 파드를 대상으로 트래픽을 전달하도록 설정한다. 서비스 포트는 클러스터 내부에서 접근 시 80번 포트를 사용하며, 실제 Grafana 파드의 컨테이너는 3000번 포트에서 요청을 수신하므로 `targetPort`를 3000으로 설정한다. `type: ClusterIP`로 정의되어 있어, 이 서비스는 클러스터 내부에서만 접근 가능하다.

10. 향후 개선 방향

본 프로젝트는 **AWS** 서울 리전을 주요 운영 환경으로, **GCP** 미국 리전을 재해 복구 환경으로 활용한 멀티 클라우드·멀티 리전 아키텍처를 구축하였다. **Terraform**을 활용한 **IaC(Infrastructure as Code)** 방식으로 인프라를 코드화하여 효율적인 관리와 확장성을 확보했으며, **Aurora RDS** 대신 스토리지 기반 덤프 전송을 이용한 멀티리전 **DB** 연동을 구현해 비용 절감과 안정적인 데이터 동기화를 달성하였다.

또한, 쿠버네티스 기반 컨테이너 환경을 통해 고가용성과 빠른 배포 환경을 마련하고, **Prometheus**와 **Grafana** 기반 모니터링 시스템으로 애플리케이션 성능, 자원 사용량, 트래픽을 실시간으로 수집·시각화하여 서비스 운영 효율성을 높였다.

10-1 향후 개선 방향

- AI 기반 개인화 매칭 고도화

- 딥러닝 기반 이미지 분석 모델(**CNN**)을 적용해 프로필 사진에서 사용자의 성향을 추정하고, 사용자 행동 데이터(클릭, 대화 패턴, 매칭 유지율 등)를 기반으로 협업 필터링(**Collaborative Filtering**)과 강화학습(**Reinforcement Learning**)을 활용한 추천 시스템을 구축한다.

이를 통해 단순 조건 기반 매칭을 넘어, 개인의 가치관·성향·행동 데이터를 종합적으로 반영한 ‘진정성 있는 관계 매칭’을 실현할 예정이다.

- 프로필 추천 및 실시간 상호작용 기능 강화

- **Graph** 기반 추천 알고리즘을 도입해 유사 성향 사용자 네트워크를 탐지하고, 추천 결과를 동적으로 갱신한다.
- **WebSocket** 기반 실시간 채팅의 알림 기능을 확장하여, 사용자가 선호하는 상대와 지속적으로 상호작용할 수 있는 환경을 제공한다.

- 모니터링 및 운영 자동화 확대

- Prometheus 지표에 머신러닝 기반 이상 탐지 모델을 추가하여 비정상 트래픽 패턴이나 성능 저하를 조기 감지한다.

이러한 개선을 통해 **Lovebridge**는 단순한 데이팅 서비스에서 벗어나, 사용자의 성향과 행동 데이터를 기반으로 진정성 있는 관계 형성을 지원하는 차세대 매칭 플랫폼으로 성장할 수 있을 것으로 기대된다.